



**ҚР ОҚУ-АҒАРТУ МИНИСТРЛІГІ
МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РК**

**АЛМАТЫ ҚАРЖЫ-ЭКОНОМИКАЛЫҚ КОЛЛЕДЖІ
АЛМАТИНСКИЙ ФИНАНСОВО-ЭКОНОМИЧЕСКИЙ
КОЛЛЕДЖ**

«Қорғауға рұқсат»
Колледж директоры
_____ Алимова К.У.
«____» _____ 2026 ж.
хаттама № _____

ДИПЛОМДЫҚ ЖҰМЫС

Тақырыбы «Іс-шаралар мен конференцияларды тіркеу және басқаруға арналған
веб-жүйе»

Мамандық	06130100 Бағдарламалық қамтамасыз ету (түрлері бойынша)
Біліктілік	4S06130105 Ақпараттық жүйелер технигі

Орындады:		Жанұзақ Ә.Т.
Ғылыми жетекші:		Тұрғантай А.Ә.

Алматы, 2026

АНДАТПА

Бұл дипломдық жұмыста іс-шараларға қатысушыларды тіркеу, конференц-залдарды брондау, өтінімдерді өңдеу және әкімшілік бақылауды автоматтандыруға арналған веб-жүйені жобалау мен іске асыру мәселелері қарастырылды. Жоба HTML, CSS, JavaScript, Node.js, Express.js, MongoDB Atlas, REST API және JWT авторизациясы негізінде әзірленді. Жүйеде көптілді интерфейс, пайдаланушыны тіркеу, іс-шаралар каталогы, залдарды брондау, күнтізбе, хабарламалар, техникалық қолдау және әкімшілік панель модульдері қамтылды.

АННОТАЦИЯ

В дипломной работе рассмотрены проектирование и реализация веб-системы для регистрации участников мероприятий, управления конференциями и бронирования конференц-залов. Система разработана на основе HTML, CSS, JavaScript, Node.js, Express.js, MongoDB Atlas, REST API и JWT-авторизации. В работе раскрыты архитектура клиент-серверного взаимодействия, структура базы данных, многоязычный интерфейс, каталог мероприятий, календарь, система заявок, уведомления, техническая поддержка и административная панель.

ABSTRACT

This diploma thesis describes the design and implementation of a web system for event registration, conference management and conference hall booking. The project is built with HTML, CSS, JavaScript, Node.js, Express.js, MongoDB Atlas, REST API and JWT authorization. The thesis covers client-server architecture, database models, multilingual interface, event catalog, booking workflow, calendar, request management, notifications, technical support and administrative dashboard.

МАЗМҰНЫ

КІРІСПЕ	4
1 ІС-ШАРАЛАР ВЕБ-ЖҮЙЕСІНІҢ НЕГІЗДЕРІ.....	7
1.1 Цифрландыру және іс-шараларды ұйымдастыру	7
1.2 Қатысушы тіркеуі және зал брондау.....	9
1.3 Ұқсас платформалар және ORDA орны	11
1.4 Технологиялық стек және қауіпсіздік.....	13
2 ORDA SMART EVENT SYSTEM ЖҮЙЕСІН ЖОБАЛАУ	16
2.1 Талаптар және пайдаланушы сценарийлері	16
2.2 Клиент-сервер архитектурасы және деректер ағыны	18
2.3 Деректер қоры және API логикасы	20
2.4 Интерфейс, көптілділік және стиль.....	23
3 ЖҮЙЕНІ ІСКЕ АСЫРУ ЖӘНЕ ТЕСТІЛЕУ	26
3.1 Клиенттік модульдерді іске асыру	26
3.2 Серверлік бөлік және әкімші модулі.....	30
3.3 Тестілеу және тәуекелдер.....	33
3.4 Практикалық және экономикалық тиімділік.....	35
4 ЕҢБЕКТІ ҚОРҒАУ	38
4.1 Компьютермен жұмыс істеу қауіпсіздігі.....	38
4.2 Жұмыс орнын ұйымдастыру талаптары.....	38
4.3 Ақпараттық қауіпсіздік және деректерді қорғау	39
5 ЭКОНОМИКАЛЫҚ БӨЛІМ	41
5.1 Жобаның экономикалық және әлеуметтік мәні.....	41
5.2 Жобаны әзірлеу шығындарын есептеу	41
5.3 Жобаның тиімділігі және даму мүмкіндіктері.....	42
ҚОРЫТЫНДЫ	43
ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР	44
ҚОСЫМША А. БАҒДАРЛАМАНЫҢ НЕГІЗГІ КОДТАРЫ.....	45

КІРІСПЕ

Қазіргі уақытта іс-шараларды ұйымдастыру саласы қарқынды түрде цифрлануда. Конференциялар, семинарлар, оқу тренингтері, бизнес-форумдар және гибриді форматтағы кездесулер қатысушылардың алдын ала тіркелуін, орын санын бақылауды, залдардың бос уақытын тексеруді, төлем және өтінім мәртебесін қадағалауды талап етеді. Егер бұл әрекеттер қолмен немесе бөлек кестелер арқылы жүргізілсе, ұйымдастырушы уақыт жоғалтады, деректер қайталанады және қатысушыға берілетін кері байланыс баяулайды.

ORDA Smart Event System жобасы осы мәселені шешуге бағытталған. Жүйе веб-платформа ретінде жасалып, бір интерфейсте іс-шаралар каталогын, қатысушыларды тіркеуді, конференц-залдарды брондауды, күнтізбелік бақылауды, әкімші панелін және техникалық қолдау хабарламаларын біріктіреді. Платформада пайдаланушы мен әкімші рөлдері бөлінген, ал деректер MongoDB базасында сақталады.

Дипломдық жұмыстың мақсаты - іс-шараларға тіркеу және конференц-залдарды басқару процесін автоматтандыратын ORDA Smart Event System веб-жүйесінің теориялық негіздерін сипаттау, архитектурасын жобалау және практикалық іске асырылуын негіздеу.

Осы мақсатқа жету үшін төмендегідей міндеттер қойылды:

- іс-шараларды ұйымдастыру мен брондау процестерін цифрландырудың маңызын талдау;
- пайдаланушы, қатысушы және әкімші сценарийлерін анықтау;
- жүйенің функционалдық және функционалдық емес талаптарын қалыптастыру;
- клиент-сервер архитектурасын, REST API және деректер қоры модельдерін жобалау;
- Node.js, Express.js және MongoDB негізінде серверлік бөлікті іске асыру;
- HTML, CSS және JavaScript негізінде көптілді веб-интерфейс әзірлеу;
- брондау, өтінім мәртебесі, төлем, күнтізбе және техникалық қолдау модульдерін сипаттау;
- жүйені функционалдық тексеруден өткізіп, практикалық тиімділігін бағалау;

Зерттеу нысаны - іс-шараларды жариялау, қатысушыларды тіркеу және конференц-залдарды брондау процесі. Зерттеу пәні - осы процесті автоматтандыруға арналған веб-жүйенің функционалдық, архитектуралық және интерфейстік шешімдері.

Зерттеу әдістері ретінде салыстырмалы талдау, жүйелік жобалау, UML элементтеріне негізделген сценарийлік талдау, деректер моделін құру, REST API жобалау, бағдарламалық іске асыру және функционалдық тестілеу әдістері қолданылды.

Жұмыстың практикалық маңызы - ORDA жүйесін оқу орындарында, конференц-залдарда, бизнес орталықтарында, курстар мен семинарлар өткізетін ұйымдарда қолдануға болады. Жүйе ұйымдастырушыға іс-шараны жариялау,

орын санын бақылау, қатысушы өтінімдерін өңдеу және залдардың бос уақытын бір ортадан басқаруға мүмкіндік береді.

Дипломдық жұмыс кіріспеден, үш негізгі бөлімнен, қорытындыдан, пайдаланылған әдебиеттер тізімінен және қосымшалардан тұрады. Бірінші бөлімде тақырыптың теориялық негіздері қарастырылды. Екінші бөлімде жүйенің жобалық шешімдері берілді. Үшінші бөлімде бағдарламалық іске асыру, тестілеу және тиімділік нәтижелері сипатталды.

Тақырыптың өзектілігі қазіргі ұйымдардың ақпаратпен жұмыс істеу тәсілінің өзгеруімен тікелей байланысты. Бұрын іс-шараны өткізу үшін қатысушылардың тізімі, залдың бос уақыты, техникалық жабдық, төлем туралы ақпарат және кері байланыс бөлек құжаттарда немесе мессенджер хаттарында жүргізілсе, қазіргі веб-жүйелер бұл деректерді бір ортаға жинауға мүмкіндік береді. Мұндай тәсіл ұйымдастырушының жұмысын жеңілдетіп қана қоймай, шешім қабылдау сапасын да арттырады, себебі әкімші нақты уақытқа жақын деректерге сүйенеді.

Конференция залдары мен іс-шараларды басқару саласында автоматтандырудың маңызы ерекше. Бір зал бірнеше топқа қажет болуы мүмкін, бір күнде бірнеше кездесу жоспарлануы ықтимал, ал қатысушылар саны зал сыйымдылығынан асып кетпеуі тиіс. Егер осы шарттар қолмен тексерілсе, адами фактордың әсері артады. ORDA Smart Event System жобасы дәл осындай қайталанатын операцияларды жүйелік түрде орындауға бағытталған: пайдаланушы өтінім береді, сервер деректерді тексереді, базаға сақтайды, ал әкімші мәртебені өзгертіп, процесті бақылайды.

Жұмыстың ғылыми-тәжірибелік маңыздылығы веб-қосымшаны тек техникалық өнім ретінде емес, нақты басқару процесін оңтайландыратын ақпараттық жүйе ретінде қарастыруында. Мұнда бағдарламалық код, интерфейс, деректер қоры және пайдаланушы сценарийлері бір-бірінен бөлек емес, біртұтас шешім ретінде талданады. Сондықтан дипломдық жұмыста тек қолданылған технологиялар аталып қана қоймай, олардың не үшін таңдалғаны, қандай міндетті шешетіні және жүйенің болашақтағы дамуына қалай әсер ететіні түсіндіріледі.

Зерттеу барысында іс-шараларды басқару платформаларының негізгі даму үрдістері де ескерілді. Қазіргі сервистер пайдаланушыға жылдам тіркелу, мобильді құрылғыдан қолжетімділік, онлайн және офлайн форматтарды қатар қолдау, жеке кабинет, төлем мәртебесі, автоматты хабарламалар және аналитикалық есептер ұсынуға тырысады. ORDA жобасы осы үрдістерді дипломдық жоба деңгейінде бейімдеп, жергілікті ұйымға түсінікті әрі басқаруға ыңғайлы форматта іске асырады.

Жүйені әзірлеу кезінде функционалдық талаптармен қатар функционалдық емес талаптар да қарастырылды. Оларға қауіпсіздік, жауап беру жылдамдығы, интерфейснің ыңғайлылығы, код құрылымының түсініктілігі, деректердің сақталу тұрақтылығы, болашақта кеңейту мүмкіндігі және әртүрлі экран өлшемдеріне бейімделу жатады. Бұл талаптар жобаның сапасын тек экранда көрінетін нәтиже арқылы емес, оның ішкі архитектурасы арқылы да бағалауға мүмкіндік береді.

Кіріспеде белгіленген мақсат пен міндеттер дипломдық жұмыстың барлық бөлімдерімен байланысады. Бірінші бөлімде мәселенің теориялық негізі ашылып, ұқсас платформалармен салыстыру жүргізіледі. Екінші бөлімде ORDA жүйесінің құрылымы, деректер ағыны және API логикасы сипатталады. Үшінші бөлімде нақты іске асыру, модульдердің жұмысы, тестілеу нәтижелері, тәуекелдер және практикалық тиімділік талданады. Осылайша жұмыс теориядан практикалық нәтижеге дейінгі толық циклді қамтиды.

1 ІС-ШАРАЛАР ВЕБ-ЖҮЙЕСІНІҢ НЕГІЗДЕРІ

1.1 Цифрландыру және іс-шараларды ұйымдастыру

Іс-шараларды ұйымдастыру - бірнеше өзара байланысты әрекеттерден тұратын процесс. Оған бағдарлама жасау, қатысушыларды шақыру, тіркеу, орын санын шектеу, залды таңдау, техникалық жабдықты дайындау, төлемді белгілеу, қатысу мәртебесін тексеру және іс-шара аяқталғаннан кейін статистика жинау кіреді. Дәстүрлі тәсілде бұл әрекеттер телефон, электрондық пошта, қағаз тізім немесе жеке кестелер арқылы орындалады.

Мұндай тәсіл шағын кездесулер үшін жеткілікті болуы мүмкін, бірақ қатысушылар саны өскен кезде қателік ықтималдығы артады. Бір қатысушы бірнеше рет тіркелуі, бос орын саны дұрыс есептелмеуі, зал басқа іс-шарамен қатар брондалып кетуі немесе әкімшіге өтінім мәртебесін қолмен жаңарту қажет болуы мүмкін. Сондықтан іс-шара мен залды басқару жүйесі тек ақпараттық сайт емес, бизнес-процесті автоматтандыратын құрал болуы тиіс.

Цифрландырудың негізгі артықшылығы - деректердің бір орталықта сақталуы. Қатысушы тіркелген кезде оның ақпараты бірден деректер қорына түседі, әкімші өтінімді көреді, жүйе орын санын жаңартады және күнтізбе жүктемені көрсетеді. Бұл ұйымдастырушының жұмысын жеңілдетіп қана қоймай, пайдаланушыға да сенімді тәжірибе береді.

ORDA Smart Event System осы логикаға сүйенеді. Жүйеде іс-шара, зал, пайдаланушы, өтінім және техникалық қолдау хабарламасы жеке деректер моделі ретінде қарастырылған. Мұндай бөлу бағдарламаны кеңейтуге және әр модульді тәуелсіз жетілдіруге мүмкіндік береді.

Іс-шараларды басқару процесін жүйелік тұрғыдан қарастырғанда, оның бірнеше кезеңнен тұратынын көруге болады. Бірінші кезеңде ұйымдастырушы іс-шара идеясын, тақырыбын, өткізу форматын, уақытын және орнын анықтайды. Екінші кезеңде қатысушыларды шақыру және тіркеу жүргізіледі. Үшінші кезеңде зал, жабдық, төлем, техникалық қолдау және қатысу мәртебесі бақыланады. Соңғы кезеңде статистика жиналып, келесі іс-шараларға арналған тәжірибе қалыптасады.

Цифрлық жүйе осы кезеңдердің әрқайсысында деректердің үздіксіз қозғалысын қамтамасыз етеді. Мысалы, іс-шара жарияланған сәтте ол бірден каталогта көрінеді, қатысушы тіркелгенде бос орын саны өзгереді, әкімші панелінде жаңа өтінім пайда болады, ал жеке кабинетте пайдаланушы өзінің қатысу мәртебесін бақылай алады. Мұндай байланыс деректерді қайта-қайта енгізу қажеттілігін азайтады және бір ақпараттың бірнеше жерде әртүрлі сақталу қаупін төмендетеді.

Іс-шара саласындағы цифрландырудың тағы бір маңызды бағыты - гибриді форматтарды қолдау. Қазіргі конференциялар тек залда өтпейді: кейбір қатысушылар онлайн қосылады, кейбірі офлайн қатысады, ал ұйымдастырушыға екі форматтың да деректерін басқару қажет болады. Сондықтан жүйеде offline, online және hybrid форматтарының болуы интерфейс үшін ғана емес, деректер моделін дұрыс жобалау үшін де маңызды. Әр форматтың өзіндік өрістері мен тексеру шарттары болуы мүмкін.

Дәстүрлі кестелік есепке алу мен веб-жүйенің айырмашылығы деректердің белсенді өңделуінде. Кестеде ақпарат сақталады, бірақ ол автоматты түрде қақтығысты анықтамайды, пайдаланушы рөлін тексермейді және жеке кабинетке нәтижені жібермейді. Ал веб-жүйеде деректер тек сақталмайды, олар бизнес-логика арқылы тексеріледі. ORDA жүйесінде бір пайдаланушының бір іс-шараға қайталанып жазылмауы, зал уақытының қақтығыспауы және мәртебелердің бақылануы осыған мысал бола алады.

Ұйымдастыру процесін автоматтандырудың әлеуметтік әсері де бар. Қатысушы үшін жүйе сенімділік сезімін береді: ол өтінімінің қабылданғанын, төлемнің қандай күйде екенін және қандай іс-шараларға жазылғанын көре алады. Әкімші үшін жүйе бақылауды күшейтеді: барлық өтінім бір жерде жиналады, статистика тез табылады, ал техникалық қолдау хабарламалары жоғалып кетпейді. Осылайша цифрландыру тек техникалық жаңарту емес, қызмет көрсету мәдениетін жақсарту құралы болып табылады.

ORDA жобасының теориялық негізінде ақпараттық жүйелерді модульге бөлу қағидасы жатыр. Іс-шара, зал, пайдаланушы, брондау және қолдау хабарламасы жеке объект ретінде қарастырылғандықтан, әр объектінің өмірлік циклі бөлек анықталады. Мысалы, іс-шара draft, published немесе closed күйінде болуы мүмкін, ал брондау new, review, pending, approved, rejected, cancelled немесе registered мәртебесіне өтеді. Мұндай модель нақты процесті деректер деңгейінде бейнелейді.

Іс-шараларды цифрландыру тек электрондық тіркеу формасын енгізумен шектелмейді. Бұл процесс қатысушылардың деректерін жинау, зал ресурстарын бөлу, уақыт кестесін бақылау, ұйымдастырушылардың шешім қабылдауын жеңілдету және барлық әрекетті бір ақпараттық кеңістікке біріктіруді қамтиды. Сондықтан веб-жүйе нақты ұйымдағы жұмыс тәртібін модельдейтін басқару құралы ретінде қарастырылады.

Дәстүрлі тәсілде іс-шараға қатысушылар тізімі кесте файлдарында, зал кестесі бөлек журналда, ал өтінім мәртебесі жеке хабарламаларда сақталуы мүмкін. Мұндай ортада ақпараттың қайталануы, бір өтінімнің екі рет өңделуі немесе бос емес залдың қайта брондалуы сияқты тәуекелдер пайда болады. Веб-жүйе бұл тәуекелдерді деректерді бір базада сақтау арқылы азайтады.

Цифрлық жүйенің артықшылығы пайдаланушы әрекеттерінің ізін сақтауында. Әрбір тіркеу, брондау, мәртебе өзгерісі және техникалық қолдау хабарламасы уақыт белгісімен деректер қорына жазылады. Бұл ұйымдастырушыға процестің қай кезеңде тұрғанын көруге, ал пайдаланушыға өз өтінімінің нәтижесін жеке кабинеттен бақылауға мүмкіндік береді.

Іс-шараларды ұйымдастыруда жылдамдық пен сенімділік қатар маңызды. Қатысушы іс-шара туралы ақпаратты бірден көріп, бос орын бар-жоғын анықтап, тіркелуі қажет. Әкімші бұл өтінімді бөлек файлан іздемей, бір панельде көріп, қажет болса мәртебесін өзгертеді. Осылайша цифрландыру қызмет көрсету уақытын қысқартады.

ORDA Smart Event System жобасы осы теориялық талаптарды практикалық деңгейде іске асыруға бағытталған. Жүйеде каталог, күнтізбе, конференц-залдар тізімі, өтінім мәртебесі, техникалық қолдау және әкімшілік

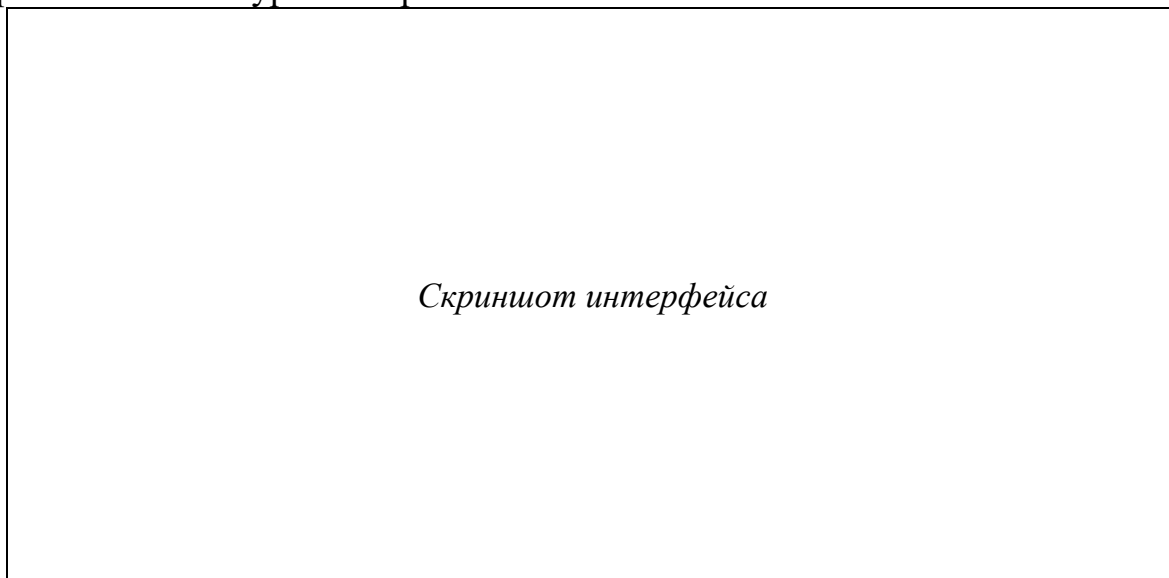
бақылау бір интерфейске біріктірілген. Мұндай тәсіл дипломдық жобаның қолданбалы маңызын арттырады.

1.2 Қатысушы тіркеуі және зал брондау

Қатысушы үшін веб-жүйенің басты құндылығы - әрекеттің түсінікті және жылдам орындалуы. Пайдаланушы сайтқа кіріп, қажетті іс-шараны іздейді, категория немесе формат бойынша сүзгіден өткізеді, толық ақпаратты қарайды және бір батырма арқылы тіркеледі. Егер тіркелу үшін күрделі форма немесе түсініксіз навигация қажет болса, жүйенің пайдалану тиімділігі төмендейді.

Залдарды брондау сценарийі күрделірек, себебі мұнда күн, уақыт, ұзақтық, қатысушылар саны, қосымша қызметтер, жабдық және төлем мәртебесі ескеріледі. Сонымен қатар жүйе бір залдың бір уақытта екі өтінімге бекітілуін болдырмауы керек. Сондықтан брондау модулінде қолжетімділік тексерісі және уақыт аралықтарының қақтығысын анықтау алгоритмі маңызды орын алады.

Пайдаланушы іс-шара туралы толық мәліметті detail бетінен көреді. Іс-шара detail беті 1-суретте көрсетілген.



Сурет 1. Іс-шара detail бетінің интерфейсі

Пайдаланушы тәжірибесі тек интерфейс әдемілігімен өлшенбейді. Ол ақпараттың табылуы, батырмалардың болжамдылығы, қате хабарламаларының түсініктілігі, тіл таңдауы, мобильді экранда ыңғайлы көрінуі және жүйенің жауап беру жылдамдығы сияқты факторлардан тұрады. ORDA жобасында осы себепті бүйірлік навигация, іздеу, фильтрлер, көптілді мәтіндер, жарық/қараңғы тақырып және жеке кабинет қарастырылған.

Пайдаланушы тәжірибесін жобалауда әрекеттердің болжамдылығы маңызды орын алады. Егер пайдаланушы іс-шара карточкасында атауды, күнді, форматты, орынды, бағаны және бос орын санын бірден көрсе, ол шешімді тез қабылдайды. Керісінше, маңызды ақпарат бірнеше терезеге бөлініп кетсе немесе батырманың мағынасы түсініксіз болса, тіркелу ықтималдығы төмендейді. Сондықтан ORDA интерфейсында карточка, фильтр, іздеу және action батырмалары бір сценарийге біріктірілген.

Зал брондау кезінде пайдаланушы тәжірибесі ақпараттың толықтығымен байланысты. Қатысушы саны, жабдық, уақыт, мақсат, төлем әдісі және қосымша қызметтер нақты көрсетілуі керек. Бұл деректер кейін әкімші шешіміне әсер етеді: залдың сыйымдылығы жеткілікті ме, сұралған уақыт бос па, қызметтер қолжетімді ме, төлем мәртебесі қандай. Осы себепті брондау формасы қарапайым көрінгенімен, оның артында толық бизнес-процесс орналасқан.

Жүйеде қате жағдайларын түсінікті көрсету де UX/UI сапасының бір бөлігі болып табылады. Мысалы, пайдаланушы авторизациясыз қорғалған бөлімге кірсе, жүйе оған қайта кіру қажеттігін көрсетуі тиіс. Егер зал уақыты бос болмаса, қате хабарламасы жалпы техникалық мәтін емес, нақты себепті түсіндіруі қажет. Мұндай хабарламалар пайдаланушыны жүйеден алыстатпайды, керісінше келесі әрекетті түсінуге көмектеседі.

Адаптивті интерфейстің рөлі де жоғары. Іс-шараға қатысушы сайтқа көбіне телефоннан кіруі мүмкін: ол жолда келе жатып тіркеледі, зал туралы ақпаратты қарайды немесе жеке кабинеттен мәртебені тексереді. Сондықтан интерфейс тек үлкен мониторға емес, мобильді экранға да бейімделуі тиіс. CSS арқылы grid, flex, responsive spacing және ыңғайлы батырма өлшемдері қолданылса, жүйенің қолданылу аясы кеңейеді.

Көптілділік пайдаланушы тәжірибесін кеңейтетін маңызды факторлардың бірі. Қазақстан жағдайында қазақ, орыс және ағылшын тілдеріндегі интерфейс әртүрлі аудиторияға қолжетімділік береді. ORDA жүйесінде мәтіндерді `data-i18n` атрибуттары арқылы бөлу бір HTML құрылымын сақтап, тілдік мазмұнды бөлек басқаруға мүмкіндік береді. Бұл тәсіл кейін жаңа тіл қосу немесе терминдерді түзету кезінде интерфейс кодын толық қайта жазбауға жағдай жасайды.

UX талдауында әкімші тәжірибесін де бөлек бағалау қажет. Әкімші үшін интерфейс әдемі ғана емес, жұмысқа ыңғайлы болуы керек: өтінімдерді жылдам сүзу, мәртебені өзгерту, статистиканы көру, жаңа іс-шара немесе зал қосу, техникалық хабарламаларды бақылау сияқты әрекеттер жиі қайталанады. Сондықтан admin dashboard бөлімінде ақпараттың жинақы орналасуы және басқару батырмаларының айқын болуы жүйенің практикалық құндылығын арттырады.

Қатысушыны тіркеу сценарийі жүйенің негізгі бизнес-процесінің бірі болып табылады. Пайдаланушы алдымен іс-шаралар каталогынан қажетті жазбаны таңдайды, содан кейін detail бетінде күнін, уақытын, форматын, мекенжайын, бағасын және бос орын санын көреді. Егер іс-шара қолжетімді болса, тіркелу батырмасы арқылы өтінім жасалады.

Залды брондау сценарийі қатысушы тіркеуінен күрделірек. Мұнда тек пайдаланушы деректері ғана емес, залдың сыйымдылығы, жабдықтары, бағасы, күн мен уақыт аралығы, қосымша қызметтер және өтінім мақсаты ескеріледі. Сервер жаңа өтінім қабылдас бұрын уақыт аралықтарының қабаттасуын тексереді.

Пайдаланушы тәжірибесі қарапайым және түсінікті болуы керек. Егер интерфейс күрделі болса, пайдаланушы қажетті іс-шараны табуға немесе зал

брондауға көп уақыт жұмсайды. Сондықтан іздеу өрісі, категория сүзгісі, формат таңдауы, календарь және жеке кабинет элементтері бір-бірімен логикалық байланыста орналасады.

Жүйеде өтінім мәртебесінің көрінуі ерекше маңызды. Қолмен ұйымдастырылатын процесте пайдаланушы өтінімі қабылданды ма, қаралуда ма, әлде қабылданбады ма деген ақпаратты бөлек сұрауы мүмкін. ORDA жүйесінде бұл ақпарат жеке кабинет пен хабарлама модулі арқылы беріледі.

Пайдаланушыға көрсетілетін әрбір экран нақты міндетті орындауға бағытталған: каталог таңдау үшін, detail бет шешім қабылдау үшін, форма өтінім жіберу үшін, жеке кабинет бақылау үшін, ал хабарлама аймағы кері байланыс үшін қолданылады. Осы құрылым веб-жүйені түсінікті және кәсіби етеді.

1.3 Ұқсас платформалар және ORDA орны

Нарықта іс-шараларды басқаруға арналған түрлі платформалар бар. Олардың ішінде халықаралық деңгейде Eventbrite және Cvent, ал ТМД аумағында TimePad сияқты сервистер белгілі. Бұл платформалар қатысушы тіркеу, билет сату, іс-шара беттерін жариялау және аналитика мүмкіндіктерін ұсынады. Алайда оқу орны немесе жергілікті конференц-зал үшін олардың кейбірі тым күрделі, кейбірі ақылы, ал кейбірінде залды ішкі кестемен басқару сценарийі шектеулі болуы мүмкін.

ORDA жүйесінің ерекшелігі - ол дипломдық жоба ретінде нақты жергілікті сценарийге бейімделген. Платформада іс-шараға қатысу және залды брондау бір жүйеде біріктірілген. Сонымен қатар әкімші клиенттік өтінімді қарап, бекіте алады, ал бекітілген жеке іс-шара автоматты түрде каталогқа жариялануы мүмкін.

Eventbrite, Cvent және TimePad сияқты платформалар ауқымды нарыққа бағытталғандықтан, олардың функционалы кең және кейде күрделі болады. Eventbrite көбіне іс-шара жариялау, билет сату және қатысушы аудиториясын тарту міндеттеріне ыңғайлы. Cvent корпоративтік деңгейдегі конференция менеджментіне, қонақ үй және ірі event-инфрақұрылыммен интеграцияға бейім. TimePad тіркеу мен билет таратуда ыңғайлы, бірақ жергілікті зал ресурстарын ұйым ішіндегі кестемен басқару барлық жағдайда негізгі сценарий бола бермейді.

ORDA жүйесін салыстырғанда негізгі критерийлер ретінде функционалдық толықтық, жергілікті бейімделу, әкімшіге түсініктілік, техникалық тәуелсіздік, кеңейту мүмкіндігі және оқу жобасына сәйкестік алынды. Коммерциялық платформалар дайын сервис ретінде күшті, бірақ оларды ұйымның ішкі процесіне толық бейімдеу қиын болуы мүмкін. Ал дипломдық жоба ретінде жасалған ORDA нақты талаптарға қарай өзгертіледі, код құрылымы ашық, деректер моделі кеңейтуге мүмкіндік береді.

Ұқсас платформаларды талдау ORDA жобасының артықшылығы мен шектеулерін қатар көруге мүмкіндік береді. Артықшылықтарына бір ортада іс-шара мен зал брондауын біріктіру, қарапайым әкімші панелі, көптілді интерфейс, MongoDB арқылы икемді дерек сақтау және нақты оқу процесіне

бейімделу жатады. Шектеулеріне толық төлем шлюзінің, email/SMS хабарламаларының және күрделі аналитиканың әзірше демонстрациялық деңгейде болуы кіреді. Бұл шектеулер жобаның болашақ даму жоспарын анықтайды.

Салыстырмалы талдаудың нәтижесінде ORDA жүйесінің нарықтағы орны шағын және орта ұйымдарға арналған бастапқы автоматтандыру платформасы ретінде анықталады. Ол ірі коммерциялық сервистермен тікелей бәсекелесуді мақсат етпейді, бірақ жергілікті конференц-зал, колледж, оқу орталығы немесе бизнес кеңістігі үшін негізгі процестерді цифрландыруға жеткілікті. Осы тұрғыдан жоба практикалық қолдануға жақын және оқу жұмысы ретінде нақты нәтижеге ие.

ORDA платформасының тағы бір ерекшелігі - жүйені өз инфрақұрылымында орналастыру мүмкіндігі. Коммерциялық сервистерде деректер көбіне сыртқы провайдердің ережелеріне тәуелді болады, ал жеке жасалған жүйеде деректер базасын, сервер ортасын, доменді және қауіпсіздік параметрлерін ұйым өзі бақылай алады. Бұл әсіресе оқу орындары мен шағын ұйымдар үшін маңызды, себебі қатысушылардың жеке деректері мен ішкі кестелерін басқару жауапкершілігі нақты анықталуы тиіс.

Осы талдау ORDA жүйесін дамытуда қандай бағыттарға көңіл бөлу керектігін көрсетеді. Біріншіден, тіркеу мен брондау логикасы тұрақты болуы керек. Екіншіден, әкімшіге арналған статистика мен экспорт мүмкіндіктері күшейтілуі қажет. Үшіншіден, қауіпсіздік пен қолжетімділік талаптары жүйенің әр қабатында ескерілуі тиіс. Төртіншіден, интерфейс қолданушыны шатастырмай, бірнеше рөлдің қажеттілігін бір ортада қамтуы керек.

Ұқсас платформаларды қарастырғанда олардың көпшілігі іс-шара жариялау, билет сату немесе қатысушы тіркеу функциясына басымдық беретіні байқалады. Алайда конференц-залдарды брондау, техникалық қолдау, admin бақылауы және көптілді интерфейсті бір дипломдық жобада біріктіру сирек кездеседі. ORDA жүйесінің ерекшелігі осы модульдердің бір жұмыс процесінде байланысуында.

Event management жүйелерінде пайдаланушы көбіне іс-шараны көріп, тіркелумен ғана шектеледі. Ал ұйым ішінде зал ресурсын басқару бөлек құрал арқылы жүргізіледі. Бұл екі процестің бөлінуі әкімші үшін артық жұмыс тудырады. ORDA жобасы іс-шара мен зал брондау логикасын бір Booking моделінің түрлері арқылы біріктіреді.

Конференц-орталықтарда залдың бос уақытын тексеру маңызды. Егер жүйе уақыт аралықтарын автоматты тексермесе, бір залға бірнеше өтінім түсуі мүмкін. ORDA жобасында availability логикасы осы мәселені шешуге бағытталған және нақты ұйымда қолдануға болатын практикалық шешім ұсынады.

Көптілділік ұқсас жүйелермен салыстырғанда маңызды артықшылық болып табылады. Қазақстан жағдайында қазақ, орыс және ағылшын тілдерін қолдау пайдаланушылардың әртүрлі топтарына ыңғайлы орта қалыптастырады. Интерфейс мәтіндерінің 18n құрылымында сақталуы жүйені кейін кеңейтуді жеңілдетеді.

Осы салыстырмалы талдау ORDA Smart Event System жобасының оқу мақсатынан бөлек нақты қолдануға жақын екенін көрсетеді. Жүйе іс-шаралар, залдар, өтінімдер, пайдаланушылар және қолдау хабарламалары арасындағы байланысты бір архитектураға жинақтайды.

1.4 Технологиялық стек және қауіпсіздік

ORDA жобасы классикалық веб-архитектура бойынша құрылған: клиенттік бөлік HTML, CSS және JavaScript арқылы жасалады, ал серверлік бөлік Node.js ортасында Express.js фреймворкімен жұмыс істейді. Деректер MongoDB құжаттық базасында сақталып, Mongoose кітапханасы арқылы модельдермен байланысады.

Қауіпсіздік тұрғысынан жүйеде пайдаланушы құпиясөзі bcryptjs арқылы хэштеледі, ал жүйеге кіргеннен кейін JWT токени беріледі. Токен пайдаланушының идентификаторын, рөлін, email мекенжайын және атын қамтиды. Сервердегі middleware қабаты қорғалған маршруттарға тек авторизацияланған пайдаланушыны, ал әкімші маршруттарына тек admin рөлін өткізеді.

HTML, CSS және JavaScript клиенттік қабаттың негізгі технологиялары ретінде таңдалды. HTML құжат құрылымын, бөлімдер мен формаларды анықтайды; CSS визуалды стиль, орналасу, түстер, адаптивті торлар және тақырып ауыстыру логикасын қамтамасыз етеді; JavaScript пайдаланушы әрекеттеріне жауап береді, API сұраныстарын жібереді, алынған деректерді DOM құрылымына шығарады және интерфейстің күйін басқарады. Бұл үш технология веб-қосымшаның пайдаланушыға көрінетін бөлігін қалыптастырады.

Node.js серверлік JavaScript ортасы ретінде жобаның backend бөлігін құруға мүмкіндік берді. Оның артықшылығы - frontend пен backend бір тілдік экожүйеде жазылады, бұл шағын команда немесе оқу жобасы үшін ыңғайлы. Express.js фреймворкі HTTP маршруттарды, middleware тізбегін, JSON body parser, CORS баптауын, статикалық файлдарды тарату және error handler механизмін қарапайым ұйымдастыруға көмектеседі. Осы арқылы сервер тек файл таратушы емес, толық REST API қызметін атқарады.

MongoDB құжаттық деректер қоры ORDA жобасының объектілік табиғатына сәйкес келеді. Іс-шара, зал, брондау және пайдаланушы құжаттары бірдей қатаң реляциялық кестеге бағынбайды, олардың кейбір өрістері уақыт өте кеңеюі мүмкін. Мысалы, іс-шараға translations, tags, agenda, meetingUrl сияқты қосымша өрістер қосылады, ал зал объектісінде equipment және advantages массивтері бар. Mongoose бұл икемділікті сақтай отырып, схема деңгейінде міндетті өрістерді, типтерді және enum мәндерін бақылауға мүмкіндік береді.

REST API жүйенің клиент пен сервер арасындағы келісімін анықтайды. Frontend белгілі endpoint-ке сұраныс жібереді, backend сұранысты тексереді, деректер қорымен жұмыс істейді және JSON форматында жауап қайтарады. Бұл тәсіл интерфейсті сервер логикасынан бөледі: кейін мобильді қосымша немесе басқа frontend жасалса, сол API-ді қайта пайдалануға болады. Сонымен қатар

API маршруттарын ашық, авторизацияланған және әкімшіге ғана қолжетімді деп бөлу қауіпсіздік архитектурасын нақтылайды.

Қауіпсіздіктің бірінші деңгейі - парольдерді ашық мәтін түрінде сақтамау. bcryptjs құпиясөзді хэшке айналдырады, сондықтан деректер базасына тікелей қолжетімділік болған жағдайда да бастапқы құпиясөз көрінбейді. Екінші деңгей - JWT арқылы сессияны басқару. Пайдаланушы login жасағанда сервер қол қойылған токен береді, ал қорғалған маршруттарға сұраныс жібергенде frontend оны Authorization header арқылы жібереді. Сервер токенді тексеріп, пайдаланушының рөлі мен құқықтарын анықтайды.

Қауіпсіздіктің үшінші деңгейі - рөлдік қолжетімділік. ORDA жүйесінде қарапайым пайдаланушы өзінің өтінімдерін, профилін және тіркелулерін көре алады, ал әкімші барлық өтінімдер мен статистикаға қол жеткізеді. 'adminOnly' middleware әкімші емес пайдаланушының admin маршрутқа кіруін тоқтатады. Бұл тәсіл интерфейсте admin батырмасын жасырумен шектелмейді, себебі нақты қорғаныс сервер деңгейінде орындалады.

CORS баптауы да қауіпсіздік пен орналастыру тұрғысынан маңызды. Жүйе localhost, 127.0.0.1 және GitHub Pages сияқты рұқсат етілген origin-дермен жұмыс істей алады, ал production режимінде бөтен origin-ді шектеуге мүмкіндік бар. Бұл браузер арқылы рұқсатсыз cross-origin сұраныстардың алдын алуға көмектеседі. Сонымен қатар JSON body көлемінің шектелуі серверді тым үлкен сұраныстардан қорғаудың қарапайым тәсілі болып табылады.

Қауіпсіздік архитектурасында деректерді енгізу сапасын бақылау да маңызды. Пайдаланушы формадан жіберген ақпаратты тек frontend деңгейінде тексеру жеткіліксіз, себебі HTTP сұранысын браузерден тыс құралдар арқылы да жіберуге болады. Сондықтан backend controller-лері міндетті өрістерді, типтерді, enum мәндерін, уақыт форматын, қатысушылар санын және пайдаланушы құқығын қайта тексереді. Бұл тәсіл жүйенің сенімділігін арттырады және қате немесе әдейі бұрмаланған деректердің базаға түсу қаупін төмендетеді.

Жеке деректермен жұмыс істегенде минимизация қағидасын сақтау қажет. ORDA жүйесінде пайдаланушыдан жүйе жұмысына керек негізгі ақпарат қана сұралады: аты, email, телефон, ұйым және қала. Қатысушының артық жеке мәліметін жинамау қауіпсіздік тәуекелін азайтады. Сонымен қатар құпия параметрлерді '.env' файлында сақтау, репозиторийге жарияламау, JWT_SECRET мәнін жеткілікті күрделі ету және MongoDB Atlas connection string-ін қорғау production орналастырудың міндетті шарты болып табылады.

Бұл технологиялық стек дипломдық жоба үшін теңгерімді шешім болып табылады. Ол бір жағынан заманауи веб-әзірлеу тәжірибесіне сәйкес келеді, екінші жағынан шамадан тыс күрделі микросервистік архитектураны қажет етпейді. Жүйе кейін масштабталса, Express API-ді бөлек серверге шығару, MongoDB Atlas кластерін кеңейту, CDN қолдану, кэштеу қосу және frontend-ті жеке hosting-ке орналастыру сияқты бағыттар қарастырылуы мүмкін.

Технологиялық стек таңдау дипломдық жобаның тұрақтылығы мен түсініктілігіне әсер етеді. HTML, CSS және JavaScript frontend бөлігін браузерде қосымша құрастыру құралдарынсыз іске қосуға мүмкіндік береді. Бұл тәсіл

жобаны қорғау кезінде демонстрацияны жеңілдетеді және интерфейстің негізгі логикасын ашық көрсетеді.

Node.js және Express.js серверлік бөлік үшін таңдалды, себебі олар REST API құруға, middleware қабатын ұйымдастыруға және статикалық frontend файлдарын таратуға ыңғайлы. Express маршрутизациясы әр модульді жеке route файлына бөлуге мүмкіндік береді, бұл код құрылымын түсінікті етеді.

MongoDB Atlas бұлттық деректер қоры ретінде қолданылды. Құжаттық модель іс-шара, зал, пайдаланушы және брондау сияқты объектілерді икемді сипаттауға қолайлы. Mongoose схемалары өрістердің түрін, міндеттілігін, enum мәндерін және бастапқы мәндерін анықтайды.

Қауіпсіздік негізінде JWT авторизациясы жатыр. Пайдаланушы жүйеге кірген кезде сервер қол қойылған token қайтарады, ал қорғалған маршруттарда бұл token тексеріледі. Әкімші функциялары adminOnly middleware арқылы бөлініп, рөлдік қолжетімділік сақталады.

Технологиялық стек жобаның болашақта кеңеюіне де мүмкіндік береді. REST API сыртқы мобильді қосымшаға немесе жеке admin интерфейске қызмет көрсете алады, MongoDB Atlas деректер көлемі артқанда масштабталуы мүмкін, ал frontend модульдері жаңа беттермен толықтырылады.

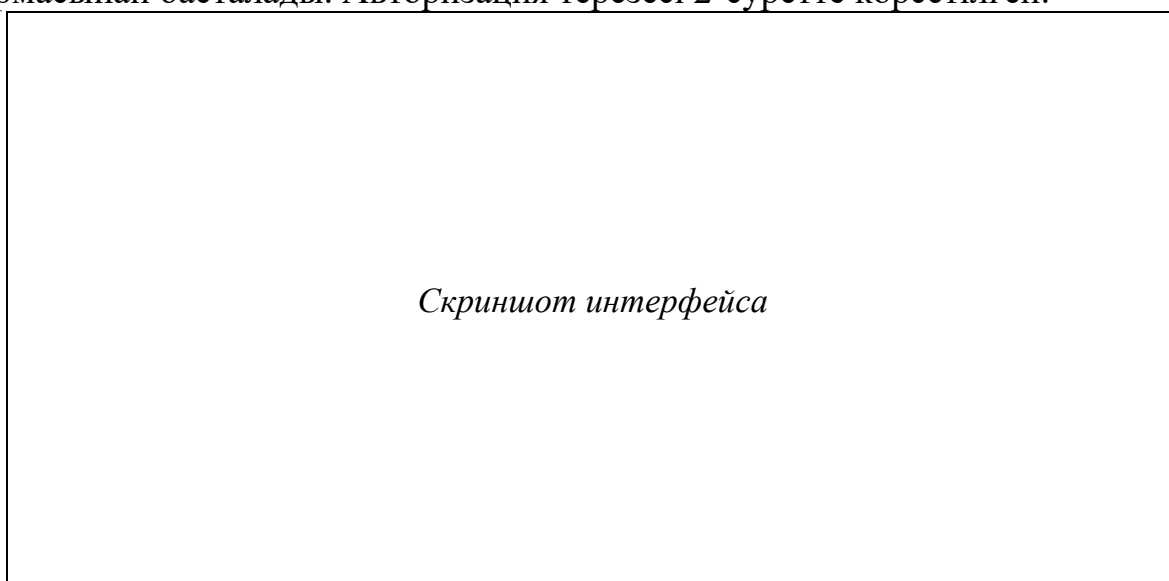
2 ORDA SMART EVENT SYSTEM ЖҮЙЕСІН ЖОБАЛАУ

2.1 Талаптар және пайдаланушы сценарийлері

Жүйені жобалау кезінде үш негізгі рөл қарастырылды: тіркелмеген қонақ, тіркелген пайдаланушы және әкімші. Қонақ іс-шаралар мен залдар туралы жалпы ақпаратты қарай алады. Тіркелген пайдаланушы іс-шараға жазылады, залға немесе өз іс-шарасын жариялауға өтінім береді, төлем мәртебесін жаңартады, жеке кабинетте өтінімдерін қарайды және әкімшіге техникалық қолдау хабарламасын жібереді. Әкімші каталогты, өтінімдерді, статистиканы, залдарды және клиент хабарламаларын басқарады.

Жүйенің негізгі функционалдық талаптары төмендегідей анықталды:

Жүйеде пайдаланушының бастапқы әрекеті тіркеу немесе кіру формасынан басталады. Авторизация терезесі 2-суретте көрсетілген.



Сурет 2. Пайдаланушыны тіркеу және авторизация терезесі

Талап түрі	Мазмұны	Жүйедегі іске асуы
Функционалдық	Тіркеу, авторизация, каталог, брондау, өтінім мәртебесі	Frontend формалары және REST API endpoint-тері
Қауіпсіздік	Жеке деректерді қорғау және рөлдік қолжетімділік	JWT token, adminOnly middleware, парольді хэштелген түрде сақтау
Қолжетімділік	Қазақ, орыс және ағылшын тіліндегі интерфейс	i18n сөздіктері және интерфейс мәтіндерін динамикалық ауыстыру
Кеңейтілу	Жаңа іс-шара, зал, хабарлама және есеп модульдерін қосу	Express routes, Mongoose модельдері және модульдік frontend құрылымы

- пайдаланушыны тіркеу және жүйеге кіргізу;
- бірінші тіркелген пайдаланушыны автоматты түрде әкімші ету;
- іс-шараларды іздеу, категория, формат және тегтер бойынша сүзгілеу;
- іс-шараға қатысушы ретінде жазылу және бос орын санын бақылау;

- конференц-залдар каталогын көрсету және сыйымдылық бойынша сүзу;
- залға өтінім беру кезінде уақыт қақтығысын анықтау;
- әкімшіге өтінім мәртебесін өзгерту және клиентке хабарлама көрсету;
- жарык/қараңғы тақырып пен RU/KZ/EN тілдерін қолдау;
- техникалық қолдау хабарламаларын жіберу және әкімші тарапынан өңдеу;

Талаптарды қалыптастыру кезеңінде жүйенің негізгі акторлары анықталды. Қонақ пайдаланушы ақпаратты қарайды және жүйенің мүмкіндіктерімен танысады. Тіркелген пайдаланушы іс-шараға жазылады, залға өтінім береді, жеке деректерін жаңартады және мәртебелерді бақылайды. Әкімші деректерді басқару, өтінімдерді қарау, статистиканы бақылау және техникалық қолдау хабарламаларын өңдеу міндетін атқарады. Мұндай рөлдік бөлу әр функцияның кімге арналғанын нақтылауға мүмкіндік береді.

Функционалдық талаптар жүйенің не істеуі керектігін сипаттайды. ORDA жағдайында бұған тіркелу, кіру, іс-шара каталогын қарау, филтрлеу, брондау, залға өтінім беру, admin dashboard, check-in және support модулі кіреді. Ал функционалдық емес талаптар жүйенің қалай жұмыс істеуі керектігін көрсетеді: жауап беру жылдамдығы, қауіпсіздік, қолжетімділік, кеңейтілу, интерфейстің түсініктілігі және деректердің тұрақты сақталуы.

Пайдаланушы сценарийлерін талдау жобалаудың маңызды бөлігі болды. Мысалы, іс-шараға тіркелу сценарийінде пайдаланушы алдымен каталогтан event таңдайды, содан кейін жүйеге кіреді немесе тіркеледі, кейін серверге booking сұранысы жіберіледі. Сервер eventId арқылы іс-шараны табады, пайдаланушының бұрын жазылған-жазылмағанын тексереді, бос орын санын есептейді және жаңа Booking құжатын жасайды. Бұл сценарий қарапайым батырма басудан басталғанымен, бірнеше ішкі тексерісті қамтиды.

Залға өтінім беру сценарийі күрделілеу. Пайдаланушы залды таңдайды, күн мен уақытты енгізеді, қатысушылар санын, мақсатты, қызметтерді және төлем әдісін көрсетеді. Сервер залдың бар-жоғын, сыйымдылығын, уақыттың дұрыс енгізілгенін және қақтығыстың жоқтығын тексереді. Егер өтінім custom-event түрінде болса, әкімші бекіткеннен кейін ол жеке іс-шара ретінде каталогқа жариялануы мүмкін. Бұл процесс жүйенің брондау мен event management логикасын біріктіретінін көрсетеді.

Әкімші сценарийлері ұйымның ішкі жұмысын қолдайды. Әкімші жаңа іс-шара жариялайды, зал туралы ақпаратты жаңартады, өтінім мәртебесін өзгертеді, қатысушыны check-in арқылы белгілейді, төлем мәртебесін қарайды және support хабарламаларын өңдейді. Әкімші әрекеттері жүйеде қадағаланатын мәртебелер арқылы көрінеді, сондықтан пайдаланушы да жеке кабинеттен шешім нәтижесін бақылай алады.

Талаптарды осылай бөлу жобаның шекарасын нақтылайды. Дипломдық жоба төлем шлюзін толық іске қоспаса да, төлем мәртебесі мен төлем әдісін сақтау арқылы болашақ интеграцияға дайындалады. Email немесе SMS хабарлама әзірше толық іске қосылмаса да, notification логикасы мен support

модулі пайдаланушыға кері байланыс беру бағытын көрсетеді. Яғни жүйе қазіргі мүмкіндігімен жұмыс істейді және кеңейту нүктелерін сақтайды.

Талаптарды анықтау кезеңінде жүйенің кім үшін жасалатыны нақты белгіленеді. ORDA платформасында негізгі пайдаланушыларға іс-шараға қатысушы, зал брондаушы клиент және әкімші жатады. Әр рөлдің интерфейстегі мүмкіндігі мен серверлік қолжетімділігі әртүрлі.

Қарапайым пайдаланушы үшін басты талаптар - тіркелу, жүйеге кіру, каталогты қарау, іс-шараға жазылу, залға өтінім беру және жеке кабинеттен мәртебені бақылау. Бұл сценарийлер жүйенің клиенттік бөлігі мен API endpoint-тері арасында тұрақты байланысты қажет етеді.

Әкімші үшін талаптар кеңірек: іс-шара қосу, өңдеу, жою, залдарды басқару, өтінім мәртебесін өзгерту, check-in белгілеу, статистика қарау және support хабарламаларын оқу. Бұл әрекеттер рөлдік қолжетімділікпен шектелуі тиіс, себебі олар жүйедегі негізгі деректерге әсер етеді.

Функционалдық емес талаптар да маңызды. Жүйе әртүрлі экран өлшемінде дұрыс көрінуі, сұраныстарға жылдам жауап беруі, қате жағдайларын түсінікті хабарламамен көрсетуі және деректерді жоғалтпай сақтауы керек. Мұндай талаптар пайдаланушы сенімін қалыптастырады.

Талаптар жүйені тек экран жиынтығы ретінде емес, толық ақпараттық процесс ретінде қарастыруға мүмкіндік береді. Әр талап нақты модульмен байланысады: авторизация auth controller арқылы, каталог event controller арқылы, залдар hall controller арқылы, ал өтінімдер booking controller арқылы орындалады.

2.2 Клиент-сервер архитектурасы және деректер ағыны

ORDA веб-жүйесінде клиенттік және серверлік қабаттар нақты бөлінген. Клиенттік бөлік `docs` каталогында орналасқан және браузерде орындалады. Серверлік бөлік `backend/src` каталогында орналасып, Express қосымшасы ретінде API сұраныстарын өңдейді. Негізгі `server.js` файлы backend серверін іске қосады, ал Express статикалық файлдарды таратып, `/api` кеңістігіндегі маршруттарға жауап береді.

Деректер ағыны келесі тәртіппен жүреді: пайдаланушы интерфейсте әрекет жасайды, JavaScript HTTP сұранысын жібереді, Express маршруты тиісті controller функциясын шақырады, controller Mongoose моделі арқылы MongoDB деректер қорымен жұмыс істейді және JSON жауап қайтарады. Клиент жауапқа байланысты интерфейсін жаңартады.

Клиенттік және серверлік қабаттардың нәтижесі пайдаланушыға басты бет арқылы ұсынылады. Жүйенің басты беті 3-суретте көрсетілген.



Скриншот интерфейса

Сурет 3. Жүйенің басты беті және жұмыс кеңістігі

Клиент-сервер архитектурасының басты артықшылығы - жауапкершілікті бөлу. Клиенттік қабат пайдаланушыға интерфейс ұсынады, енгізілген деректерді бастапқы деңгейде тексереді және серверге сұраныс жібереді. Серверлік қабат негізгі бизнес-логиканы орындайды, деректер қорымен жұмыс істейді және қауіпсіздік тексерістерін жүргізеді. Деректер қоры тұрақты сақтау міндетін атқарады. Бұл бөлініс жүйені түсінуді, тестілеуді және кеңейтуді жеңілдетеді.

ORDA жүйесінде frontend `docs` каталогында орналасқан статикалық файлдардан тұрады. `index.html` бет құрылымын береді, `styles.css` визуалды дизайнды қалыптастырады, ал `app.js` негізгі интерактивті логиканы басқарады. Core модульдерде http helper, session helper, i18n, theme, toast және форматтау функциялары бөлінген. Бұл frontend кодын толық монолит емес, бірнеше жауапкершілік аймағына бөлінген құрылым ретінде қарастыруға мүмкіндік береді.

Backend `backend/src` құрылымында қабаттарға бөлінген. `config` каталогында орта айнымалылары мен MongoDB connection логикасы орналасады. `models` Mongoose схемаларын сипаттайды. `middleware` авторизация, рөл тексеру, валидация және қате өңдеу міндеттерін орындайды. `controllers` бизнес-логиканы қамтиды, ал `routes` HTTP endpoint-терді controller функцияларымен байланыстырады. Мұндай құрылым Express жобалары үшін түсінікті және қолдауға ыңғайлы.

Деректер ағынын мысал арқылы көрсетуге болады. Пайдаланушы login формасын толтырғанда frontend `/login` endpoint-іне POST сұраныс жібереді. Backend email мәнін normalize етеді, пайдаланушыны деректер қорынан іздейді, bcrypt арқылы құпиясөзді салыстырады және JWT токенін қайтарады. Frontend бұл токенді session helper арқылы сақтап, келесі қорғалған сұраныстарға қосады. Осылайша authentication күйі клиентте сақталған белгі мен сервердегі тексерістің үйлесімі арқылы орындалады.

Іс-шаралар каталогын жүктеу кезінде frontend search, category, format немесе free сияқты query параметрлерін API-ге жібереді. Event controller

MongoDB query құрып, жарияланған іс-шараларды дата мен уақыт бойынша сұрыптайды. Әр іс-шара үшін booked count және пайдаланушының бұрын тіркелгені туралы белгі есептеледі. Бұл ақпарат frontend-ке қайтып келген соң карточкадағы бос орын, тіркелу батырмасы және мәртебе визуалды түрде жаңартылады.

Қате өңдеу архитектураның маңызды бөлігі болып табылады. Егер сұраныс дұрыс болмаса, controller ApiError арқылы мағыналы статус пен хабарлама қайтарады. Error handler бұл қатені JSON жауапқа айналдырады. Frontend showMessage немесе toast механизмі арқылы пайдаланушыға түсінікті хабарлама көрсетеді. Осылайша техникалық қате ішкі деңгейде өңделіп, интерфейсте басқарылатын кері байланысқа айналады.

Жүйе `/api` namespace арқылы да, кейбір бұрынғы frontend сұраныстарымен үйлесімді болу үшін namespace-сыз да маршруттарды қолдайды. Бұл шешім әзірлеу кезінде пайдалы, себебі frontend кодын бірден толық ауыстырмай, API құрылымын біртіндеп жаңартуға мүмкіндік береді. Дегенмен production деңгейінде endpoint келісімін нақты құжаттап, бір негізгі namespace қолдану жүйені қолдауды жеңілдетеді.

Клиент-сервер архитектурасында frontend пайдаланушы әрекетін қабылдайды, ал backend деректерді тексеріп, бизнес-логиканы орындайды. Бұл бөлініс интерфейстің көрінісін серверлік есептеулерден ажыратады және жүйені кезең-кезеңімен дамытуға мүмкіндік береді.

Frontend HTTP сұраныстарды арнайы helper арқылы жібереді. Бұл тәсіл әр жерде fetch логикусын қайталамай, авторизация token-ін, қате өңдеуді және JSON жауаптарын бір тәртіпке келтіреді. Нәтижесінде код құрылымы ықшам әрі қолдауға ыңғайлы болады.

Backend Express қосымшасы бірнеше қабаттан тұрады: route сұранысты қабылдайды, middleware қауіпсіздік пен валидацияны тексереді, controller бизнес-логиканы орындайды, ал model деректер қорымен жұмыс істейді. Мұндай құрылым MVC қағидасына жақын және дипломдық жобада түсіндіруге ыңғайлы.

Деректер ағыны нақты мысал арқылы жақсы көрінеді. Пайдаланушы зал брондау формасын толтырғанда frontend `/api/hall-bookings` endpoint-іне сұраныс жібереді. Сервер пайдаланушы token-ін тексереді, уақыт аралықтарының қабаттасуын анықтайды, жаңа Booking құжатын сақтайды және JSON жауап қайтарады.

Архитектурадағы маңызды шешімдердің бірі - frontend пен backend бір жобада орналасса да, API namespace арқылы байланысуы. Бұл шешім кейін production деңгейінде қабаттарды бөлек орналастыруға мүмкіндік береді және қазіргі кодты толық қайта жазуды қажет етпейді.

2.3 Деректер қоры және API логикусы

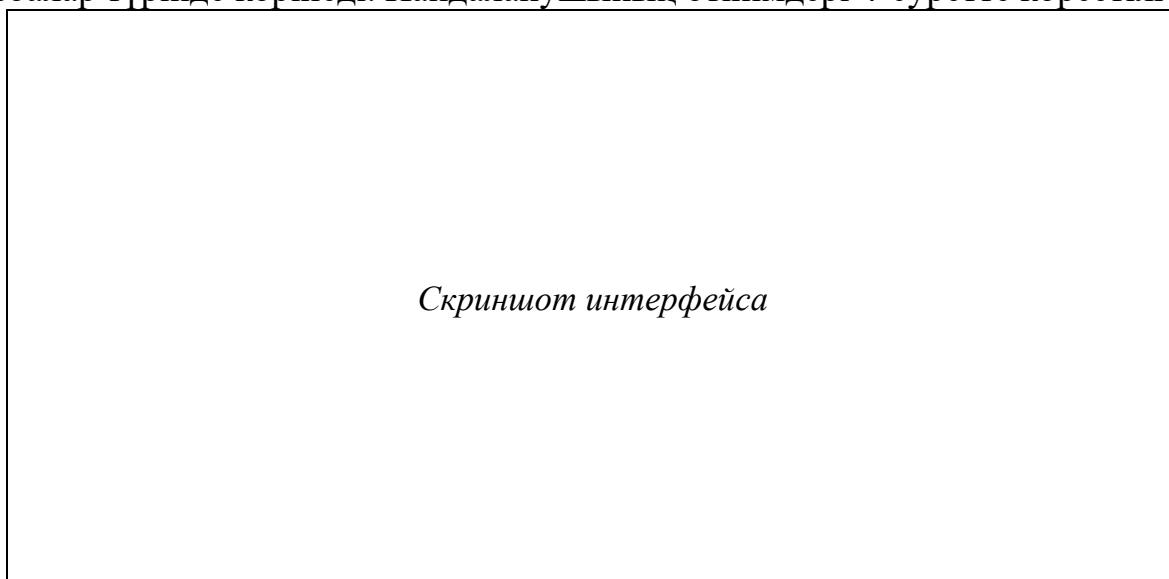
MongoDB құжаттық деректер қоры ORDA жүйесіне ыңғайлы, себебі іс-шара, зал, өтінім және пайдаланушы объектілері әртүрлі өрістерден тұрады және уақыт өте кеңейтілуі мүмкін. Mongoose схемалары арқылы әр

коллекцияның міндетті өрістері, типтері, enum мәндері және бастапқы мәндері анықталған.

API логикасында маршруттар екі нұсқада қолжетімді: `/api` namespace арқылы және кейбір ескі frontend сұраныстарымен үйлесімді болу үшін namespace-сыз. Бұл шешім интерфейсті біртіндеп жаңартуға мүмкіндік береді. Қорғалған маршруттарда `auth` middleware, әкімші маршруттарында `adminOnly` middleware қолданылады.

Модель	Негізгі өрістер	Мақсаты
User	name, email, password, role, phone, organization, city	Пайдаланушы профилін және рөлін сақтау
Event	title, date, category, format, location, capacity, status	Іс-шаралар каталогын қалыптастыру
Hall	name, floor, location, capacity, pricePerHour, equipment, status	Конференц-залдар туралы деректерді сақтау
Booking	type, userEmail, eventId, hallId, date, status, paymentStatus	Іс-шараға тіркелу және зал брондау өтінімдерін басқару
SupportMessage	userName, userEmail, text, status, readAt	Техникалық қолдау хабарламаларын өңдеу

Деректер қоры мен API жұмысының нәтижесі жеке кабинетте нақты жазбалар түрінде көрінеді. Пайдаланушының өтінімдері 4-суретте көрсетілген.



Сурет 4. Жеке кабинеттегі өтінімдер тізімі

Деректер қоры құрылымында User коллекциясы пайдаланушының негізгі деректерін сақтайды. Мұнда name, email, password, role, phone, organization және city өрістері бар. Email бірегей болуы тиіс, ал role user немесе admin мәндерінің бірін қабылдайды. Бұл модель авторизация мен рөлдік қолжетімділіктің негізін құрайды. Құпиясөздің базаға хэш түрінде сақталуы жеке деректерді қорғаудың маңызды шарты болып табылады.

Event коллекциясы іс-шара туралы толық ақпаратты қамтиды: title, date, endDate, time, category, description, agenda, organizer, location, city, format, meetingUrl, image, price, currency, featured, tags, capacity және status. Осындай өрістер каталог карточкасын, event detail бөлімін, фильтрлеуді және admin

формасын қамтамасыз етеді. Status өрісінің draft, published және closed мәндері іс-шараның өмірлік циклін көрсетуге мүмкіндік береді.

Hall коллекциясы конференц-залдың сипаттамасын сақтайды. Name, floor, location, capacity, pricePerHour, equipment, description, translations, status, image және advantages өрістері зал туралы толық ақпарат беруге арналған. Capacity залға өтінім беру кезінде қатысушылар санын тексеруге қолданылады, ал status available, busy немесе maintenance күйін көрсетуі мүмкін. Equipment және advantages массивтері залдың техникалық мүмкіндіктерін интерфейсте көрсетуге ыңғайлы.

Booking коллекциясы жүйенің ең маңызды байланыс объектілерінің бірі. Ол event тіркелуі, hall өтінімі және custom-event сұранысы сияқты бірнеше типті бір модельде сақтайды. Type өрісі event, hall немесе custom-event мәнін алады. Сонымен қатар userEmail, userName, eventId, hallId, date, time, duration, attendees, services, equipment, amount, paymentStatus, status, checkedIn және adminReason сияқты өрістер процестің толық күйін сипаттайды.

SupportMessage коллекциясы пайдаланушы мен әкімші арасындағы техникалық байланыс арнасын қамтамасыз етеді. Онда userId, userName, userEmail, text, status және readAt өрістері сақталады. Мұндай модель қарапайым көрінгенімен, жүйені қолдау сапасын арттырады: пайдаланушы сұрақ қалдырады, әкімші оны оқиды, мәртебесін белгілейді және қажет болған жағдайда жауап беру процесін ұйымдастырады.

API логикасында әр модельге сәйкес controller жауапкершілігі бөлінген. Auth controller тіркелу мен кіруді өңдейді, Event controller каталог пен event CRUD операцияларын басқарады, Hall controller залдарды өңдейді, Booking controller пайдаланушы өтінімдерін жасайды және қайтарады, Admin controller жалпы басқару функцияларын орындайды, Support controller хабарламалармен жұмыс істейді. Бұл бөлініс кодты іздеуді және өзгертуді жеңілдетеді.

Деректердің тұтастығын сақтау үшін сервер деңгейінде бірнеше тексеріс қолданылады. Event жасау кезінде міндетті өрістер тексеріледі, зал брондау кезінде уақыт қақтығысы анықталады, іс-шараға тіркелуде қайталанған booking пен capacity шектеуі бақыланады. Бұл тексерістерді тек frontend формасына сеніп қою жеткіліксіз, себебі API-ге сұранысты басқа құрал арқылы да жіберуге болады. Сондықтан негізгі валидация backend деңгейінде орындалуы тиіс.

MongoDB Atlas сияқты бұлттық база қолданылған жағдайда жүйе локалды компьютерден тәуелсіз сақтауға ие болады. Бұл демонстрация кезінде де, болашақ production орналастыруда да ыңғайлы. Дегенмен бұлттық базаға IP whitelist, connection string құпиялығы, `.env` файлын қорғау және backup стратегиясы қажет. Осы талаптар дипломдық жобадан қауіпсіздік пен масштабталудың практикалық аспектілерін түсіндіруге мүмкіндік береді.

База құрылымын болашақта оңтайландыру үшін индекстер мәселесі де қарастырылуы мүмкін. Мысалы, Event коллекциясында date, status, category және format өрістеріне сұраныс жиі түседі, ал Booking коллекциясында userEmail, eventId, hallId, date және status бойынша іздеу маңызды. Индекстер дұрыс қойылса, деректер саны артқан кезде каталог жүктелуі, жеке кабинет сұраныстары және admin dashboard статистикасы жылдамырақ орындалады.

Алайда индекстерді шамадан тыс көбейту жазу операцияларын баяулатуы мүмкін, сондықтан олар нақты сұраныс үлгісіне сүйеніп таңдалуы тиіс.

Деректер қорын жобалауда резервтік көшіру және қалпына келтіру жоспары да практикалық мәнге ие. Іс-шаралар, өтінімдер және қатысушылар тізімі ұйым үшін маңызды ақпарат болып табылады, сондықтан база бұзылған немесе кездейсоқ өшірілген жағдайда деректерді қайта қалпына келтіру мүмкіндігі болуы керек. MongoDB Atlas snapshot, backup және monitoring құралдарын ұсына алады, ал шағын орналастыруда экспорт файлдарын кезеңдік сақтау бастапқы қорғаныс шарасы ретінде қарастырылуы мүмкін.

Деректер қоры құрылымы жүйенің тұрақты жұмысын анықтайды. Егер модельдер дұрыс бөлінбесе, бір объектіге қатысты ақпарат бірнеше жерде қайталанып, кейін жаңарту қиындайды. ORDA жүйесінде пайдаланушы, іс-шара, зал, брондау және қолдау хабарламасы жеке модельдер ретінде қарастырылған.

User моделінде пайдаланушының аты, email мекенжайы, құпиясөзі, рөлі, телефоны, ұйымы және қаласы сақталады. Email бірегей болуы тиіс, себебі авторизация осы өріс арқылы орындалады. Role өрісі user және admin мәндері арқылы қолжетімділікті басқаруға негіз болады.

Event моделінде іс-шараның атауы, күні, аяқталу күні, уақыты, категориясы, сипаттамасы, ұйымдастырушысы, орны, форматы, бағасы, сыйымдылығы және жариялану мәртебесі бар. Бұл өрістер каталог карточкасын, detail бетін және admin формасын қалыптастыруға жеткілікті.

Hall моделінде зал атауы, қабаты, орналасқан жері, сыйымдылығы, сағаттық бағасы, жабдықтары, сипаттамасы, аудармалары, мәртебесі және артықшылықтары сақталады. Бұл құрылым конференц-залды тек атау ретінде емес, брондауға болатын ресурс ретінде сипаттайды.

Booking моделі жүйедегі ең күрделі объектілердің бірі. Ол event, hall және custom-event түрлерін қолдайды, пайдаланушы деректерін, уақыт аралығын, төлем мәртебесін, admin ескертпесін және check-in белгісін сақтайды. Осы модель арқылы қатысушы тіркеуі мен зал брондау бір логикада өңделеді.

2.4 Интерфейс, көптілділік және стиль

Интерфейс SaaS стиліндегі жұмыс панелі ретінде жобаланған. Бетте жоғарғы header, жаһандық іздеу, тіл таңдауы, тақырып ауыстырғыш, account аймағы және бүйірлік навигация бар. Навигацияда басты бет, іс-шаралар, залдар, күнтізбе, карта, профиль, өтінім және ақпарат бөлімдері орналасады. Әкімші кірген кезде қосымша admin бөлімі ашылады.

Көптілділік `docs/js/core/i18n.js` файлы арқылы ұйымдастырылған. Интерфейстегі мәтіндер `data-i18n` атрибуттары арқылы байланыстырылады, ал пайдаланушы RU, KZ немесе EN тілін таңдай алады. Бұл шешім бір HTML құрылымын сақтай отырып, мәтіндік қабатты бөлек басқаруға мүмкіндік береді.

Интерфейс мәтіндері таңдалған тілге байланысты ауысып отырады. Көптілді навигация 5-суретте көрсетілген.



Скриншот интерфейса

Сурет 5. Көптілді интерфейс және негізгі навигация

Интерфейсті жобалауда негізгі мақсат - пайдаланушыны ақпаратпен шамадан тыс жүктемей, жиі орындалатын әрекеттерді тез табуға мүмкіндік беру. ORDA жүйесінде dashboard логикасы қолданылған: бүйірлік навигация негізгі бөлімдерді тұрақты көрсетеді, ал орталық аймақта таңдалған view мазмұны ауысады. Бұл тәсіл жеке беттер арасында қайта жүктеусіз ауысуға және қолданушыны бір жұмыс кеңістігінде ұстауға көмектеседі.

Визуалдық стильде SaaS панельдеріне тән қағидалар байқалады: карточкалар, статус белгілері, фильтрлер, форма блоктары, статистикалық көрсеткіштер және түс арқылы ерекшелену. Мұндай стиль іс-шара жүйесіне сәйкес келеді, себебі пайдаланушы бірден бірнеше объектіні салыстырады: events, halls, bookings, calendar entries және notifications. Әр объектінің карточка немесе жол түрінде көрсетілуі ақпаратты сканерлеуді жеңілдетеді.

Адаптивті дизайн кезінде мазмұнның иерархиясын сақтау қажет. Мобильді экранда бүйірлік навигация, карточка торы және формалар бір бағанға түсуі мүмкін, бірақ негізгі әрекеттер жоғалмауы тиіс. Сондықтан батырмалар жеткілікті үлкен болуы, форма өрістері оқылатын болуы, ал мәтіндер экраннан шықпауы керек. Бұл талаптар CSS деңгейінде breakpoints, flexible layout және spacing жүйесі арқылы шешіледі.

Жарық және қараңғы тақырып пайдаланушының жұмыс жағдайына бейімделуге мүмкіндік береді. Қараңғы тақырып кешкі уақытта немесе проекторда көрсету кезінде ыңғайлы болуы мүмкін, ал жарық тақырып күнделікті кеңсе жұмысында оқуға жеңіл. Тақырып ауыстыру тек түсті өзгерту емес: контраст, мәтіннің оқылуы, border және фон реңктерінің үйлесімі сақталуы тиіс. ORDA жобасында theme helper бұл күйді басқаруға арналған.

Көптілді интерфейсте терминдердің тұрақтылығы маңызды. Бір тілде 'booking', екінші тілде 'өтінім', үшінші тілде 'заявка' деп берілген ұғымдар бір функцияны білдіруі керек. Егер терминдер әр бөлімде әртүрлі қолданылса, пайдаланушы шатасуы мүмкін. Сондықтан i18n сөздіктерін бөлек файлда сақтау тек техникалық шешім ғана емес, интерфейс мәтіндерінің сапасын басқару құралы болып табылады.

UX/UI шешімдерінің тиімділігі жүйенің қабылдануына тікелей әсер етеді. Тіпті backend дұрыс жұмыс істесе де, пайдаланушы интерфейсті түсінбесе, жүйе практикалық тұрғыдан әлсіз болады. Сол себепті дипломдық жұмыста интерфейс функционалдық модульдердің көрінісі ретінде ғана емес, пайдаланушы мен ақпараттық жүйе арасындағы негізгі байланыс арнасы ретінде қарастырылды.

Интерфейсті жобалауда негізгі мақсат - пайдаланушыға әрекетті тез орындауға мүмкіндік беру. Басты бет жүйенің жалпы мәнін көрсетеді, каталог қажетті іс-шараны табуға көмектеседі, залдар бөлімі ресурстарды салыстыруға мүмкіндік береді, ал жеке кабинет өтінімдерді бақылауға арналған.

Көптілділік интерфейстің кәсіби деңгейін арттырады. Қазақ, орыс және ағылшын тілдерінің болуы оқу орны, конференц-орталық немесе бизнес алаңы сияқты әртүрлі ұйымдарға жүйені бейімдеуге жағдай жасайды. Мәтіндер i18n сөздігінде сақталып, таңдалған тілге қарай динамикалық ауысады.

Визуалдық стильде ақпараттың оқылуына басымдық беріледі. Батырмалар, карточкалар, форма өрістері және кестелер бір жүйелі дизайнға бағынады. Бұл пайдаланушыға әр бөлімде жаңа логиканы қайта үйренбей, бұрынғы әрекет үлгісін қолдануға мүмкіндік береді.

Жеке кабинет интерфейсі пайдаланушы үшін ерекше маңызды, себебі ол жүйемен өзара әрекетінің нәтижесін осы жерден көреді. Мұнда тіркелген іс-шаралар, зал өтінімдері, мәртебе және хабарламалар көрсетіледі. Осындай кері байланыс жүйенің сенімділігін арттырады.

Admin интерфейсте ақпарат тығыз әрі басқаруға ыңғайлы орналасуы керек. Әкімші бірнеше өтінімді салыстырады, мәртебе өзгертеді, статистика қарайды және support хабарламаларына жауап береді. Сондықтан бұл бөлімде артық визуалды элементтерден гөрі нақты деректердің көрінуі маңызды.

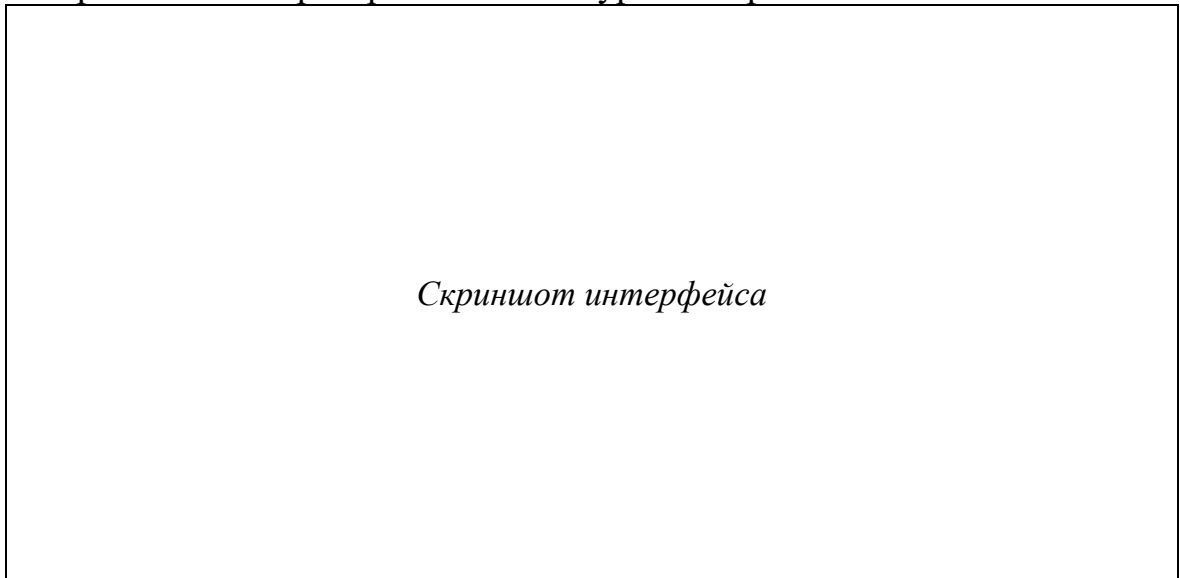
3 ЖҮЙЕНІ ІСКЕ АСЫРУ ЖӘНЕ ТЕСТІЛЕУ

3.1 Клиенттік модульдерді іске асыру

Клиенттік бөліктің негізгі мақсаты - пайдаланушыға іс-шара немесе зал туралы ақпаратты тез табуға, өтінім жіберуге және мәртебесін бақылауға мүмкіндік беру. `docs/index.html` файлында барлық негізгі view бөлімдері орналасқан, ал `docs/app.js` пайдаланушы әрекеттерін, деректерді жүктеуді, формаларды жіберуді және интерфейсті жаңартуды басқарады.

Авторизация модулі login және register формаларынан тұрады. Пайдаланушы email және құпиясөз енгізгеннен кейін сервер JWT токенін қайтарады. Токен браузердегі session helper арқылы сақталып, кейінгі қорғалған сұраныстардың `Authorization: Bearer <token>` header-іне қосылады.

Каталогта іздеу, категория және формат бойынша сүзгілеу қарастырылған. Іс-шаралар каталогы 6-суретте көрсетілген.

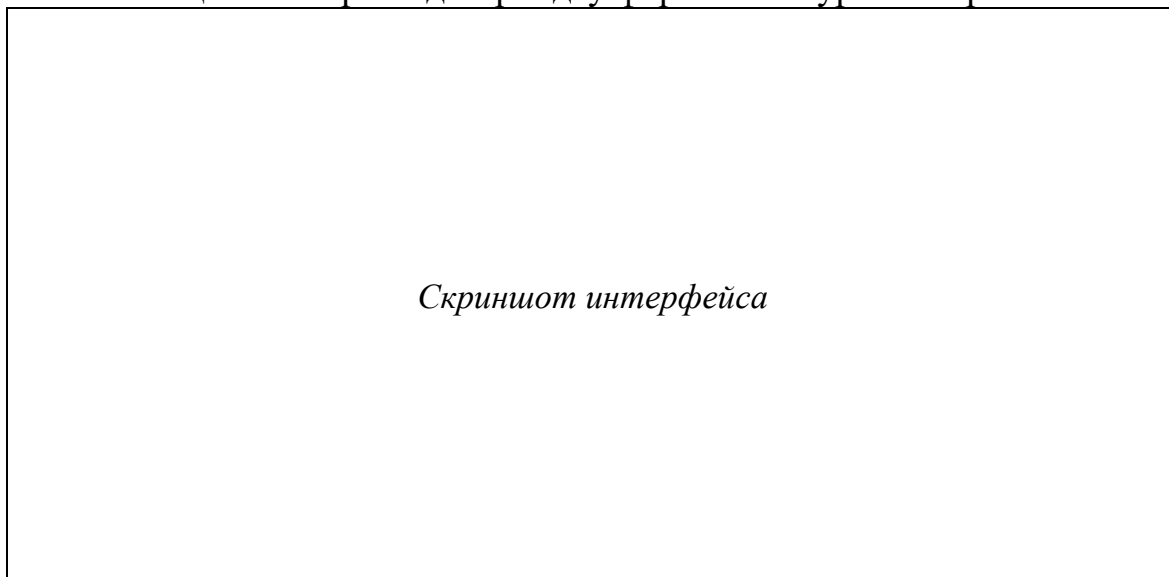


Сурет 6. Іс-шаралар каталогы және сүзгілеу құралдары

Іс-шаралар каталогы пайдаланушыға жүйедегі негізгі ақпаратты бірден көруге мүмкіндік береді. Карточка құрылымында атау, күн, формат, орын, баға және бос орын саны беріледі, сондықтан пайдаланушы шешімді бірнеше қосымша бет ашпай-ақ қабылдай алады.

Каталогтағы сүзгілеу құралдары деректер көлемі көбейген кезде ерекше маңызды болады. Search, category және format параметрлері frontend арқылы API сұранысына жіберіліп, серверден нақты шартқа сәйкес келетін events тізімі алынады.

Залға өтінім беру кезінде пайдаланушы күнді, уақытты, қатысушылар санын және мақсатты көрсетеді. Брондау формасы 7-суретте көрсетілген.

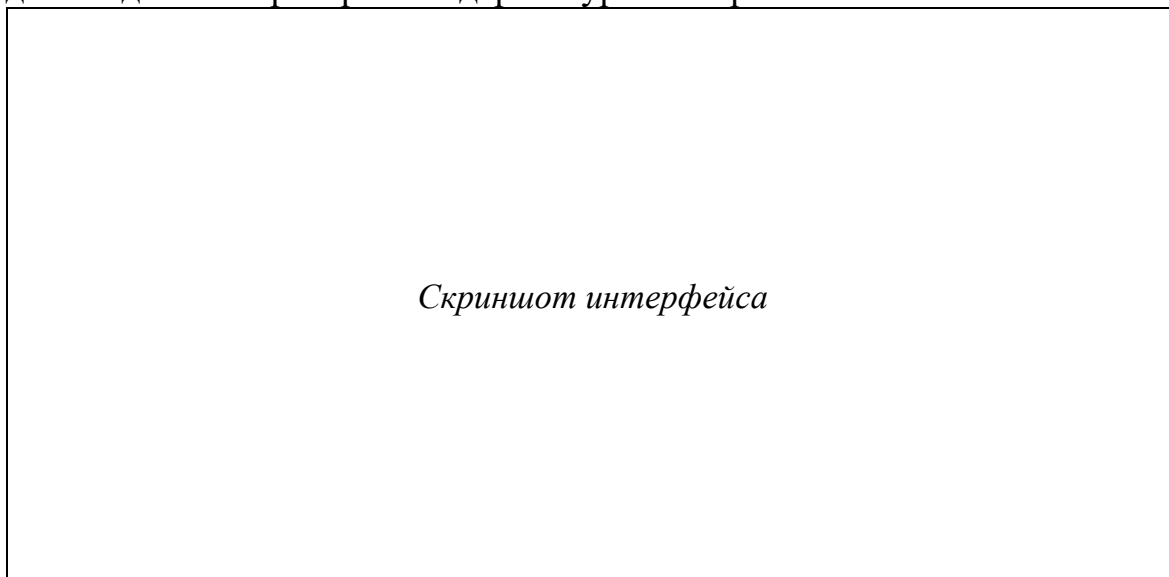


Сурет 7. Конференц-залды брондау формасы

Брондау формасында енгізілетін деректер нақты ұйымдастыру процесімен байланысты. Күн, басталу уақыты, аяқталу уақыты, қатысушылар саны және мақсат өрістері кейін серверлік деңгейде тексеріліп, залдың қолжетімділігімен салыстырылады.

Формадағы мәліметтер толық және түсінікті болған сайын әкімші өтінімді тезірек өңдей алады. Сондықтан интерфейсте міндетті өрістер, қосымша қызметтер және төлем мәртебесі логикалық тәртіппен орналастырылған.

Жоспарланған іс-шараларды уақыт бойынша бақылау үшін календарь қолданылады. Іс-шаралар календарі 8-суретте көрсетілген.



Скриншот интерфейса

Сурет 8. Календарь және жоспарланған іс-шаралар тізімі

Frontend деңгейінде авторизация формалары пайдаланушының жүйемен алғашқы белсенді байланысын құрайды. Register формасында аты, email және құпиясөз енгізіледі, ал login формасы бұрын тіркелген пайдаланушыны тануға арналған. Email мәнін normalize ету және құпиясөздің минималды ұзындығын тексеру қате деректер санын азайтады. Сәтті кіргеннен кейін интерфейс пайдаланушы рөліне қарай бейімделеді: қарапайым user мен admin әртүрлі бөлімдерді көреді.

Session helper токенді сақтау және оны кейінгі сұраныстарға қосу міндетін атқарады. Бұл механизм пайдаланушы әр әрекет сайын қайта логин жасамауы үшін қажет. Сонымен қатар мерзімі өткен немесе жарамсыз токен анықталса, session reset логикасы қолданушыны қайта кіруге бағыттайды. Мұндай шешім қауіпсіздік пен ыңғайлылық арасындағы теңгерімді сақтайды.

Іс-шара модулінде пайдаланушы жарияланған events тізімін көреді. Карточкада атауы, күні, уақыты, орны, форматы, бағасы, бос орын саны және тіркелу батырмасы көрсетіледі. Жүйе пайдаланушының бұрын тіркелгенін анықтап, `isBookedByMe` белгісі арқылы интерфейсті өзгертеді.

Іс-шара модулінде деректерді көрсету ғана емес, оны іздеу және сүзу маңызды. Search өрісі атау, сипаттама, орын, ұйымдастырушы немесе тегтер бойынша ақпарат табуға көмектеседі. Category және format филтрлері пайдаланушыға қажетті мазмұнды тез шектеуге мүмкіндік береді. Featured белгісі маңызды іс-шараларды бөлек көрсетуге, ал free филтри ақысыз events тізімін алуға қолданылады.

Event карточкасының құрылымы шешім қабылдауға қажетті қысқа ақпаратты біріктіреді. Күні мен уақыты қатысу мүмкіндігін, location зал немесе қала контекстін, price қаржылық шартты, capacity және booked count бос орын санын көрсетеді. Егер пайдаланушы бұрын тіркелген болса, интерфейс сол күйді көрсетуі керек. Бұл қайталанған әрекетті азайтады және пайдаланушыға жүйенің оның тарихын есте сақтайтынын байқатады.

Залдар модулі конференц-залдар каталогын көрсетеді. Пайдаланушы залды сыйымдылық бойынша сүзеді, сипаттамасын, бағасын, жабдығын және бос уақытын қарайды. Залға өтінім беру кезінде күн, уақыт, ұзақтық, қатысушылар саны, мақсат, қосымша қызметтер және төлем әдісі енгізіледі.

Зал модулі кеңістікті ресурс ретінде басқаруға арналған. Әр залдың сыйымдылығы, қабаты, орналасуы, сағаттық бағасы, жабдықтары және сипаттамасы болады. Бұл деректер клиентке дұрыс зал таңдауға көмектеседі, ал әкімші үшін зал ресурстарын құрылымдауға мүмкіндік береді. Егер зал maintenance күйінде болса немесе оған белсенді өтінімдер болса, оны жою немесе қайта брондау қосымша тексерісті қажет етеді.

Брондау формасында енгізілетін деректер кейін бірнеше есептеуге қатысады. Duration мәні startTime арқылы endTime есептеуге көмектеседі, attendees зал capacity мәнімен салыстырылады, services және equipment қосымша қажеттіліктерді көрсетеді, paymentMethod төлем сценарийін сипаттайды. Осындай деректердің бір модельге жиналуы әкімшіге өтінімді толық бағалауға мүмкіндік береді.

Күнтізбе және floorplan модульдері ұйымдастырушыға жүктемені визуалды түрде көруге көмектеседі. Күнтізбеде іс-шаралар, зал брондаулары және клиент өтінімдері бір күнге біріктіріліп көрсетіледі. Карта немесе floorplan бөлімі кеңістікті таңдауды түсінікті етеді.

Күнтізбе модулі уақытқа байланысты ақпаратты визуалды ұйымдастыру құралы болып табылады. Тізім түріндегі өтінімдер көп болған кезде әкімші нақты күндегі жүктемені бірден байқамауы мүмкін, ал calendar view оқиғаларды уақыт осінде көруге мүмкіндік береді. Бұл зал қақтығысын алдын ала байқауға, күнді жоспарлауға және event пен hall booking арасындағы байланысты түсінуге көмектеседі.

Floorplan немесе карта модулі кеңістікті мәтіндік тізімнен гөрі түсініктірек көрсетеді. Пайдаланушы залдың қай жерде орналасқанын, сыйымдылығын және ағымдағы күйін визуалды қабылдай алады. Егер жүйе болашақта нақты ғимарат жоспарына немесе интерактивті картаға толық қосылса, зал таңдау процесі одан әрі жеңілдейді. Бұл әсіресе үлкен конференц-орталықтар мен бірнеше қабатты оқу ғимараттары үшін пайдалы.

Жеке кабинетте пайдаланушы өзінің профилін жаңарта алады, іс-шараға жазылуларын, зал өтінімдерін және техникалық қолдау хабарламаларын қарайды. Бұл бөлім клиент пен әкімші арасындағы байланыс арнасын толықтырады.

Жеке кабинет пайдаланушы үшін жүйедегі негізгі бақылау орталығы қызметін атқарады. Мұнда профиль деректері, іс-шараға тіркелулер, зал өтінімдері және хабарламалар бір жерде көрсетіледі. Бұл пайдаланушыға бұрын жіберген өтінімдерін іздеп жүрмеуге, мәртебені өз бетінше тексеруге және қажет болса тіркелуді тоқтатуға мүмкіндік береді. Жеке кабинет жүйенің ашықтығын арттырады.

Профиль деректерін жаңарту ұйыммен байланысты нақтылауға көмектеседі. Phone, organization және city өрістері қатысушыны немесе клиентті кейін тануға, әкімшіге қосымша байланыс жасауға және аудитория құрылымын

талдауға мүмкіндік береді. Мұндай деректерді жинау кезінде артық ақпарат сұрамау және тек жүйе жұмысына қажет мәліметтермен шектелу жеке деректерді қорғау қағидасына сәйкес келеді.

Клиенттік бөлікті іске асыруда басты файлдар ретінде `'index.html'`, `'styles.css'` және `'app.js'` қолданылды. HTML құжат беттердің құрылымын береді, CSS интерфейстің визуалдық көрінісін қалыптастырады, ал JavaScript деректерді жүктеу, формаларды өңдеу және бөлімдер арасында ауысу логикасын орындайды.

Каталог модулі event деректерін серверден алып, оларды карточка түрінде көрсетеді. Әр карточкада іс-шара атауы, күні, форматы, категориясы, орны және тіркелу мүмкіндігі беріледі. Іздеу және сүзгілеу элементтері пайдаланушыға көп жазба арасынан қажеттісін тез табуға көмектеседі.

Конференц-залдар модулі Hall деректеріне сүйенеді. Зал карточкасында сыйымдылық, қабат, орналасқан жер, жабдықтар және баға көрсетіледі. Пайдаланушы нақты залды таңдап, брондау формасын аша алады. Бұл сценарий ресурс басқару міндетін клиенттік деңгейде түсінікті етеді.

Календарь модулі іс-шараларды күн бойынша көруге мүмкіндік береді. Мұндай көрініс ұйымдастырушы үшін де, қатысушы үшін де пайдалы, себебі бір күндегі бірнеше іс-шараны салыстыру жеңілдейді. Календарь іздеу мен сүзгілеу құралдарымен бірге жұмыс істейді.

Frontend формалары пайдаланушы енгізген деректерді бастапқы деңгейде тексереді, бірақ негізгі тексеру серверде орындалады. Бұл қауіпсіздік үшін қажет, себебі браузердегі тексеруді айналып өту мүмкін. Сондықтан frontend ыңғайлылықты, ал backend деректер тұтастығын қамтамасыз етеді.

3.2 Серверлік бөлік және әкімші модулі

Серверлік бөлік Express қосымшасы ретінде құрылған. `'backend/src/app.js'` файлында CORS баптауы, JSON body parser, статикалық frontend тарату, `'/health'` маршруты, API router және error handler қосылған. Мұндай құрылым серверді бір жағынан web app ретінде, екінші жағынан REST API ретінде қолдануға мүмкіндік береді.

Авторизация controller-і email мекенжайын normalize етеді, құпиясөз ұзындығын тексереді, қайталанған email-ді болдырмайды және бірінші пайдаланушыны admin рөлімен тіркейді. Бұл дипломдық демонстрация үшін ыңғайлы, себебі бос базада жеке admin құрудың қосымша скрипті қажет емес.

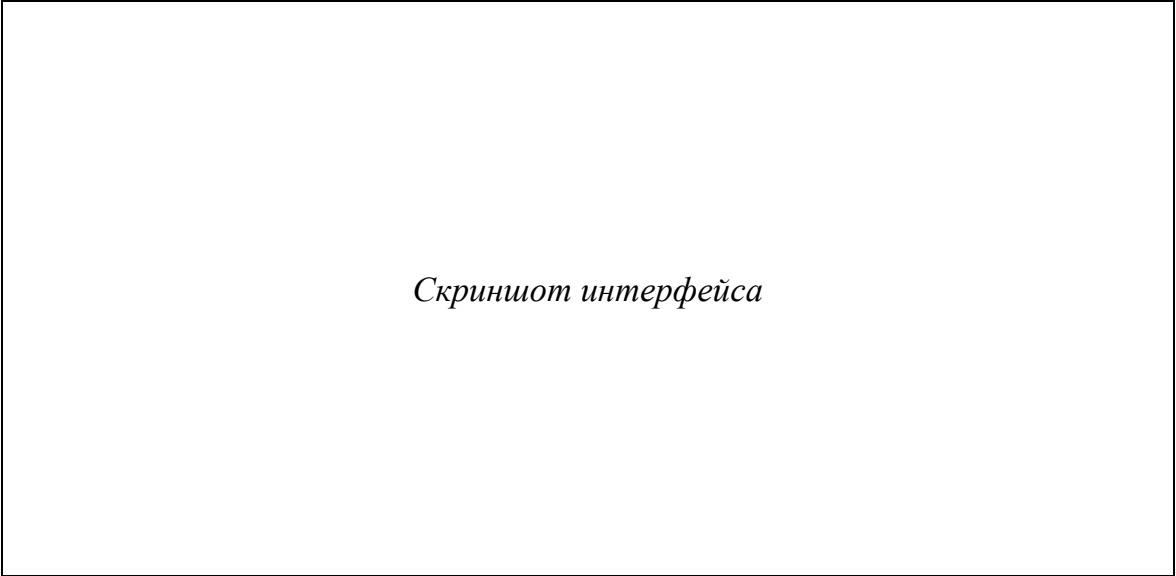
Әкімші өтінімдерді қарап, мәртебесін өзгертіп, статистикалық көрсеткіштерді бақылайды. Әкімшілік панель 9-суретте көрсетілген.



Скриншот интерфейса

Сурет 9. Әкімшілік панель және өтінім мәртебелерін басқару

Пайдаланушы сұрағы support модулі арқылы әкімшіге жеткізіледі. Техникалық қолдау экраны 10-суретте көрсетілген.



Скриншот интерфейса

Сурет 10. Техникалық қолдау хабарламаларының интерфейсі

Брондау логикасында екі маңызды тексеріс бар: біріншісі - пайдаланушының бір іс-шараға екі рет тіркелмеуі, екіншісі - орын санының capacity мәнінен аспауы. Зал брондау кезінде `findScheduleConflicts` функциясы белгілі бір зал, күн және уақыт аралығы бойынша бұрынғы өтінімдер мен іс-шараларды салыстырады. Егер уақыт аралықтары қиылысса, сервер қате қайтарады.

Әкімші модулі жүйені басқарудың орталық бөлігі болып табылады. Әкімші барлық өтінімдерді көреді, мәртебесін `approved`, `rejected`, `cancelled`, `review` сияқты мәндерге өзгерте алады, check-in жасайды, статистиканы қарайды және техникалық қолдау хабарламаларын оқылған деп белгілейді. Егер custom-event өтінімі бекітілсе, жүйе оны Event коллекциясына жарияланған іс-шара ретінде қоса алады.

Серверлік бөлікте controller логикасы нақты бизнес-әрекеттерге бағытталған. Auth controller тіркелу және кіру кезінде пайдаланушыны тексереді; Event controller іс-шараны жасау, жаңарту, жою және тізімдеу міндетін орындайды; Hall controller залдарды басқарады; Booking controller пайдаланушы өтінімдерін жасайды және жеке өтінімдерді қайтарады; Admin controller жалпы бақылау, статистика, мәртебе өзгерту және check-in функцияларын қамтамасыз етеді.

Брондау логикасындағы `'findScheduleConflicts'` функциясы уақыт аралықтарының қиылысуын тексереді. Ол start және end минуттарын есептеп, жаңа өтінімнің бұрынғы booking немесе event уақыттарымен қабаттасуын анықтайды. Қақтығыс шарты `'startA < endB && endA > startB'` түрінде беріледі. Бұл формула екі интервалдың ортақ бөлігі бар-жоғын анықтаудың кең таралған және түсінікті тәсілі болып табылады.

Зал атауларының бірнеше тілде берілуі қақтығысты тексеруді күрделендіреді. Бір зал қазақша, орысша немесе ағылшынша атаумен көрсетілуі мүмкін, сондықтан жүйе hallId ғана емес, hallName, location және translations өрістерін де салыстырады. Бұл көптілді интерфейстің backend логикасына да әсер ететінін көрсетеді: тіл тек мәтін ауыстыру емес, деректерді сәйкестендіру мәселесімен де байланысты.

Admin controller өтінім мәртебесін өзгерткенде бірнеше қосымша әрекет орындай алады. Егер custom-event өтінімі бекітілсе, жүйе оны Event коллекциясына жарияланған іс-шара ретінде қоса алады. Егер өтінім қабылданбаса немесе cancelled болса, төлем мәртебесін refund логикасына байланысты жаңартуға болады. Мұндай байланыстар admin әрекетінің деректер базасында бірнеше объектіге әсер етуі мүмкін екенін көрсетеді.

Check-in функциясы іс-шара күні қатысушының нақты келгенін белгілеуге арналған. Бұл функция event management жүйелерінде жиі қолданылады, себебі тіркелген қатысушылардың барлығы іс-шараға келе бермейді. CheckedIn және checkedInAt өрістері кейін статистика жасауға, қатысу көрсеткішін бағалауға және ұйымдастыру сапасын талдауға мүмкіндік береді.

Error handler серверлік архитектураның соңғы қорғаныс деңгейі ретінде жұмыс істейді. Егер controller ішінде қате шықса, ол қолданушыға түсініксіз stack trace ретінде емес, статус коды мен хабарламасы бар JSON жауап ретінде қайтарылады. Бұл frontend-ке қателерді бірдей форматта өңдеуге мүмкіндік береді және жүйенің тұрақтылығын арттырады.

Backend орналастыру тұрғысынан да икемді. Root `'server.js'` негізгі іске қосу нүктесі ретінде backend серверін шақырады, ал Express қосымшасы frontend статикалық файлдарын да тарата алады. Осылайша жоба бір портта толық веб-қосымша ретінде жұмыс істейді. Кейін production деңгейінде frontend пен backend бөлек орналастырылса да, API логикасы сақталады.

Серверлік бөлік Express қосымшасы ретінде ұйымдастырылды. Негізгі app файлы CORS, JSON parser, static файлдарды тарату және API route-тарын қосады. Бұл қабат барлық сұраныстардың бір кіру нүктесі болып табылады және жүйенің backend тәртібін анықтайды.

Auth controller пайдаланушыны тіркеу және жүйеге кіргізу міндетін атқарады. Тіркелу кезінде email бірегейлігі тексеріледі, құпиясөз хэштеледі, ал сәтті кіргеннен кейін JWT token беріледі. Бұл token кейін қорғалған сұраныстарда қолданылады.

Booking controller іс-шараға тіркелу, зал брондау, төлем мәртебесін жаңарту, өтінімді жою, хабарламаларды оқу және availability тексеру логикасын қамтиды. Бұл controller жүйедегі ең негізгі бизнес-процестерді біріктіреді.

Admin controller әкімші әрекеттерін басқарады. Ол барлық өтінімдерді көруге, пайдаланушылар тізімін алуға, статистика есептеуге, check-in белгілеуге, өтінім мәртебесін өзгертуге және demo деректерді жаңартуға мүмкіндік береді. Бұл модуль ұйым қызметкерінің күнделікті жұмысын автоматтандырады.

Support controller пайдаланушы хабарламаларын қабылдап, admin панельде көрсету үшін қолданылады. Техникалық қолдау модулі пайдаланушы мен ұйымдастырушы арасындағы қосымша байланыс арнасын қалыптастырады және жүйенің қызмет көрсету сапасын арттырады.

3.3 Тестілеу және тәуекелдер

Жүйені тексеру бірнеше деңгейде жүргізілді. Бірінші деңгейде JavaScript файлдарының синтаксисі `npm test` командасы арқылы тексеріледі. Бұл команда root `server.js`, backend сервер файлдары және frontend core модульдері үшін `node --check` орындайды. Екінші деңгейде негізгі пайдаланушы сценарийлері қолмен тексеріледі: тіркелу, кіру, іс-шараға жазылу, залға өтінім беру, әкімші арқылы мәртебені өзгерту және жеке кабинетте нәтижені көру.

Тестілеу барысында негізгі тәуекелдер де анықталды. Біріншісі - MongoDB Atlas қолжетімсіз болса, деректер жүктелмейді. Бұл жағдайда `/health` endpoint арқылы database мәртебесін тексеру және `.env` параметрлерін қарау қажет. Екіншісі - JWT мерзімі аяқталса, пайдаланушы қайта кіруі керек. Үшіншісі - әкімші емес пайдаланушы admin маршрутқа сұраныс жіберсе, сервер оны тоқтатуы тиіс. Төртіншісі - уақыт қақтығысын тексеру барлық зал атаулары мен аудармаларында дұрыс жұмыс істеуі керек.

Тест атауы	Күтілетін нәтиже	Нәтиже
Пайдаланушыны тіркеу	Жаңа аккаунт құрылып, JWT token беріледі	Сәтті
Іс-шараға тіркелу	Booking жазбасы жасалып, жеке кабинетте көрсетіледі	Сәтті
Залды брондау	Уақыт аралығы тексеріліп, өтінім қабылданады	Сәтті
Admin мәртебе өзгертуі	Өтінім күйі жаңарып, клиент хабарлама алады	Сәтті
Көптілді интерфейс	RU/KZ/EN мәтіндері дұрыс ауысады	Сәтті

Жалпы тест нәтижелері жүйенің негізгі дипломдық мақсаттарын орындайтынын көрсетеді. Интерфейс пен API арасындағы байланыс бар, қорғалған маршруттар жұмыс істейді, өтінім мәртебесі сақталады және әкімші бақылауы қамтамасыз етіледі.

Тестілеу кезінде тек сәтті сценарийлерді емес, қате сценарийлерді де тексеру маңызды. Мысалы, бос email арқылы тіркелу, қысқа құпиясөз енгізу, жарамсыз токенмен protected route ашу, бір іс-шараға қайта жазылу, capacity толған event-ке тіркелу және бос емес залға өтінім беру сияқты жағдайлар қарастырылуы тиіс. Мұндай тексерістер жүйенің тұрақтылығын арттырады.

Қауіпсіздік тестілеуі бірнеше бағытты қамтиды. Біріншіден, парольдің ашық мәтінде сақталмауы тексеріледі. Екіншіден, JWT жоқ немесе жарамсыз болған кезде сервер 401 қате қайтаруы керек. Үшіншіден, қарапайым пайдаланушы admin маршрутка кірсе, 403 деңгейіндегі тыйым орындалуы тиіс. Төртіншіден, CORS баптауы production режимінде рұқсат етілмеген origin-дерге шектеу қоя алуы керек.

Деректер қоры тәуекелдері де бөлек бағаланды. MongoDB Atlas қолжетімсіз болса, events, halls және bookings жүктелмейді, сондықтан health endpoint арқылы database мәртебесін тексеру маңызды. Connection string пен JWT_SECRET сияқты құпия мәндер '.env' файлында сақталуы керек және репозиторийге жарияланбауы тиіс. Бұл шағын дипломдық жоба үшін де ақпараттық қауіпсіздік мәдениетін қалыптастырады.

Frontend пен backend арасындағы интеграцияны тексеру кезінде API жауаптарының интерфейсте дұрыс көрінуі қаралады. Мысалы, event тізімі келгенде карточкалар жаңаруы, booking жасалған соң жеке кабинетте жаңа жазба пайда болуы, admin мәртебені өзгерткенде пайдаланушыға notification көрінуі қажет. Егер backend дұрыс жауап берсе, бірақ frontend оны көрсетпесе, пайдаланушы үшін жүйе толық жұмыс істемейді.

Өнімділік тұрғысынан дипломдық жоба шағын және орта көлемдегі деректермен жұмыс істеуге бағытталған. Дегенмен болашақта events және bookings саны көбейсе, MongoDB query тиімділігі, индекстер, pagination және кэштеу маңызды болады. Қазіргі архитектура бұл бағыттарға дайын, себебі API фильтрлері мен деректер модельдері бөлек ұйымдастырылған.

Тестілеу нәтижелерін талдау жүйені әрі қарай жетілдіру бағыттарын көрсетті. Автоматты unit және integration тесттер қосу, API contract

құжаттамасын кеңейту, browser-based end-to-end тесттер енгізу, accessibility audit жүргізу және қауіпсіздік сканерін пайдалану production деңгейіне жақындау үшін маңызды. Осылайша тестілеу тек қате табу емес, жүйенің даму жоспарын нақтылау құралы ретінде қарастырылды.

Тестілеу жүйенің тек сәтті сценарийлерін емес, қате жағдайларын да қамтуы тиіс. Мысалы, бос email арқылы тіркелу, жарамсыз token жіберу, admin маршрутын қарапайым пайдаланушымен ашу, бос емес залға өтінім беру сияқты жағдайлар тексеріледі.

Frontend деңгейінде синтаксистік қателерді анықтау үшін ``npm test`` командасы қолданылды. Бұл команда негізгі JavaScript файлдарының дұрыс жазылғанын тексереді. Мұндай тексеру браузер ашпай тұрып-ақ бастапқы кодтағы қателерді табуға мүмкіндік береді.

Backend деңгейінде endpoint жауаптары, status code мәндері, JSON құрылымы және middleware жұмысы қарастырылады. Auth middleware token жоқ кезде 401 қатесін, adminOnly middleware құқық жеткіліксіз болғанда 403 қатесін қайтаруы керек.

Деректер қорымен байланысты тәуекелдер де маңызды. MongoDB Atlas қолжетімсіз болса, events, halls және bookings деректері жүктелмейді. Сондықтан health endpoint арқылы database күйін тексеру және ``.env`` параметрлерін дұрыс сақтау қажет.

Тестілеу нәтижелері жүйенің негізгі дипломдық мақсаттарды орындайтынын көрсетті. Дегенмен болашақта автоматты unit test, integration test және end-to-end test сценарийлерін қосу жүйені production деңгейіне жақындатады.

3.4 Практикалық және экономикалық тиімділік

Жобаның практикалық тиімділігі ұйымдастырушының уақытын үнемдеумен және деректерді бір ортаға жинаумен байланысты. Бұрын қатысушы тізімі, зал кестесі, төлем белгісі және клиент хабарламалары бөлек құралдарда сақталуы мүмкін еді. ORDA жүйесінде бұл ақпарат бір деректер қорында орналасады және интерфейс арқылы тез табылады.

Экономикалық тұрғыдан жүйе дайын коммерциялық платформаны толық алмастыруды мақсат етпейді, бірақ оқу орны немесе шағын конференц-орталық үшін бастапқы автоматтандыру құралы бола алады. Жоба ашық технологиялар негізінде жасалғандықтан, лицензиялық шығын азаяды, ал серверді Render, VPS немесе ішкі инфрақұрылымда орналастыруға болады.

Жүйенің кеңейту мүмкіндігі де маңызды. Болашақта QR check-in, нақты төлем жүйесімен интеграция, email/SMS хабарламалары, аналитикалық графиктер, экспорт, push-хабарлама, залдардың 3D картасы және role-based permission жүйесін тереңдету сияқты мүмкіндіктер қосуға болады.

Практикалық тұрғыдан ORDA жүйесі ұйымдастырушының бірнеше күнделікті міндетін қысқартады. Қатысушы тізімін қолмен жинау, залдың бос уақытын бөлек тексеру, өтінім мәртебесін мессенджер арқылы түсіндіру және статистиканы қолмен есептеу орнына жүйе бұл ақпаратты бір ортада сақтайды.

Нәтижесінде әкімші уақыты үнемделеді, ал пайдаланушы өз өтінімінің күйін өз бетінше бақылай алады.

Экономикалық тиімділік тек лицензиялық шығынды азайтумен шектелмейді. Жүйе қызмет көрсету сапасын арттыру арқылы да пайда береді: қателер азаяды, өтінім жоғалмайды, зал қақтығысы алдын ала анықталады, қатысушылар саны нақты бақыланады. Мұндай нәтижелер ұйымның беделіне әсер етеді, себебі қатысушы тіркелу және келу процесін ыңғайлы деп қабылдайды.

Жоба ашық веб-технологияларға сүйенгендіктен, оны қолдау үшін сирек немесе жабық платформа қажет емес. HTML, CSS, JavaScript, Node.js, Express.js және MongoDB кең таралған технологиялар қатарына жатады, олардың құжаттамасы мол және маман табу салыстырмалы түрде жеңіл. Бұл жүйенің ұзақ мерзімді қолдау мүмкіндігін арттырады.

Масштабталу бірнеше деңгейде қарастырылуы мүмкін. Қолданушылар саны артса, backend серверін қуаттырақ ортаға көшіру немесе бірнеше instance арқылы іске қосу қажет болуы мүмкін. Деректер көлемі өссе, MongoDB Atlas кластерін кеңейту, индекстер құру және сұраныстарды оңтайландыру қолданылады. Frontend статикалық файлдарын CDN немесе жеке hosting арқылы тарату жүктемені азайтады.

Перспективалық даму бағыттарының бірі - нақты төлем жүйелерімен интеграция. Қазіргі жүйеде paymentStatus және paymentMethod сақталады, бұл болашақта Kaspi, bank card немесе басқа төлем провайдерімен қосылуға негіз болады. Төлем интеграциясы іске қосылған жағдайда webhook өңдеу, төлем қауіпсіздігі, транзакция тарихы және refund логикасы бөлек модуль ретінде жобалануы керек.

QR check-in функциясы да практикалық маңызы жоғары кеңейту бағыты. Қатысушы тіркелген соң QR код алса, іс-шара күні әкімші оны сканерлеп, checkedIn мәртебесін автоматты түрде белгілей алады. Бұл үлкен конференцияларда кезекті азайтады және нақты келушілер статистикасын жылдам жинауға мүмкіндік береді.

Аналитикалық модуль қосылған жағдайда әкімші қай іс-шараға қанша адам тіркелгенін, қандай залдар жиі брондалатынын, қандай формат сұранысқа ие екенін және өтінімдердің қандай бөлігі бекітілетінін көре алады. Мұндай деректер келесі іс-шараларды жоспарлауға, баға саясатын өзгертуге және зал ресурстарын тиімді бөлуге көмектеседі.

Жүйені дамытуда accessibility бағытына да көңіл бөлу қажет. Веб-интерфейс пернетақтамен басқаруға қолайлы болуы, түстер контрасты жеткілікті болуы, форма өрістерінде түсінікті label болуы және қате хабарламалары экран оқитын құралдарға да қолжетімді берілуі тиіс. WCAG қағидаларын ескеру жүйені тек техникалық тұрғыдан емес, әлеуметтік қолжетімділік тұрғысынан да сапалы етеді.

Интеграция мүмкіндіктері ORDA жүйесінің қолданылу аясын кеңейтеді. Болашақта Google Calendar немесе Outlook Calendar арқылы іс-шараны жеке күнтізбеге қосу, email арқылы растау хатын жіберу, Telegram немесе WhatsApp хабарламаларын қосу, CSV/Excel экспорт жасау және бухгалтерлік есеп

жүйелерімен байланысу сияқты функциялар енгізілуі мүмкін. Мұндай интеграциялар жүйені ұйымның басқа цифрлық құралдарымен байланыстырып, оның практикалық құндылығын арттырады.

Жобаның практикалық құндылығы оның аяқталған күйімен ғана емес, кеңейтуге дайын архитектурасымен де анықталады. Модульдік backend, бөлек frontend helper-лер, Mongoose модельдері және REST API келісімі жаңа функцияларды салыстырмалы түрде қауіпсіз қосуға мүмкіндік береді. Сондықтан ORDA жүйесін оқу жобасы ретінде ғана емес, нақты ұйымға бейімделетін бастапқы платформа ретінде қарастыруға болады.

Жобаның практикалық тиімділігі ұйымдастырушының жұмыс уақытын үнемдеуінен көрінеді. Өтінімдерді бір тізімнен қарау, мәртебені батырма арқылы өзгерту, check-in белгілеу және статистика алу қолмен жүргізілетін жұмыстарды азайтады.

Пайдаланушы үшін тиімділік ақпараттың қолжетімді болуымен байланысты. Ол іс-шаралар каталогын қарайды, залдарды салыстырады, өтінім жібереді және нәтижесін жеке кабинеттен көреді. Бұл қызметтің ашықтығын арттырады.

Экономикалық тұрғыдан жүйе қымбат лицензиялық құралдарды қажет етпейді. Ашық веб-технологиялар, Node.js, Express.js және MongoDB Atlas-тың бастапқы тегін мүмкіндіктері шағын ұйымдарға жобаны төмен шығынмен іске қосуға мүмкіндік береді.

Жүйенің даму мүмкіндігі жоғары. Болашақта QR check-in, email хабарлама, SMS сервис, онлайн төлем, PDF есептер, зал жүктемесін талдау және сыртқы календарьмен синхрондау функцияларын қосуға болады.

Осылайша ORDA Smart Event System дипломдық жоба ретінде ғана емес, нақты ұйымда енгізуге болатын бастапқы платформа ретінде де бағаланады. Оның архитектурасы жаңа функциялар қосуға, интерфейсі пайдаланушыға түсінікті жұмыс істеуге бағытталған.

4 ЕҢБЕКТІ ҚОРҒАУ

4.1 Компьютермен жұмыс істеу қауіпсіздігі

Веб-жүйені әзірлеу процесі тікелей өндірістік жабдықпен емес, компьютерлік техникамен, бағдарламалық құралдармен және интернет қызметтерімен байланысты. Сондықтан еңбекті қорғау бөлімінде бағдарламашының жұмыс орнына қойылатын санитарлық-гигиеналық, эргономикалық және ақпараттық қауіпсіздік талаптары қарастырылады.

Бағдарламалық өнімді жасау кезінде қызметкер ұзақ уақыт монитор алдында отырады, мәтіндік редакторлармен, браузермен, серверлік консольмен, MongoDB Atlas басқару панелімен және тестілеу құралдарымен жұмыс істейді. Мұндай еңбек түрінде көзге түсетін жүктеме, омыртқаға түсетін статикалық күш және қол буындарының қайталанатын қозғалысы негізгі тәуекелдер қатарына жатады.

Жұмыс орны жеткілікті жарықтандырылуы тиіс. Табиғи жарық сол жақтан түскені дұрыс, ал жасанды жарық көздері мониторда шағылыс тудырмауы қажет. Экран жарықтығы бөлме жарығымен теңестіріліп, өте ашық немесе өте күңгірт болмауы керек. Бұл көздің шаршауын азайтады және кодпен ұзақ жұмыс істеу кезінде қателердің алдын алады.

Монитор мен көз арасындағы арақашықтық шамамен 50-70 см болуы ұсынылады. Экранның жоғарғы бөлігі пайдаланушының көз деңгейінен сәл төмен орналасқанда мойын бұлшықеттеріне түсетін күш азаяды. Пернетақта мен тінтуір жұмысшының қолы табиғи қалыпта тұратындай деңгейде орналасуы керек.

Әрбір 45-60 минут жұмыс істегеннен кейін қысқа үзіліс жасау, көзге арналған жаттығулар орындау және дене қалпын өзгерту ұсынылады. Бұл талаптар дипломдық жобаны әзірлеу кезінде де маңызды, себебі бағдарламалау, интерфейсті тексеру және құжаттама дайындау бірнеше сағаттық үздіксіз жұмысқа әкелуі мүмкін.

Электр қауіпсіздігі тұрғысынан компьютер, маршрутизатор және қосымша құрылғылар жарамды розеткаға қосылып, сымдар зақымданбауы тиіс. Жұмыс орнында сұйықтықты электр құрылғыларына жақын қоюға болмайды. Құрылғылардың қызып кетуін болдырмау үшін желдеткіш тесіктері жабылмауы қажет.

Ақпараттық қауіпсіздік те еңбекті қорғаудың заманауи бөлігі ретінде қарастырылады. Жоба `.env` файлында сақталатын MongoDB Atlas қосылу жолын және JWT құпия кілтін қолданады. Бұл деректер ашық репозиторийге шығарылмауы, бөтен адамдарға берілмеуі және жұмыс аяқталғаннан кейін қауіпсіз сақталуы тиіс.

4.2 Жұмыс орнын ұйымдастыру талаптары

Жұмыс орнын дұрыс ұйымдастыру жүйені әзірлеушінің өнімділігіне тікелей әсер етеді. ORDA Smart Event System жобасында frontend және backend бөліктері қатар қарастырылғандықтан, әзірлеуші бір уақытта код редакторын, браузерді, терминалды, деректер қоры панелін және құжаттама файлдарын

қолданады. Сондықтан экран кеңістігін, файл құрылымын және жұмыс құралдарын реттеу маңызды.

Құжаттарды, кодты және тест нәтижелерін бір папка құрылымында сақтау жобаның бақылануын жеңілдетеді. Бұл жұмыста backend бөлігі `backend/src` каталогында, frontend файлдары `docs` каталогында, ал құжаттама мен қорғау материалдары бөлек файлдарда орналасқан. Мұндай құрылым жобаны әрі қарай дамытуға ыңғайлы.

Жұмыс орнының үстелі таза, қажет емес қағаздар мен кабельдерден бос болуы керек. Құрылғылардың орналасуы пайдаланушының жиі қолданатын әрекеттерін жылдам орындауына мүмкіндік беруі тиіс. Мысалы, тестілеу кезінде браузер мен терминалдың қатар орналасуы сервер қатесін тез көруге көмектеседі.

Бағдарламашының креслосы биіктігі реттелетін, арқалық тірегі бар болуы қажет. Аяқ еденге толық тиіп тұруы немесе арнайы тірек қолданылуы тиіс. Дене қалпы тұрақты болғанда ұзақ жұмыс кезінде шаршау азаяды және назар бағдарламалық логикаға шоғырланады.

Жүйені әзірлеу кезінде психологиялық жүктеме де ескеріледі. Қате іздеу, серверлік endpoint-терді тексеру, JWT авторизациясын баптау немесе деректер қоры байланысын түзету кейде ұзақ уақыт алады. Мұндай жағдайда тапсырмаларды шағын кезеңдерге бөлу, нұсқаларды сақтау және тест нәтижелерін жазып отыру тиімді.

Еңбекті қорғау талаптарын сақтау тек әзірлеушінің денсаулығын қорғап қана қоймай, бағдарламалық өнімнің сапасын арттырады. Дұрыс ұйымдастырылған ортада кодтағы қателер азаяды, интерфейсті тексеру жүйелі өтеді және дипломдық жобаның қорытынды нәтижесі тұрақты болады.

4.3 Ақпараттық қауіпсіздік және деректерді қорғау

ORDA Smart Event System пайдаланушылардың аты-жөні, электрондық поштасы, ұйымы, қаласы, телефон нөмірі, іс-шараға қатысу өтінімі және залды брондау мәліметтері сияқты деректерді өңдейді. Сондықтан жүйеде қауіпсіздік талаптары тек техникалық функция емес, пайдаланушы сенімінің негізгі шарты ретінде қарастырылады.

Авторизация JWT токени арқылы ұйымдастырылған. Пайдаланушы жүйеге кіргеннен кейін сервер қол қойылған token береді, ал қорғалған сұраныстарда бұл token `Authorization: Bearer` тақырыбы арқылы жіберіледі. Сервер token жарамды екенін тексергеннен кейін ғана жеке кабинет, өтінімдер немесе әкімші функциялары ашылады.

Әкімші құқықтары бөлек middleware арқылы тексеріледі. Бұл тәсіл қарапайым пайдаланушының іс-шараны жоюына, басқа адамдардың өтінімдерін өзгертуіне немесе статистикалық деректерді көруіне жол бермейді. Рөлдік қолжетімділік ақпараттық жүйенің тұтастығын сақтайды.

Құпия сөздер ашық мәтін түрінде сақталмауы керек. Жобада парольді хэштелген түрде сақтау қағидасы қарастырылған, ал `.env` файлындағы құпия кілттер репозиторийге енгізілмейді. Бұл MongoDB Atlas қосылу жолын және JWT құпиясын қорғауға мүмкіндік береді.

Деректерді қорғаудың тағы бір бағыты - енгізілетін ақпаратты тексеру. Міндетті өрістердің толтырылуы, ObjectId мәндерінің дұрыстығы, өтінім мәртебелерінің рұқсат етілген тізімге сәйкес болуы серверлік деңгейде бақыланады. Мұндай тексерулер қате немесе зиянды сұраныстардан қорғайды.

Жүйе болашақта HTTPS, rate limiting, аудит журналдары, резервтік көшіру және хабарлама сервистерімен толықтырылуы мүмкін. Бұл шаралар нақты ұйым жағдайында пайдаланушы деректерінің қауіпсіздігін күшейтіп, веб-жүйенің сенімділігін арттырады.

5 ЭКОНОМИКАЛЫҚ БӨЛІМ

5.1 Жобаның экономикалық және әлеуметтік мәні

Іс-шаралар мен конференцияларды басқару саласында ақпаратты қолмен өңдеу көп уақыт алады. Қатысушылар тізімін бөлек файлда жүргізу, залдың бос уақытын телефон арқылы нақтылау, өтінім мәртебесін қолмен хабарлау және статистиканы бөлек есептеу ұйымдастырушының еңбек шығынын көбейтеді. ORDA Smart Event System осы процестерді бір веб-жүйеге біріктіру арқылы уақытты үнемдейді.

Жүйенің экономикалық мәні бірнеше бағытта көрінеді. Біріншіден, іс-шараға тіркелу және залды брондау өтінімдері автоматты түрде сақталады. Екіншіден, әкімші бір панель арқылы өтінімдерді қарап, мәртебесін өзгерте алады. Үшіншіден, пайдаланушы жеке кабинетінде өз өтінімдерін көріп, қосымша байланыс шығынын азайтады.

Әлеуметтік тұрғыдан жүйе оқу орындары, конференц-орталықтар, бизнес алаңдары және қоғамдық ұйымдар үшін қолжетімді цифрлық құрал ұсынады. Пайдаланушы іс-шаралар туралы ақпаратты бір жерден табады, тілін таңдай алады, қажет залды брондайды және техникалық қолдау арқылы сұрақ қояды. Бұл қызмет сапасын арттырады.

Жүйенің кеңейтілу мүмкіндігі де экономикалық артықшылық береді. Node.js және Express.js негізіндегі модульдік backend жаңа endpoint қосуға ыңғайлы, ал MongoDB Atlas бұлттық деректер қоры серверлік инфрақұрылымды бөлек ұстау қажеттілігін азайтады. Сондықтан алғашқы нұсқаны аз шығынмен іске қосып, кейін нақты ұйым талабына қарай кеңейтуге болады.

Дайын жүйені қолдану қағаз құжат айналымын азайтады, ақпаратты қайталап енгізуді қысқартады және әкімші жұмысын жүйелейді. Бұл тікелей қаржылық үнеммен қатар, қызметкерлер уақытын тиімді бөлуге мүмкіндік береді.

5.2 Жобаны әзірлеу шығындарын есептеу

Дипломдық жоба оқу мақсатына арналғандықтан, көптеген бағдарламалық құралдар тегін немесе ашық лицензиямен қолданылды. HTML, CSS, JavaScript, Node.js, Express.js және MongoDB драйверлері үшін бөлек лицензиялық төлем қажет емес. Негізгі шығындар әзірлеушінің уақытына, компьютер амортизациясына, интернетке және бұлттық қызметтердің ықтимал тарифтеріне байланысты есептеледі.

Есептеу кезінде екі айлық әзірлеу кезеңі алынды. Бұл кезеңге талаптарды талдау, интерфейс жасау, backend логикасын жазу, MongoDB Atlas-пен байланыстыру, тестілеу, құжат дайындау және қорғауға арналған демонстрациялық сценарийді тексеру кіреді.

№	Шығын атауы	Есептеу жолы	Сума
1	Компьютер амортизациясы	350 000 тг / 60 ай × 2 ай	11 666 тг
2	Интернет шығыны	7 000 тг × 2 ай	14 000 тг
3	Электр энергиясы	орташа 2 500 тг × 2 ай	5 000 тг
4	Бұлттық деректер қоры	MongoDB Atlas free tier	0 тг
5	Домен және хостинг резерві	демонстрациялық орналастыруға шартты шығын	15 000 тг
6	Құжаттама және басып шығару	дипломдық материалдарды дайындау	8 000 тг
	Барлығы		53 666 тг

Кестеде көрсетілген шығындар жобаның оқу және демонстрациялық нұсқасына арналған. Нақты ұйымға енгізу кезінде домен, тұрақты сервер, SMS немесе email хабарлама сервисі, техникалық сүйемелдеу және резервтік көшіру шығындары бөлек есептелуі мүмкін.

Жобаның бастапқы құны төмен болуы оның технологиялық стек таңдауымен байланысты. Ашық веб-технологиялар мен бұлттық деректер қоры шағын ұйымдарға жүйені үлкен инфрақұрылымсыз іске қосуға мүмкіндік береді.

5.3 Жобаның тиімділігі және даму мүмкіндіктері

ORDA Smart Event System енгізілген жағдайда ұйымдастырушының негізгі уақыты өтінімдерді іздеу, қатысушыларды салыстыру және залдың бос уақытын тексеру сияқты қайталанатын әрекеттерден босайды. Әкімші өтінімдерді бір тізімнен қарап, сүзгілеу арқылы қажетті жазбаны жылдам табады.

Пайдаланушы үшін тиімділік жеке кабинет пен хабарлама жүйесі арқылы байқалады. Ол тіркелген іс-шараларын, залға берген өтінімдерін және мәртебе өзгерістерін бір жерде көреді. Бұл ақпараттың жоғалуын азайтып, қызмет сапасын жақсартады.

Жүйе болашақта QR check-in, email/SMS хабарламалар, онлайн төлем, аналитикалық есептер, зал жүктемесін болжау және сыртқы күнтізбелермен синхрондау модульдерімен толықтырылуы мүмкін. Осы кеңейтулер енгізілген жағдайда платформа тек дипломдық жоба емес, нақты ұйымға арналған толыққанды басқару құралына айналады.

Экономикалық бөлімнің қорытындысы бойынша, жобаны әзірлеуге кететін бастапқы шығын салыстырмалы түрде төмен, ал процестерді автоматтандырудан алынатын уақыт үнемі жоғары. Сондықтан веб-жүйе практикалық қолдануға жарамды және одан әрі дамытуға перспективалы деп бағаланады.

ҚОРЫТЫНДЫ

Дипломдық жұмыста іс-шаралар мен конференцияларды тіркеу және басқаруға арналған ORDA Smart Event System веб-жүйесі қарастырылды. Жұмыстың негізгі мақсаты - іс-шараларды жариялау, қатысушыларды тіркеу, конференц-залдарды брондау, өтінімдерді өңдеу және әкімшілік бақылау процестерін бір ақпараттық ортада ұйымдастыру болды.

Зерттеу барысында іс-шараларды басқару саласындағы цифрландырудың маңызы талданып, пайдаланушы, қатысушы және әкімші сценарийлері анықталды. Жүйенің функционалдық және функционалдық емес талаптары белгіленіп, клиент-сервер архитектурасы, REST API логикасы және MongoDB Atlas деректер қорының құрылымы сипатталды.

Жоба frontend бөлігінде HTML, CSS және JavaScript технологияларын, backend бөлігінде Node.js және Express.js платформасын қолданады. Деректерді сақтау үшін MongoDB Atlas және Mongoose модельдері пайдаланылды. Авторизация JWT token арқылы ұйымдастырылып, әкімші функциялары рөлдік қолжетімділікпен қорғалды.

Жүйеде іс-шаралар каталогы, конференц-залдар тізімі, календарь, пайдаланушы профилі, іс-шараға тіркелу, залды брондау, өтінім мәртебесін бақылау, хабарламалар, техникалық қолдау және әкімшілік панель сияқты негізгі модульдер іске асырылды. Көптілді интерфейс қазақ, орыс және ағылшын тілдерінде жұмыс істеуге мүмкіндік береді.

Тестілеу нәтижесінде негізгі пайдаланушы сценарийлері орындалатыны анықталды: тіркеу, жүйеге кіру, іс-шараны таңдау, залға өтінім беру, admin панель арқылы мәртебе өзгерту және жеке кабинетте нәтижені көру. Бұл жүйенің дипломдық жоба деңгейінде толық жұмыс істейтінін көрсетеді.

Еңбекті қорғау бөлімінде компьютермен жұмыс істеу қауіпсіздігі, жұмыс орнын ұйымдастыру және ақпараттық қауіпсіздік талаптары қарастырылды. Экономикалық бөлімде жобаның бастапқы шығындары мен практикалық тиімділігі есептелді.

Қорытындылай келе, әзірленген веб-жүйе оқу орындары, конференц-орталықтар және ұйымдар үшін іс-шараларды басқару процесін жеңілдететін заманауи ақпараттық құрал болып табылады. Жоба болашақта онлайн төлем, QR check-in, email/SMS хабарламалар және кеңейтілген аналитика модульдерімен толықтырылуы мүмкін.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР

1. Node.js Documentation. URL: <https://nodejs.org/docs>
2. Express.js Guide. URL: <https://expressjs.com>
3. MongoDB Atlas Documentation. URL: <https://www.mongodb.com/docs/atlas>
4. Mongoose Documentation. URL: <https://mongoosejs.com/docs>
5. JSON Web Token Introduction. URL: <https://jwt.io/introduction>
6. MDN Web Docs: HTML, CSS және JavaScript негіздері. URL: <https://developer.mozilla.org>
7. REST API Design Best Practices. Microsoft Learn, 2025.
8. OWASP Web Security Testing Guide. URL: <https://owasp.org>
9. Қазақстан Республикасының ақпараттандыру туралы заңнамалық актілері.
10. Ақпараттық жүйелерді жобалау негіздері: оқу құралы. Алматы, 2022.
11. Веб-қосымшаларды әзірлеу технологиялары: оқу құралы. Астана, 2023.
12. Бұлттық деректер қорларын қолдану тәжірибесі. Техникалық шолу, 2024.
13. User Experience Design Principles for Web Applications. Nielsen Norman Group, 2024.
14. Agile Web Development және бағдарламалық өнімді тестілеу әдістері. Оқу материалы, 2023.
15. ORDA Smart Event System жобасының бастапқы коды және ішкі құжаттамасы, 2026.

ҚОСЫМША А. БАҒДАРЛАМАНЫҢ НЕГІЗГІ КОДТАРЫ

Бұл қосымшада ORDA Smart Event System жобасының негізгі бағдарламалық кодтары берілді. Код үзінділері жүйенің серверлік қосымшасын, деректер қоры модельдерін, маршруттарын, авторизация middleware қабатын, брондау логикасын және frontend helper модульдерін сипаттайды.

Кодтар дипломдық жұмыстың жалпы стилін сақтай отырып құрылымдалды: алдымен файл атауы көрсетіледі, содан кейін жол нөмірлері бар негізгі бөлік беріледі. Мұндай рәсімдеу жобаның архитектурасын қорғау кезінде түсіндіруге және нақты модульдердің қызметін көрсетуге ыңғайлы.

A.1 Сервердің негізгі іске қосылу бөлігі

// Файл: server.js

```
0001 require("./backend/server");
```

A.2 Express қосымшасының конфигурациясы

// Файл: backend/src/app.js

```
0001 const express = require("express");
0002 const cors = require("cors");
0003 const path = require("path");
0004 const { dbStatus } = require("./config/db");
0005 const { config } = require("./config/env");
0006 const createApiRouter = require("./routes");
0007 const { notFound, errorHandler } = require("./middleware/errorHandler");
0008
0009 const app = express();
0010 const staticDir = path.join(__dirname, "..", "..", "docs");
0011
0012 function splitOrigins(value) {
0013   return String(value || "")
0014     .split(",")
0015     .map((origin) => origin.trim())
0016     .filter(Boolean);
0017 }
0018
0019 const allowedOrigins = new Set([
0020   "http://localhost:3000",
0021   "http://127.0.0.1:3000",
0022   "http://localhost:5500",
0023   "http://127.0.0.1:5500",
0024   ...splitOrigins(config.frontendOrigin),
0025 ]);
0026 const githubPagesOrigin = /^https:\/\/[a-z0-9-]+\.\github\.io$/i;
0027
0028 app.use(
0029   cors({
0030     origin(origin, callback) {
0031       if (!origin) return callback(null, true);
0032       if (allowedOrigins.has(origin) || githubPagesOrigin.test(origin)) return callback(null, true);
0033       if (config.nodeEnv !== "production") return callback(null, true);
0034       return callback(new Error("Origin is not allowed by CORS"));
0035     },
0036   })
0037 );
0038 app.use(express.json({ limit: "5mb" }));
0039
0040 app.use((req, res, next) => {
```

```

0041 if (req.path.endsWith(".js") || req.path.endsWith(".css") || req.path === "/") {
0042   res.setHeader("Cache-Control", "no-store");
0043 }
0044 next();
0045 });
0046
0047 app.use(express.static(staticDir));
0048
0049 app.get("/health", (req, res) => {
0050   res.json({
0051     status: "ok",
0052     brand: "ORDA Smart Event System",
0053     database: dbStatus(),
0054   });
0055 });
0056
0057 app.use(createApiRouter());
0058 app.use("/api", createApiRouter());
0059
0060 app.use((req, res, next) => {
0061   const accept = String(req.get("accept") || "");
0062   if (req.method === "GET" && accept.includes("text/html")) {
0063     return res.sendFile(path.join(staticDir, "index.html"));
0064   }
0065
0066   return next();
0067 });
0068
0069 app.use(notFound);
0070 app.use(errorHandler);
0071
0072 module.exports = app;

```

A.3 Орта айнымалылары және MongoDB қосылымы

```

// Файл: backend/src/config/env.js
0001 const fs = require("fs");
0002 const path = require("path");
0003
0004 function loadEnvFile() {
0005   const envPath = path.join(__dirname, "..", "..", "..", ".env");
0006
0007   if (!fs.existsSync(envPath)) {
0008     return;
0009   }
0010
0011   const lines = fs.readFileSync(envPath, "utf8").split(/\r?\n/);
0012
0013   for (const line of lines) {
0014     const trimmed = line.trim();
0015     if (!trimmed || trimmed.startsWith("#")) continue;
0016
0017     const separatorIndex = trimmed.indexOf("=");
0018     if (separatorIndex === -1) continue;
0019
0020     const key = trimmed.slice(0, separatorIndex).trim();
0021     const value = trimmed.slice(separatorIndex + 1).trim();
0022
0023     if (!process.env[key]) {
0024       process.env[key] = value;
0025     }

```

```

0026 }
0027 }
0028
0029 loadEnvFile();
0030
0031 function isPlaceholderMongoUri(value) {
0032   return (
0033     !value ||
0034     /username:password/i.test(value) ||
0035     /cluster\.mongodb\.net/i.test(value) ||
0036     /<your-mongodb-atlas-uri>/i.test(value)
0037   );
0038 }
0039
0040 const mongodbUri = process.env.MONGODB_URI || "";
0041 const nodeEnv = process.env.NODE_ENV || "development";
0042
0043 const config = {
0044   port: process.env.PORT || 3000,
0045   nodeEnv,
0046   jwtSecret: process.env.JWT_SECRET,
0047   mongodbUri,
0048   mongodbEnabled: !isPlaceholderMongoUri(mongodbUri),
0049   frontendOrigin: process.env.FRONTEND_ORIGIN || process.env.CORS_ORIGIN || "",
0050 };
0051
0052 function validateConfig() {
0053   const missing = [];
0054   if (!config.jwtSecret) missing.push("JWT_SECRET");
0055   if (config.nodeEnv === "production" && !config.mongodbEnabled) missing.push("MONGODB_URI");
0056
0057   if (missing.length) {
0058     throw new Error(`Missing required environment variables: ${missing.join(", ")}`);
0059   }
0060 }
0061
0062 module.exports = {
0063   config,
0064   validateConfig,
0065 };

```

А.4 Деректер қорына қосылу модулі

```

// Файл: backend/src/config/db.js
0001 const mongoose = require("mongoose");
0002 const { config } = require("./env");
0003
0004 async function connectDB() {
0005   if (!config.mongodbEnabled) {
0006     console.warn("MongoDB disabled: set a real MONGODB_URI to use the backend database.");
0007     return;
0008   }
0009
0010   await mongoose.connect(config.mongodbUri, {
0011     serverSelectionTimeoutMS: 10000,
0012   });
0013   console.log("MongoDB connected");
0014 }
0015
0016 function isDbReady() {
0017   return mongoose.connection.readyState === 1;

```

```

0018 }
0019
0020 function dbStatus() {
0021   if (!config.mongooseEnabled) return "disabled";
0022   return isDbReady() ? "connected" : "connecting";
0023 }
0024
0025 module.exports = {
0026   connectDB,
0027   dbStatus,
0028   isDbReady,
0029 };

```

A.5 User моделі

```

// Файл: backend/src/models/User.js
0001 const mongoose = require("mongoose");
0002
0003 const userSchema = new mongoose.Schema(
0004   {
0005     name: { type: String, required: true, trim: true },
0006     email: { type: String, unique: true, required: true, trim: true, lowercase: true },
0007     password: { type: String, required: true },
0008     role: { type: String, default: "user", enum: ["user", "admin"] },
0009     phone: { type: String, trim: true, default: "" },
0010     organization: { type: String, trim: true, default: "" },
0011     city: { type: String, trim: true, default: "" },
0012   },
0013   { timestamps: true }
0014 );
0015
0016 module.exports = mongoose.model("User", userSchema);

```

A.6 Event моделі

```

// Файл: backend/src/models/Event.js
0001 const mongoose = require("mongoose");
0002
0003 const eventSchema = new mongoose.Schema(
0004   {
0005     title: { type: String, required: true, trim: true },
0006     date: { type: String, required: true },
0007     endDate: { type: String, trim: true, default: "" },
0008     time: { type: String, trim: true, default: "10:00" },
0009     category: { type: String, required: true, trim: true },
0010     description: { type: String, trim: true },
0011     agenda: { type: String, trim: true, default: "" },
0012     organizer: { type: String, trim: true, default: "ORDA" },
0013     location: { type: String, required: true, trim: true },
0014     city: { type: String, trim: true, default: "" },
0015     translations: { type: Object, default: {} },
0016     format: { type: String, default: "offline", enum: ["offline", "online", "hybrid"] },
0017     meetingUrl: { type: String, trim: true, default: "" },
0018     image: { type: String, trim: true, default: "" },
0019     price: { type: Number, default: 0, min: 0 },
0020     currency: { type: String, trim: true, default: "KZT" },
0021     featured: { type: Boolean, default: false },
0022     tags: [{ type: String, trim: true }],
0023     capacity: { type: Number, default: 80, min: 1 },
0024     status: { type: String, default: "published", enum: ["draft", "published", "closed"] },
0025   },
0026   { timestamps: true }

```

```

0027 );
0028
0029 module.exports = mongoose.model("Event", eventSchema);

```

A.7 Hall моделі

```

// Файл: backend/src/models/Hall.js
0001 const mongoose = require("mongoose");
0002
0003 const hallSchema = new mongoose.Schema(
0004   {
0005     name: { type: String, trim: true, required: true },
0006     floor: { type: String, trim: true, required: true },
0007     location: { type: String, trim: true, required: true },
0008     capacity: { type: Number, required: true, min: 1 },
0009     pricePerHour: { type: Number, default: 0, min: 0 },
0010     equipment: { type: [String], default: [] },
0011     description: { type: String, trim: true, default: "" },
0012     translations: { type: Object, default: {} },
0013     status: { type: String, enum: ["available", "busy", "maintenance"], default: "available" },
0014     image: { type: String, trim: true, default: "" },
0015     advantages: { type: [String], default: [] },
0016   },
0017   { timestamps: true }
0018 );
0019
0020 module.exports = mongoose.model("Hall", hallSchema);

```

A.8 Booking моделі

```

// Файл: backend/src/models/Booking.js
0001 const mongoose = require("mongoose");
0002
0003 const bookingSchema = new mongoose.Schema(
0004   {
0005     type: {
0006       type: String,
0007       default: "event",
0008       enum: ["event", "hall", "custom-event"],
0009     },
0010     userId: { type: String, trim: true, default: "" },
0011     userEmail: { type: String, required: true },
0012     userName: { type: String, required: true },
0013     userPhone: { type: String, trim: true, default: "" },
0014     organization: { type: String, trim: true, default: "" },
0015     eventId: {
0016       type: mongoose.Schema.Types.ObjectId,
0017       ref: "Event",
0018       required() {
0019         return this.type === "event";
0020       },
0021     },
0022     eventTitle: { type: String, trim: true, default: "" },
0023     category: { type: String, trim: true, default: "" },
0024     format: { type: String, trim: true, default: "" },
0025     description: { type: String, trim: true, default: "" },
0026     image: { type: String, trim: true, default: "" },
0027     location: { type: String, trim: true, default: "" },
0028     city: { type: String, trim: true, default: "" },
0029     hallId: { type: String, trim: true, default: "" },
0030     hallName: { type: String, trim: true, default: "" },
0031     date: { type: String, trim: true, default: "" },

```

```

0032   time: { type: String, trim: true, default: "" },
0033   startTime: { type: String, trim: true, default: "" },
0034   endTime: { type: String, trim: true, default: "" },
0035   duration: { type: Number, default: 0 },
0036   attendees: { type: Number, default: 0 },
0037   purpose: { type: String, trim: true, default: "" },
0038   adminNotes: { type: String, trim: true, default: "" },
0039   services: { type: Array, default: [] },
0040   equipment: { type: Array, default: [] },
0041   price: { type: Number, default: 0 },
0042   ticketPrice: { type: Number, default: 0 },
0043   pricePerHour: { type: Number, default: 0 },
0044   rentAmount: { type: Number, default: 0 },
0045   amount: { type: Number, default: 0 },
0046   paymentStatus: {
0047     type: String,
0048     default: "free",
0049     enum: ["free", "unpaid", "paid", "refunded"],
0050   },
0051   paymentMethod: { type: String, trim: true, default: "" },
0052   paidAt: { type: Date, default: null },
0053   refundedAt: { type: Date, default: null },
0054   status: {
0055     type: String,
0056     default: "new",
0057     enum: ["new", "review", "pending", "approved", "rejected", "cancelled", "registered"],
0058   },
0059   adminReason: { type: String, trim: true, default: "" },
0060   clientNotificationReadAt: { type: Date, default: null },
0061   checkedIn: { type: Boolean, default: false },
0062   checkedInAt: { type: Date, default: null },
0063   },
0064   { timestamps: true }
0065 );
0066
0067 module.exports = mongoose.model("Booking", bookingSchema);

```

A.9 SupportMessage моделі

```

// Файл: backend/src/models/SupportMessage.js
0001 const mongoose = require("mongoose");
0002
0003 const supportMessageSchema = new mongoose.Schema(
0004   {
0005     userId: { type: String, trim: true, default: "" },
0006     userName: { type: String, trim: true, required: true },
0007     userEmail: { type: String, trim: true, required: true },
0008     text: { type: String, trim: true, required: true },
0009     status: { type: String, enum: ["new", "read"], default: "new" },
0010     readAt: { type: Date, default: null },
0011   },
0012   { timestamps: true }
0013 );
0014
0015 module.exports = mongoose.model("SupportMessage", supportMessageSchema);

```

A.10 JWT авторизация middleware

```

// Файл: backend/src/middleware/auth.js
0001 const jwt = require("jsonwebtoken");
0002 const { config } = require("../config/env");
0003 const ApiError = require("../utils/apiError");

```

```

0004
0005 function auth(req, res, next) {
0006   const token = req.headers.authorization?.split(" ")[1];
0007
0008   if (!token) {
0009     return next(new ApiError(401, "Authorization token is missing."));
0010   }
0011
0012   try {
0013     req.user = jwt.verify(token, config.jwtSecret);
0014     return next();
0015   } catch (err) {
0016     return next(new ApiError(401, "Authorization token is invalid or expired."));
0017   }
0018 }
0019
0020 function adminOnly(req, res, next) {
0021   if (req.user?.role !== "admin") {
0022     return next(new ApiError(403, "Admin access is required."));
0023   }
0024
0025   return next();
0026 }
0027
0028 function optionalAuth(req, res, next) {
0029   const token = req.headers.authorization?.split(" ")[1];
0030
0031   if (!token) {
0032     return next();
0033   }
0034
0035   try {
0036     req.user = jwt.verify(token, config.jwtSecret);
0037   } catch (err) {
0038     req.user = null;
0039   }
0040
0041   return next();
0042 }
0043
0044 module.exports = {
0045   auth,
0046   adminOnly,
0047   optionalAuth,
0048 };

```

A.11 Валидация және қате өңдеу middleware

// Файл: backend/src/middleware/validate.js

```

0001 const ApiError = require("../utils/apiError");
0002
0003 function requireFields(fields) {
0004   return (req, res, next) => {
0005     const missing = fields.filter((field) => {
0006       const value = req.body[field];
0007       return value === undefined || value === null || String(value).trim() === "";
0008     });
0009
0010     if (missing.length) {
0011       return next(new ApiError(400, `Missing required fields: ${missing.join(", ")}`));
0012     }

```

```

0013
0014     return next();
0015 };
0016 }
0017
0018 function validateObjectId(paramName = "id") {
0019     return (req, res, next) => {
0020         const value = req.params[paramName];
0021
0022         if (!/^[a-fd]{24}$/i.test(String(value))) {
0023             return next(new ApiError(400, `Invalid ${paramName}.`));
0024         }
0025
0026         return next();
0027     };
0028 }
0029
0030 module.exports = {
0031     requireFields,
0032     validateObjectId,
0033 };

```

A.12 API маршруттарын біріктіру

```

// Файл: backend/src/routes/index.js
0001 const express = require("express");
0002 const authRoutes = require("./auth.routes");
0003 const userRoutes = require("./user.routes");
0004 const eventRoutes = require("./event.routes");
0005 const hallRoutes = require("./hall.routes");
0006 const bookingRoutes = require("./booking.routes");
0007 const adminRoutes = require("./admin.routes");
0008 const supportRoutes = require("./support.routes");
0009
0010 function createApiRouter() {
0011     const router = express.Router();
0012
0013     router.use(authRoutes);
0014     router.use(userRoutes);
0015     router.use(eventRoutes);
0016     router.use(hallRoutes);
0017     router.use(bookingRoutes);
0018     router.use(supportRoutes);
0019     router.use(adminRoutes);
0020
0021     return router;
0022 }
0023
0024 module.exports = createApiRouter;

```

A.13 Авторизация маршруттары

```

// Файл: backend/src/routes/auth.routes.js
0001 const express = require("express");
0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { requireFields } = require("../middleware/validate");
0005 const { register, login } = require("../controllers/auth.controller");
0006
0007 const router = express.Router();
0008
0009 router.post("/register", dbReady, requireFields(["name", "email", "password"]), asyncHandler(register)

```

```

    ));
0010 router.post("/login", dbReady, requireFields(["email", "password"]), asyncHandler(login));
0011
0012 module.exports = router;

```

А.14 Ис-шаралар маршруттары

```

// Файл: backend/src/routes/event.routes.js
0001 const express = require("express");
0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { auth, adminOnly, optionalAuth } = require("../middleware/auth");
0005 const { requireFields, validateObjectId } = require("../middleware/validate");
0006 const {
0007   getEvents,
0008   getEvent,
0009   createEvent,
0010   updateEvent,
0011   deleteEvent,
0012 } = require("../controllers/event.controller");
0013
0014 const router = express.Router();
0015
0016 router.get("/events", dbReady, optionalAuth, asyncHandler(getEvents));
0017 router.get("/events/:id", dbReady, validateObjectId("id"), optionalAuth, asyncHandler(getEvent));
0018 router.post("/events", dbReady, auth, adminOnly, requireFields(["title", "date", "category",
0019   "location"]), asyncHandler(createEvent));
0019 router.put("/events/:id", dbReady, validateObjectId("id"), auth, adminOnly, requireFields(["title",
0020   "date", "category", "location"]), asyncHandler(updateEvent));
0020 router.delete("/events/:id", dbReady, validateObjectId("id"), auth, adminOnly,
0021   asyncHandler(deleteEvent));
0021
0022 module.exports = router;

```

А.15 Конференц-зал маршруттары

```

// Файл: backend/src/routes/hall.routes.js
0001 const express = require("express");
0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { auth, adminOnly } = require("../middleware/auth");
0005 const { validateObjectId } = require("../middleware/validate");
0006 const {
0007   getHalls,
0008   createHall,
0009   updateHall,
0010   deleteHall,
0011 } = require("../controllers/hall.controller");
0012
0013 const router = express.Router();
0014
0015 router.get("/halls", dbReady, asyncHandler(getHalls));
0016 router.post("/halls", dbReady, auth, adminOnly, asyncHandler(createHall));
0017 router.put("/halls/:id", dbReady, auth, adminOnly, validateObjectId("id"), asyncHandler(updateHall));
0018 router.delete("/halls/:id", dbReady, auth, adminOnly, validateObjectId("id"), asyncHandler(deleteHall
0019   ));
0019
0020 module.exports = router;

```

А.16 Брондау маршруттары

```

// Файл: backend/src/routes/booking.routes.js
0001 const express = require("express");

```

```

0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { auth } = require("../middleware/auth");
0005 const { requireFields, validateObjectId } = require("../middleware/validate");
0006 const {
0007   bookEvent,
0008   getMyBookings,
0009   updateMyBookingPayment,
0010   cancelMyBooking,
0011   createHallBooking,
0012   getMyHallBookings,
0013   updateMyHallBookingPayment,
0014   cancelMyHallBooking,
0015   getMyNotifications,
0016   markMyNotificationRead,
0017   getAvailability,
0018 } = require("../controllers/booking.controller");
0019
0020 const router = express.Router();
0021
0022 router.post("/book", dbReady, auth, requireFields(["eventId"]), asyncHandler(bookEvent));
0023 router.get("/my-bookings", dbReady, auth, asyncHandler(getMyBookings));
0024 router.patch("/my-bookings/:id", dbReady, auth, validateObjectId("id"),
0025   asyncHandler(updateMyBookingPayment));
0026 router.delete("/my-bookings/:id", dbReady, auth, validateObjectId("id"), asyncHandler(cancelMyBooking
0027 ));
0028 router.post("/hall-bookings", dbReady, auth, asyncHandler(createHallBooking));
0029 router.get("/my-hall-bookings", dbReady, auth, asyncHandler(getMyHallBookings));
0030 router.patch("/my-hall-bookings/:id", dbReady, auth, validateObjectId("id"),
0031   asyncHandler(updateMyHallBookingPayment));
0032 router.delete("/my-hall-bookings/:id", dbReady, auth, validateObjectId("id"),
0033   asyncHandler(cancelMyHallBooking));
0034 router.get("/my-notifications", dbReady, auth, asyncHandler(getMyNotifications));
0035 router.patch("/my-notifications/:id/read", dbReady, auth, asyncHandler(markMyNotificationRead));
0036 router.get("/availability", dbReady, asyncHandler(getAvailability));
0037
0038 module.exports = router;

```

A.17 Әкімші маршруттары

// Файл: backend/src/routes/admin.routes.js

```

0001 const express = require("express");
0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { auth, adminOnly } = require("../middleware/auth");
0005 const { validateObjectId } = require("../middleware/validate");
0006 const {
0007   getBookings,
0008   getUsers,
0009   getStats,
0010   checkInBooking,
0011   updateBooking,
0012   updateBookingStatus,
0013   deleteBooking,
0014   seed,
0015 } = require("../controllers/admin.controller");
0016
0017 const router = express.Router();
0018
0019 router.get("/bookings", dbReady, auth, adminOnly, asyncHandler(getBookings));
0020 router.get("/users", dbReady, auth, adminOnly, asyncHandler(getUsers));

```

```

0021 router.get("/stats", dbReady, auth, adminOnly, asyncHandler(getStats));
0022 router.patch("/bookings/:id/check-in", dbReady, auth, adminOnly, validateObjectId("id"),
    asyncHandler(checkInBooking));
0023 router.patch("/bookings/:id/status", dbReady, auth, adminOnly, validateObjectId("id"),
    asyncHandler(updateBookingStatus));
0024 router.patch("/bookings/:id", dbReady, auth, adminOnly, validateObjectId("id"),
    asyncHandler(updateBooking));
0025 router.delete("/bookings/:id", dbReady, auth, adminOnly, validateObjectId("id"),
    asyncHandler(deleteBooking));
0026 router.get("/seed", dbReady, auth, adminOnly, asyncHandler(seed));
0027
0028 module.exports = router;

```

А.18 Техникалық қолдау маршруттары

// Файл: backend/src/routes/support.routes.js

```

0001 const express = require("express");
0002 const asyncHandler = require("../utils/asyncHandler");
0003 const dbReady = require("../middleware/dbReady");
0004 const { auth, adminOnly } = require("../middleware/auth");
0005 const { validateObjectId } = require("../middleware/validate");
0006 const {
0007   createSupportMessage,
0008   getSupportMessages,
0009   markSupportMessageRead,
0010   deleteSupportMessage,
0011 } = require("../controllers/support.controller");
0012
0013 const router = express.Router();
0014
0015 router.post("/support-messages", dbReady, auth, asyncHandler(createSupportMessage));
0016 router.get("/support-messages", dbReady, auth, adminOnly, asyncHandler(getSupportMessages));
0017 router.patch(
0018   "/support-messages/:id/read",
0019   dbReady,
0020   auth,
0021   adminOnly,
0022   validateObjectId("id"),
0023   asyncHandler(markSupportMessageRead)
0024 );
0025 router.delete(
0026   "/support-messages/:id",
0027   dbReady,
0028   auth,
0029   adminOnly,
0030   validateObjectId("id"),
0031   asyncHandler(deleteSupportMessage)
0032 );
0033
0034 module.exports = router;

```

А.19 Авторизация controller бөлігі

// Файл: backend/src/controllers/auth.controller.js

```

0001 const bcrypt = require("bcryptjs");
0002 const jwt = require("jsonwebtoken");
0003 const User = require("../models/User");
0004 const ApiError = require("../utils/apiError");
0005 const { config } = require("../config/env");
0006
0007 function normalizeEmail(email) {
0008   return String(email || "").trim().toLowerCase();

```

```

0009 }
0010
0011 async function register(req, res) {
0012   const name = String(req.body.name || "").trim();
0013   const email = normalizeEmail(req.body.email);
0014   const password = String(req.body.password || "");
0015
0016   if (!name || !email || !password) {
0017     throw new ApiError(400, "Name, email and password are required.");
0018   }
0019
0020   if (password.length < 6) {
0021     throw new ApiError(400, "Password must contain at least 6 characters.");
0022   }
0023
0024   const exists = await User.findOne({ email });
0025   if (exists) {
0026     throw new ApiError(400, "This email is already registered.");
0027   }
0028
0029   const adminsCount = await User.countDocuments({ role: "admin" });
0030   const hash = await bcrypt.hash(password, 10);
0031   const user = await User.create({
0032     name,
0033     email,
0034     password: hash,
0035     role: adminsCount === 0 ? "admin" : "user",
0036   });
0037
0038   res.status(201).json({
0039     success: true,
0040     message: adminsCount === 0 ? "User has been created as administrator." : "User has been created.",
0041     role: user.role,
0042     data: {
0043       role: user.role,
0044     },
0045   });
0046 }
0047
0048 async function login(req, res) {
0049   const email = normalizeEmail(req.body.email);
0050   const password = String(req.body.password || "");
0051
0052   if (!email || !password) {
0053     throw new ApiError(400, "Email and password are required.");
0054   }
0055
0056   const user = await User.findOne({ email });
0057   if (!user) {
0058     throw new ApiError(404, "User was not found.");
0059   }
0060
0061   const ok = await bcrypt.compare(password, user.password);
0062   if (!ok) {
0063     throw new ApiError(401, "Password is incorrect.");
0064   }
0065
0066   const token = jwt.sign(
0067     { id: user._id, role: user.role, email: user.email, name: user.name },
0068     config.jwtSecret,

```

```

0069   { expiresIn: "24h" }
0070 );
0071
0072 const payload = {
0073   token,
0074   role: user.role,
0075   email: user.email,
0076   name: user.name,
0077   phone: user.phone,
0078   organization: user.organization,
0079   city: user.city,
0080 };
0081
0082 res.json({
0083   success: true,
0084   message: "Login successful.",
0085   ...payload,
0086   data: payload,
0087 });
0088 }
0089
0090 module.exports = {
0091   register,
0092   login,
0093 };

```

A.20 Ic-shapa controller bəlihi

```

// Файл: backend/src/controllers/event.controller.js
0001 const Event = require("../models/Event");
0002 const Booking = require("../models/Booking");
0003 const ApiError = require("../utils/apiError");
0004 const { findScheduleConflicts } = require("../utils/availability");
0005
0006 const activeBookingStatuses = ["registered", "new", "review", "pending", "approved"];
0007
0008 function eventPayload(body) {
0009   const payload = {
0010     title: String(body.title || "").trim(),
0011     date: String(body.date || "").trim(),
0012     time: String(body.time || "10:00").trim(),
0013     category: String(body.category || "").trim(),
0014     description: String(body.description || "").trim(),
0015     agenda: String(body.agenda || "").trim(),
0016     organizer: String(body.organizer || "ORDA").trim(),
0017     location: String(body.location || "").trim(),
0018     format: String(body.format || "offline").trim(),
0019     meetingUrl: String(body.meetingUrl || "").trim(),
0020     price: Number(body.price || 0),
0021     currency: String(body.currency || "KZT").trim(),
0022     featured: body.featured === true || body.featured === "true" || body.featured === "on",
0023     tags: String(body.tags || "")
0024       .split(",")
0025       .map((tag) => tag.trim())
0026       .filter(Boolean),
0027     capacity: Number(body.capacity || 80),
0028     status: String(body.status || "published").trim(),
0029   };
0030
0031   if (Object.prototype.hasOwnProperty.call(body, "endDate")) payload.endDate = String(body.endDate ||
    "").trim();

```

```

0032 if (Object.prototype.hasOwnProperty.call(body, "city")) payload.city = String(body.city || ""
).trim();
0033 if (Object.prototype.hasOwnProperty.call(body, "image")) payload.image = String(body.image || ""
).trim();
0034 if (body.translations && typeof body.translations === "object") payload.translations =
body.translations;
0035
0036 return payload;
0037 }
0038
0039 function validateEventPayload(data) {
0040   if (!data.title || !data.date || !data.category || !data.location) {
0041     throw new ApiError(400, "Title, date, category and location are required.");
0042   }
0043
0044   if (!Number.isFinite(data.capacity) || data.capacity < 1) {
0045     throw new ApiError(400, "Capacity must be greater than 0.");
0046   }
0047
0048   if (!Number.isFinite(data.price) || data.price < 0) {
0049     throw new ApiError(400, "Price cannot be negative.");
0050   }
0051 }
0052
0053 async function eventWithBookingCount(event, userEmail) {
0054   const [booked, userBooking] = await Promise.all([
0055     Booking.countDocuments({ eventId: event._id, status: { $in: activeBookingStatuses } }),
0056     userEmail ? Booking.findOne({ eventId: event._id, userEmail, status: { $in: activeBookingStatuses
} }) : null,
0057   ]);
0058
0059   return {
0060     ...event.toObject(),
0061     booked,
0062     seatsLeft: Math.max(Number(event.capacity || 0) - booked, 0),
0063     isBookedByMe: Boolean(userBooking),
0064   };
0065 }
0066
0067 async function getEvents(req, res) {
0068   const query = {};
0069   const search = String(req.query.search || "").trim();
0070
0071   if (req.query.category && req.query.category !== "all") query.category = req.query.category;
0072   if (req.query.status && req.query.status !== "all") query.status = req.query.status;
0073   if (req.query.format && req.query.format !== "all") query.format = req.query.format;
0074   if (req.query.free === "true") query.price = 0;
0075   if (req.query.featured === "true") query.featured = true;
0076
0077   if (search) {
0078     query.$or = [
0079       { title: new RegExp(search, "i") },
0080       { description: new RegExp(search, "i") },
0081       { location: new RegExp(search, "i") },
0082       { city: new RegExp(search, "i") },
0083       { organizer: new RegExp(search, "i") },
0084       { tags: new RegExp(search, "i") },
0085       { "translations.ru.title": new RegExp(search, "i") },
0086       { "translations.ru.description": new RegExp(search, "i") },
0087       { "translations.ru.location": new RegExp(search, "i") },

```

```

0088   { "translations.ru.city": new RegExp(search, "i") },
0089   { "translations.kk.title": new RegExp(search, "i") },
0090   { "translations.kk.description": new RegExp(search, "i") },
0091   { "translations.kk.location": new RegExp(search, "i") },
0092   { "translations.kk.city": new RegExp(search, "i") },
0093   { "translations.en.title": new RegExp(search, "i") },
0094   { "translations.en.description": new RegExp(search, "i") },
0095   { "translations.en.location": new RegExp(search, "i") },
0096   { "translations.en.city": new RegExp(search, "i") },
0097 ];
0098 }
0099
0100 const events = await Event.find(query).sort({ date: 1, time: 1, title: 1 });
0101 const result = await Promise.all(events.map((event) => eventWithBookingCount(event, req.user?.email
0102   )));
0103 res.json(result);
0104 }
0105 async function getEvent(req, res) {
0106   const event = await Event.findById(req.params.id);
0107
0108   if (!event) {
0109     throw new ApiError(404, "Event was not found.");
0110   }
0111
0112   const result = await eventWithBookingCount(event, req.user?.email);
0113   res.json({
0114     success: true,
0115     message: "Event loaded.",
0116     data: result,
0117     ...result,
0118   });
0119 }
0120
0121 async function createEvent(req, res) {
0122   const data = eventPayload(req.body);
0123   validateEventPayload(data);
0124
0125   const conflicts = await findScheduleConflicts({
0126     location: data.location,
0127     date: data.date,
0128     time: data.time,
0129     duration: 2,
0130   });
0131   if (conflicts.length) {
0132     throw new ApiError(400, "This date and time are already occupied.");
0133   }
0134
0135   const event = await Event.create(data);
0136   const result = await eventWithBookingCount(event);
0137   res.status(201).json({ success: true, message: "Event has been created.", event: result, data:
0138     result });
0139 }
0140 async function updateEvent(req, res) {
0141   const data = eventPayload(req.body);
0142   validateEventPayload(data);
0143
0144   const conflicts = await findScheduleConflicts({
0145     location: data.location,

```

```

0146   date: data.date,
0147   time: data.time,
0148   duration: 2,
0149   excludeEventId: req.params.id,
0150 });
0151 if (conflicts.length) {
0152   throw new ApiError(400, "This date and time are already occupied.");
0153 }
0154
0155 const event = await Event.findByIdAndUpdate(req.params.id, data, {
0156   new: true,
0157   runValidators: true,
0158 });
0159
0160 if (!event) {
0161   throw new ApiError(404, "Event was not found.");
0162 }
0163
0164 const result = await eventWithBookingCount(event);
0165 res.json({ success: true, message: "Event has been updated.", event: result, data: result });
0166 }
0167
0168 async function deleteEvent(req, res) {
0169   const event = await Event.findByIdAndDelete(req.params.id);
0170
0171   if (!event) {
0172     throw new ApiError(404, "Event was not found.");
0173   }
0174
0175   await Booking.deleteMany({ eventId: event._id });
0176   res.json({ success: true, message: "Event has been deleted." });
0177 }
0178
0179 module.exports = {
0180   getEvents,
0181   getEvent,
0182   createEvent,
0183   updateEvent,
0184   deleteEvent,
0185   eventWithBookingCount,
0186 };

```

A.21 Зал controller бўлиги

```

// Файл: backend/src/controllers/hall.controller.js
0001 const Hall = require("../models/Hall");
0002 const Event = require("../models/Event");
0003 const Booking = require("../models/Booking");
0004 const ApiError = require("../utils/apiError");
0005
0006 const activeBookingStatuses = ["registered", "new", "review", "pending", "approved"];
0007
0008 function hallPayload(body) {
0009   const equipment = Array.isArray(body.equipment)
0010     ? body.equipment
0011     : String(body.equipment || "").
0012       .split(",")
0013       .map((item) => item.trim())
0014       .filter(Boolean);
0015   const payload = {
0016     name: String(body.name || "").trim(),

```

```

0017   floor: String(body.floor || "").trim(),
0018   location: String(body.location || "").trim(),
0019   capacity: Number(body.capacity || 0),
0020   pricePerHour: Number(body.pricePerHour || 0),
0021   equipment,
0022   description: String(body.description || "").trim(),
0023   status: String(body.status || "available").trim(),
0024 };
0025
0026 if (body.translations && typeof body.translations === "object") {
0027   payload.translations = body.translations;
0028 }
0029
0030 return payload;
0031 }
0032
0033 function assertHallPayload(payload) {
0034   if (!payload.name || !payload.floor || !payload.location) {
0035     throw new ApiError(400, "Hall name, floor and location are required.");
0036   }
0037   if (!Number.isFinite(payload.capacity) || payload.capacity < 1) {
0038     throw new ApiError(400, "Capacity must be positive.");
0039   }
0040   if (!Number.isFinite(payload.pricePerHour) || payload.pricePerHour < 0) {
0041     throw new ApiError(400, "Price must not be negative.");
0042   }
0043   if (!["available", "busy", "maintenance"].includes(payload.status)) {
0044     throw new ApiError(400, "Invalid hall status.");
0045   }
0046 }
0047
0048 async function withActiveRequests(hall) {
0049   const activeRequests = await Booking.countDocuments({
0050     hallId: String(hall._id),
0051     status: { $in: activeBookingStatuses },
0052   });
0053
0054   return {
0055     ...hall.toObject(),
0056     activeRequests,
0057   };
0058 }
0059
0060 async function getHalls(req, res) {
0061   const search = String(req.query.search || "").trim();
0062   const capacity = String(req.query.capacity || "all").trim();
0063   const query = {};
0064
0065   if (search) {
0066     query.$or = [
0067       { name: new RegExp(search, "i") },
0068       { location: new RegExp(search, "i") },
0069       { description: new RegExp(search, "i") },
0070       { "translations.ru.name": new RegExp(search, "i") },
0071       { "translations.ru.location": new RegExp(search, "i") },
0072       { "translations.ru.description": new RegExp(search, "i") },
0073       { "translations.kk.name": new RegExp(search, "i") },
0074       { "translations.kk.location": new RegExp(search, "i") },
0075       { "translations.kk.description": new RegExp(search, "i") },
0076       { "translations.en.name": new RegExp(search, "i") },

```

```

0077     { "translations.en.location": new RegExp(search, "i") },
0078     { "translations.en.description": new RegExp(search, "i") },
0079   ];
0080 }
0081
0082 if (capacity === "small") query.capacity = { $lte: 50 };
0083 if (capacity === "medium") query.capacity = { $gt: 50, $lte: 150 };
0084 if (capacity === "large") query.capacity = { $gt: 150 };
0085
0086 const halls = await Hall.find(query).sort({ createdAt: -1 });
0087 const result = await Promise.all(halls.map(withActiveRequests));
0088 res.json(result);
0089 }
0090
0091 async function createHall(req, res) {
0092   const payload = hallPayload(req.body);
0093   assertHallPayload(payload);
0094
0095   const hall = await Hall.create(payload);
0096   res.status(201).json({ success: true, message: "Hall has been created.", hall, data: hall });
0097 }
0098
0099 async function updateHall(req, res) {
0100   const payload = hallPayload(req.body);
0101   assertHallPayload(payload);
0102
0103   const hall = await Hall.findByIdAndUpdate(req.params.id, payload, { new: true });
0104   if (!hall) {
0105     throw new ApiError(404, "Hall was not found.");
0106   }
0107
0108   res.json({ success: true, message: "Hall has been updated.", hall, data: hall });
0109 }
0110
0111 async function deleteHall(req, res) {
0112   const hall = await Hall.findById(req.params.id);
0113   if (!hall) {
0114     throw new ApiError(404, "Hall was not found.");
0115   }
0116
0117   const hasActiveRequests = await Booking.exists({
0118     hallId: String(hall._id),
0119     status: { $in: activeBookingStatuses },
0120   });
0121   if (hasActiveRequests) {
0122     throw new ApiError(400, "Hall has active requests.");
0123   }
0124
0125   const usedByEvents = await Event.exists({ location: hall.name, status: { $ne: "closed" } });
0126   if (usedByEvents) {
0127     throw new ApiError(400, "Hall is used by active events.");
0128   }
0129
0130   await hall.deleteOne();
0131   res.json({ success: true, message: "Hall has been deleted." });
0132 }
0133
0134 module.exports = {
0135   getHalls,
0136   createHall,

```

```

0137   updateHall,
0138   deleteHall,
0139 };

```

A.22 Брондау controller негізгі бөлігі

```

// Файл: backend/src/controllers/booking.controller.js
0001 const Event = require("../models/Event");
0002 const Hall = require("../models/Hall");
0003 const User = require("../models/User");
0004 const Booking = require("../models/Booking");
0005 const ApiError = require("../utils/apiError");
0006 const {
0007   activeBookingStatuses,
0008   buildAvailability,
0009   endTimeFrom,
0010   findScheduleConflicts,
0011 } = require("../utils/availability");
0012
0013 const inactiveBookingStatuses = ["rejected", "cancelled"];
0014 const customEventAnnouncementPrice = 10000;
0015
0016 function normalizePaymentStatus(value, amount) {
0017   const status = String(value || "").trim();
0018   if (["free", "unpaid", "paid", "refunded"].includes(status)) return status;
0019   return Number(amount || 0) > 0 ? "unpaid" : "free";
0020 }
0021
0022 function checkedEquipment(body) {
0023   if (Array.isArray(body.equipment)) return body.equipment;
0024
0025   return ["projector", "mic", "webcast", "catering"]
0026     .filter((name) => body[`eq_${name}`] === true || body[`eq_${name}`] === "true" ||
0027       body[`eq_${name}`] === "on")
0028     .map((name) => ({ name }));
0029 }
0030
0031 async function userPayload(req) {
0032   const user = await User.findById(req.user.id);
0033   return {
0034     userId: String(user?._id || req.user.id || ""),
0035     userEmail: user?.email || req.user.email,
0036     userName: user?.name || req.user.name || req.user.email,
0037     userPhone: user?.phone || "",
0038     organization: user?.organization || "",
0039   };
0040 }
0041
0042 async function bookEvent(req, res) {
0043   const event = await Event.findById(req.body.eventId);
0044   if (!event) {
0045     throw new ApiError(404, "Event was not found.");
0046   }
0047
0048   if (event.status !== "published") {
0049     throw new ApiError(400, "Registration is closed for this event.");
0050   }
0051
0052   const exists = await Booking.findOne({ userEmail: req.user.email, eventId: event._id, status: {
     $in: activeBookingStatuses } });

```

```

0053
0054 if (exists) {
0055   throw new ApiError(400, "You are already registered for this event.");
0056 }
0057
0058 const booked = await Booking.countDocuments({ eventId: event._id, status: { $in:
activeBookingStatuses } });
0059 if (booked >= event.capacity) {
0060   throw new ApiError(400, "There are no free seats for this event.");
0061 }
0062
0063 const price = Number(event.price || 0);
0064 const paymentStatus = normalizePaymentStatus(req.body.paymentStatus, price);
0065 const requester = await userPayload(req);
0066 const booking = await Booking.create({
0067   ...requester,
0068   userName: String(req.body.userName || req.body.contactName || requester.userName || "").trim(),
0069   userEmail: String(req.body.userEmail || req.body.email || requester.userEmail || "").trim(),
0070   userPhone: String(req.body.userPhone || req.body.phone || requester.userPhone || "").trim(),
0071   type: "event",
0072   eventId: event._id,
0073   eventTitle: event.title,
0074   date: event.date,
0075   time: event.time,
0076   price,
0077   amount: price,
0078   paymentStatus,
0079   paymentMethod: price > 0 ? String(req.body.paymentMethod || "card").trim() : "free",
0080   paidAt: paymentStatus === "paid" ? new Date() : null,
0081   status: "new",
0082 });
0083
0084 res.json({ success: true, message: "Participant registration has been completed.", booking, data:
booking });
0085 }
0086
0087 async function getMyBookings(req, res) {
0088   const bookings = await Booking.find({
0089     userEmail: req.user.email,
0090     $or: [{ type: "event" }, { type: { $exists: false } }],
0091   }).sort({ createdAt: -1 });
0092   res.json(bookings);
0093 }
0094
0095 async function cancelMyBooking(req, res) {
0096   const booking = await Booking.findOneAndUpdate(
0097     { _id: req.params.id, userEmail: req.user.email, $or: [{ type: "event" }, { type: { $exists:
false } } ] },
0098     { status: "cancelled" },
0099     { new: true }
0100   );
0101
0102   if (!booking) {
0103     throw new ApiError(404, "Booking was not found.");
0104   }
0105
0106   res.json({ success: true, message: "Booking has been cancelled." });
0107 }
0108
0109 async function updateMyBookingPayment(req, res) {

```

```

0110 const booking = await Booking.findOne({
0111   _id: req.params.id,
0112   userEmail: req.user.email,
0113   $or: [{ type: "event" }, { type: { $exists: false } }],
0114 });
0115
0116 if (!booking) {
0117   throw new ApiError(404, "Booking was not found.");
0118 }
0119
0120 if (inactiveBookingStatuses.includes(booking.status)) {
0121   throw new ApiError(400, "Request status cannot be updated.");
0122 }
0123
0124 booking.paymentStatus = "paid";
0125 booking.paymentMethod = String(req.body.paymentMethod || booking.paymentMethod || "card").trim();
0126 booking.paidAt = new Date();
0127 await booking.save();
0128
0129 res.json({ success: true, message: "Payment has been completed.", booking, data: booking });
0130 }
0131
0132 async function createHallBooking(req, res) {
0133   const type = String(req.body.type || "hall").trim() === "custom-event" ? "custom-event" : "hall";
0134   const date = String(req.body.date || "").trim();
0135   const time = String(req.body.time || req.body.startTime || "").trim();
0136   const duration = Number(req.body.duration || 0);
0137   const attendees = Number(req.body.attendees || 0);
0138   const purpose = String(req.body.purpose || req.body.description || "").trim();
0139   const hallId = String(req.body.hallId || "").trim();
0140   const ownLocation = String(req.body.location || "").trim();
0141   const ownCity = String(req.body.city || "").trim();
0142
0143   if (!date || !time || !duration || !attendees || !purpose) {
0144     throw new ApiError(400, "Hall request fields are required.");
0145   }
0146   if (type === "hall" && !hallId) {
0147     throw new ApiError(400, "Hall request fields are required.");
0148   }
0149   if (type === "custom-event" && (!req.body.eventTitle || !ownLocation || !ownCity)) {
0150     throw new ApiError(400, "Event title, location and city are required.");
0151   }
0152
0153   const hall = type === "hall" ? await Hall.findById(hallId) : null;
0154   if (type === "hall" && !hall) {
0155     throw new ApiError(404, "Hall was not found.");
0156   }
0157   if (hall && attendees > hall.capacity) {
0158     throw new ApiError(400, "Guest count exceeds hall capacity.");
0159   }
0160   if (type === "hall") {
0161     const exists = await Booking.findOne({
0162       userEmail: req.user.email,
0163       type: "hall",
0164       hallId: String(hall._id),
0165       status: { $in: activeBookingStatuses },
0166     });
0167     if (exists) {
0168       throw new ApiError(400, "You have already booked this conference hall.");
0169     }

```

```

0170 }
0171
0172 const conflictVenue = type === "hall" ? hall?.name : ownLocation;
0173 const conflicts = await findScheduleConflicts({
0174   hallId: type === "hall" ? hallId : "",
0175   hallName: type === "hall" ? hall?.name : "",
0176   location: conflictVenue,
0177   date,
0178   time,
0179   duration,
0180 });
0181 if (conflicts.length) {
0182   throw new ApiError(400, "This date and time are already occupied.");
0183 }
0184
0185 const hallName = String(req.body.hallName || hall?.name || "").trim();
0186 const pricePerHour = Number(req.body.pricePerHour || hall?.pricePerHour || 0);
0187 const amount = Number(req.body.amount || (type === "custom-event" ? customEventAnnouncementPrice
:
    pricePerHour * duration) || 0);
0188 const paymentStatus = normalizePaymentStatus(req.body.paymentStatus, amount);
0189 const booking = await Booking.create({
0190   ...(await userPayload(req)),
0191   type,
0192   status: "new",
0193   eventTitle: String(req.body.eventTitle || req.body.title || purpose || hallName || "Hall request"
).trim(),
0194   category: String(req.body.category || "").trim(),
0195   format: String(req.body.format || "").trim(),
0196   description: String(req.body.description || "").trim(),
0197   image: String(req.body.image || "").trim(),
0198   location: type === "custom-event" ? ownLocation : hallName,
0199   city: ownCity,
0200   hallId,
0201   hallName,
0202   date,
0203   time,
0204   startTime: String(req.body.startTime || time).trim(),
0205   endTime: String(req.body.endTime || endTimeFrom(time, duration)).trim(),
0206   duration,
0207   attendees,
0208   purpose,
0209   adminNotes: String(req.body.adminNotes || req.body.notes || "").trim(),
0210   services: Array.isArray(req.body.services) ? req.body.services : [],
0211   equipment: checkedEquipment(req.body),
0212   ticketPrice: Number(req.body.ticketPrice || 0),
0213   pricePerHour,
0214   rentAmount: Number(req.body.rentAmount || pricePerHour * duration || 0),
0215   amount,
0216   paymentStatus,
0217   paymentMethod: amount > 0 ? String(req.body.paymentMethod || "card").trim() : "free",
0218   paidAt: paymentStatus === "paid" ? new Date() : null,
0219 });
0220
0221 res.status(201).json({ success: true, message: "Hall request has been created.", booking, data:
    booking });
0222 }
0223
0224 async function getAvailability(req, res) {
0225   const availability = await buildAvailability({

```

```

0226   hallId: req.query.hallId,
0227   date: req.query.date,
0228   });
0229   res.json(availability);
0230 }
0231
0232 async function getMyNotifications(req, res) {
0233   const bookings = await Booking.find({
0234     userEmail: req.user.email,
0235     status: { $in: ["approved", "rejected", "cancelled"] },
0236     type: { $in: ["hall", "custom-event"] },
0237   }).sort({ updatedAt: -1 });
0238
0239   res.json(
0240     bookings.map((booking) => ({
0241       _id: `booking-${booking._id}`,
0242       type: booking.status === "approved" ? "success" : "warn",
0243       messageKey:
0244         booking.status === "approved"
0245           ? "msgRequestApproved"
0246           : booking.status === "rejected"
0247             ? "msgRequestRejected"
0248             : "msgRequestCancelledByAdmin",
0249       reason: booking.adminReason || "",
0250       requestId: String(booking._id),
0251       requestType: booking.type,
0252       read: Boolean(booking.clientNotificationReadAt),
0253       createdAt: booking.updatedAt || booking.createdAt,
0254     })))
0255   );
0256 }
0257
0258 async function markMyNotificationRead(req, res) {
0259   const id = String(req.params.id || "").replace(/^booking-/ , "");
0260   const booking = await Booking.findOneAndUpdate(
0261     { _id: id, userEmail: req.user.email },
0262     { clientNotificationReadAt: new Date() },
0263     { new: true }
0264   );
0265   if (!booking) throw new ApiError(404, "Notification was not found.");
0266   res.json({ success: true, message: "Notification has been marked as read." });
0267 }
0268
0269 async function getMyHallBookings(req, res) {
0270   const bookings = await Booking.find({
0271     userEmail: req.user.email,
0272     type: { $in: ["hall", "custom-event"] },
0273   }).sort({ createdAt: -1 });
0274   res.json(bookings);
0275 }
0276
0277 async function updateMyHallBookingPayment(req, res) {
0278   const booking = await Booking.findOne({
0279     _id: req.params.id,
0280     userEmail: req.user.email,
0281     type: { $in: ["hall", "custom-event"] },
0282   });
0283
0284   if (!booking) {
0285     throw new ApiError(404, "Booking was not found.");

```

```

0286 }
0287
0288 if (inactiveBookingStatuses.includes(booking.status)) {
0289   throw new ApiError(400, "Request status cannot be updated.");
0290 }
0291
0292 booking.paymentStatus = "paid";
0293 booking.paymentMethod = String(req.body.paymentMethod || booking.paymentMethod || "card").trim();
0294 booking.paidAt = new Date();
0295 await booking.save();
0296
0297 res.json({ success: true, message: "Payment has been completed.", booking, data: booking });
0298 }
0299
0300 async function cancelMyHallBooking(req, res) {
0301   const booking = await Booking.findOneAndUpdate(
0302     { _id: req.params.id, userEmail: req.user.email, type: { $in: ["hall", "custom-event"] } },
0303     { status: "cancelled" },
0304     { new: true }
0305   );
0306
0307   if (!booking) {
0308     throw new ApiError(404, "Booking was not found.");
0309   }
0310
0311   res.json({ success: true, message: "Booking has been cancelled." });
0312 }
0313
0314 module.exports = {
0315   bookEvent,
0316   getMyBookings,
0317   updateMyBookingPayment,
0318   cancelMyBooking,
0319   createHallBooking,
0320   getMyHallBookings,
0321   updateMyHallBookingPayment,
0322   cancelMyHallBooking,
0323   getMyNotifications,
0324   markMyNotificationRead,
0325   getAvailability,
0326 };

```

A.23 Әкімші controller негізгі бөлігі

```

// Файл: backend/src/controllers/admin.controller.js
0001 const User = require("../models/User");
0002 const Event = require("../models/Event");
0003 const Booking = require("../models/Booking");
0004 const ApiError = require("../utils/apiError");
0005 const demoEvents = require("../utils/seedData");
0006 const {
0007   activeBookingStatuses,
0008   confirmedBookingStatuses,
0009   endTimeFrom,
0010   findScheduleConflicts,
0011 } = require("../utils/availability");
0012
0013 const adminBookingStatuses = ["new", "review", "pending", "approved", "rejected", "cancelled",
  "registered"];
0014
0015 async function getBookings(req, res) {

```

```

0016 const bookings = await Booking.find().sort({ createdAt: -1 }).lean();
0017 res.json(bookings.map(normalizeBookingForAdmin));
0018 }
0019
0020 async function getUsers(req, res) {
0021   const users = await User.find().select("-password").sort({ createdAt: -1 });
0022   res.json(users);
0023 }
0024
0025 async function getStats(req, res) {
0026   const [events, bookings, users, publishedEvents, checkedIn, paidTickets, approvedHallBookings] =
    await Promise.all([
0027     Event.countDocuments(),
0028     Booking.countDocuments({ status: { $in: activeBookingStatuses } }),
0029     User.countDocuments(),
0030     Event.countDocuments({ status: "published" }),
0031     Booking.countDocuments({ status: { $in: activeBookingStatuses }, checkedIn: true }),
0032     Booking.countDocuments({ type: "event", paymentStatus: "paid" }),
0033     Booking.countDocuments({ type: { $in: ["hall", "custom-event"] }, status: "approved" }),
0034   ]);
0035
0036   const stats = { events, bookings, users, publishedEvents, checkedIn, paidTickets,
    approvedHallBookings };
0037   res.json({ success: true, message: "Stats loaded.", ...stats, data: stats });
0038 }
0039
0040 async function checkInBooking(req, res) {
0041   const existing = await Booking.findById(req.params.id);
0042
0043   if (!existing) {
0044     throw new ApiError(404, "Booking was not found.");
0045   }
0046
0047   if (!["approved", "registered"].includes(existing.status)) {
0048     throw new ApiError(400, "Check-in is available only for approved requests.");
0049   }
0050
0051   const booking = await Booking.findByIdAndUpdate(
0052     req.params.id,
0053     { checkedIn: true, checkedInAt: new Date() },
0054     { new: true }
0055   );
0056
0057   if (!booking) {
0058     throw new ApiError(404, "Booking was not found.");
0059   }
0060
0061   res.json({ success: true, message: "Participant has been checked in.", booking, data: booking });
0062 }
0063
0064 function bookingStatusPayload(body, fallbackStatus) {
0065   const status = String(body.status || fallbackStatus || "").trim();
0066
0067   if (!adminBookingStatuses.includes(status)) {
0068     throw new ApiError(400, "Invalid request status.");
0069   }
0070
0071   return {
0072     status,
0073     adminReason: String(body.reason || body.adminReason || "").trim(),

```

```

0074 };
0075 }
0076
0077 function normalizeBookingForAdmin(booking = {}) {
0078   const type = ["event", "hall", "custom-event"].includes(booking.type)
0079     ? booking.type
0080     : booking.hallId || booking.hallName
0081       ? "hall"
0082       : "event";
0083   const status = booking.status === "registered" ? "approved" : booking.status || "new";
0084   const amount = Number(booking.amount ?? booking.price ?? booking.rentAmount ?? 0);
0085   const paymentStatus = booking.paymentStatus || (amount > 0 ? "unpaid" : "free");
0086
0087   return {
0088     ...booking,
0089     type,
0090     status,
0091     paymentStatus,
0092   };
0093 }
0094
0095 async function assertRequestSlotAvailable(booking, statuses = confirmedBookingStatuses) {
0096   if (!["hall", "custom-event"].includes(booking.type)) return;
0097
0098   const conflicts = await findScheduleConflicts({
0099     hallId: booking.type === "hall" ? booking.hallId : "",
0100     hallName: booking.type === "hall" ? booking.hallName : "",
0101     location: booking.type === "custom-event" ? booking.location || booking.hallName :
0102       booking.hallName || booking.location,
0103     date: booking.date,
0104     time: booking.time || booking.startTime,
0105     duration: Number(booking.duration || 2),
0106     endTime: booking.endTime,
0107     excludeBookingId: String(booking._id),
0108     statuses,
0109   });
0110   if (conflicts.length) {
0111     throw new ApiError(400, "This date and time are already occupied.");
0112   }
0113 }
0114
0115 async function publishCustomEvent(booking) {
0116   if (booking.type !== "custom-event" || booking.eventId) return;
0117
0118   await assertRequestSlotAvailable(booking);
0119
0120   const event = await Event.create({
0121     title: booking.eventTitle || "Client event",
0122     date: booking.date,
0123     time: booking.time || booking.startTime || "10:00",
0124     category: booking.category || "conference",
0125     description: booking.description || booking.purpose || "",
0126     image: booking.image || "",
0127     agenda: `Длительность: ${booking.duration || 1} ч. ${booking.adminNotes || ""}`.trim(),
0128     organizer: booking.organization || booking.userName || "Client",
0129     location: booking.location || booking.hallName || "Client venue",
0130     city: booking.city || "",
0131     format: ["offline", "online", "hybrid"].includes(booking.format) ? booking.format : "offline",
0132     price: Number(booking.ticketPrice || 0),

```

```

0133   currency: "KZT",
0134   capacity: Number(booking.attendees || 80),
0135   status: "published",
0136   tags: [booking.category, "client-event"].filter(Boolean),
0137 });
0138
0139 booking.eventId = event._id;
0140 }
0141
0142 async function updateBooking(req, res) {
0143   const booking = await Booking.findById(req.params.id);
0144
0145   if (!booking) {
0146     throw new ApiError(404, "Booking was not found.");
0147   }
0148
0149   const previousStatus = booking.status;
0150   const payload = bookingStatusPayload(req.body, booking.status);
0151   booking.status = payload.status;
0152   booking.adminReason = payload.adminReason;
0153   if (payload.status !== previousStatus && ["approved", "rejected",
    "cancelled"].includes(payload.status)) {
0154     booking.clientNotificationReadAt = null;
0155   }
0156   if (booking.type === "hall" || booking.type === "custom-event") {
0157     if (req.body.date) booking.date = String(req.body.date).trim();
0158     if (req.body.time) booking.time = String(req.body.time).trim();
0159     if (Number(req.body.duration) > 0) booking.duration = Number(req.body.duration);
0160     if (Number(req.body.attendees) > 0) booking.attendees = Number(req.body.attendees);
0161     if (typeof req.body.purpose === "string") booking.purpose = String(req.body.purpose).trim();
0162     if (booking.time && Number(booking.duration || 0) > 0) {
0163       booking.startTime = booking.startTime || booking.time;
0164       booking.endTime = booking.endTime || endTimeFrom(booking.time || booking.startTime,
        booking.duration);
0165     }
0166     booking.amount = Number(booking.pricePerHour || 0) * Number(booking.duration || 0) ||
      Number(booking.amount || 0);
0167   }
0168
0169   if (payload.status !== "approved" && payload.status !== "registered") {
0170     booking.checkedIn = false;
0171     booking.checkedInAt = null;
0172   }
0173
0174   if (payload.status === "approved") {
0175     await assertRequestSlotAvailable(booking);
0176     await publishCustomEvent(booking);
0177   }
0178
0179   await booking.save();
0180
0181   res.json({ success: true, message: "Request status updated.", booking, data: booking });
0182 }
0183
0184 async function updateBookingStatus(req, res) {
0185   return updateBooking(req, res);
0186 }
0187
0188 async function deleteBooking(req, res) {
0189   const booking = await Booking.findByIdAndDelete(req.params.id);

```

```

0190
0191 if (!booking) {
0192   throw new ApiError(404, "Booking was not found.");
0193 }
0194
0195 res.json({ success: true, message: "Request has been deleted." });
0196 }
0197
0198 async function seed(req, res) {
0199   await Event.deleteMany();
0200   await Booking.deleteMany();
0201   await Event.insertMany(demoEvents);
0202
0203   res.json({ success: true, message: "Demo data has been added." });
0204 }
0205
0206 module.exports = {
0207   getBookings,
0208   getUsers,
0209   getStats,
0210   checkInBooking,
0211   updateBooking,
0212   updateBookingStatus,
0213   deleteBooking,
0214   seed,
0215 };

```

A.24 Support controller бөлүгү

```

// Файл: backend/src/controllers/support.controller.js
0001 const User = require("../models/User");
0002 const SupportMessage = require("../models/SupportMessage");
0003 const ApiError = require("../utils/apiError");
0004
0005 async function createSupportMessage(req, res) {
0006   const text = String(req.body.message || req.body.text || "").trim();
0007
0008   if (!text) {
0009     throw new ApiError(400, "Support message text is required.");
0010   }
0011
0012   const user = await User.findById(req.user.id);
0013   const message = await SupportMessage.create({
0014     userId: String(user?.id || req.user.id || ""),
0015     userName: user?.name || req.user.name || req.user.email,
0016     userEmail: user?.email || req.user.email,
0017     text,
0018   });
0019
0020   res.status(201).json({ success: true, message: "Support message has been sent.", data: message,
    supportMessage: message });
0021 }
0022
0023 async function getSupportMessages(req, res) {
0024   const messages = await SupportMessage.find().sort({ createdAt: -1 });
0025   res.json(messages);
0026 }
0027
0028 async function markSupportMessageRead(req, res) {
0029   const message = await SupportMessage.findByIdAndUpdate(
0030     req.params.id,

```

```

0031   { status: "read", readAt: new Date() },
0032   { new: true }
0033 );
0034
0035 if (!message) {
0036   throw new ApiError(404, "Support message was not found.");
0037 }
0038
0039 res.json({ success: true, message: "Support message has been marked as read.", data: message,
    supportMessage: message });
0040 }
0041
0042 async function deleteSupportMessage(req, res) {
0043   const message = await SupportMessage.findByIdAndDelete(req.params.id);
0044
0045   if (!message) {
0046     throw new ApiError(404, "Support message was not found.");
0047   }
0048
0049   res.json({ success: true, message: "Support message has been deleted.", data: message,
    supportMessage: message });
0050 }
0051
0052 module.exports = {
0053   createSupportMessage,
0054   getSupportMessages,
0055   markSupportMessageRead,
0056   deleteSupportMessage,
0057 };

```

A.25 Уақыт қактығысын тексеру утилитасы

```

// Файл: backend/src/utills/availability.js
0001 const Booking = require("../models/Booking");
0002 const Event = require("../models/Event");
0003 const Hall = require("../models/Hall");
0004
0005 const activeBookingStatuses = ["registered", "new", "review", "pending", "approved"];
0006 const confirmedBookingStatuses = ["registered", "approved"];
0007 const activeEventStatuses = ["published", "draft"];
0008
0009 function cleanText(value) {
0010   const text = String(value ?? "").trim();
0011   if (!text || ["undefined", "null", "nan"].includes(text.toLowerCase())) return "";
0012   return text;
0013 }
0014
0015 function minutesFromTime(value) {
0016   const [hours, minutes] = cleanText(value || "00:00")
0017     .split(":")
0018     .map((part) => Number(part || 0));
0019   return Number(hours || 0) * 60 + Number(minutes || 0);
0020 }
0021
0022 function timeFromMinutes(value) {
0023   const safe = Math.max(0, Number(value || 0));
0024   const hours = Math.floor(safe / 60) % 24;
0025   const minutes = safe % 60;
0026   return `${String(hours).padStart(2, "0")}:${String(minutes).padStart(2, "0")}`;
0027 }
0028

```

```

0029 function endTimeFrom(startTime, duration = 2, explicitEnd = "") {
0030   if (cleanText(explicitEnd)) return cleanText(explicitEnd);
0031   return timeFromMinutes(minutesFromTime(startTime) + Number(duration || 2) * 60);
0032 }
0033
0034 function overlaps(startA, endA, startB, endB) {
0035   return startA < endB && endA > startB;
0036 }
0037
0038 function sameText(left, right) {
0039   return cleanText(left).toLowerCase() === cleanText(right).toLowerCase();
0040 }
0041
0042 function bookingStart(booking) {
0043   return cleanText(booking.time || booking.startTime || "00:00");
0044 }
0045
0046 function bookingEnd(booking) {
0047   return endTimeFrom(bookingStart(booking), Number(booking.duration || 2), booking.endTime);
0048 }
0049
0050 function eventEnd(event) {
0051   return endTimeFrom(event.time || "10:00", 2, event.endTime);
0052 }
0053
0054 function hallNames(hall) {
0055   return [hall?.name, hall?.translations?.ru?.name, hall?.translations?.en?.name,
    hall?.translations?.kk?.name]
0056     .map(cleanText)
0057     .filter(Boolean);
0058 }
0059
0060 function findHallForVenue(halls, { hallId = "", hallName = "", location = "" } = {}) {
0061   const id = cleanText(hallId);
0062   if (id) {
0063     const byId = halls.find((hall) => String(hall._id) === id);
0064     if (byId) return byId;
0065   }
0066
0067   const candidates = [hallName, location].map(cleanText).filter(Boolean);
0068   if (!candidates.length) return null;
0069
0070   return halls.find((hall) => hallNames(hall).some((name) => candidates.some((candidate) =>
    sameText(name, candidate)))) || null;
0071 }
0072
0073 function entryStatus(status) {
0074   return confirmedBookingStatuses.includes(status) || status === "published" ? "busy" : "planned";
0075 }
0076
0077 function bookingMatchesVenue(booking, hall) {
0078   if (!booking || !hall) return false;
0079   if (cleanText(booking.hallId) && cleanText(booking.hallId) === String(hall._id)) return true;
0080   return hallNames(hall).some((name) => sameText(name, booking.hallName) || sameText(name,
    booking.location));
0081 }
0082
0083 function eventMatchesVenue(event, hall) {
0084   if (!event || !hall) return false;
0085   return hallNames(hall).some((name) => sameText(name, event.location) || sameText(name,

```

```

        event.hallName));
0086 }
0087
0088 async function buildAvailability(filters = {}) {
0089   const date = cleanText(filters.date);
0090   const hallId = cleanText(filters.hallId);
0091   const [halls, bookings, events] = await Promise.all([
0092     Hall.find().lean(),
0093     Booking.find({
0094       status: { $in: activeBookingStatuses },
0095       ...(date ? { date } : {}),
0096       type: { $in: ["hall", "custom-event"] },
0097     }).lean(),
0098     Event.find({
0099       status: { $in: activeEventStatuses },
0100       ...(date ? { date } : {}),
0101     }).lean(),
0102   ]);
0103
0104   const entries = [];
0105
0106   bookings.forEach((booking) => {
0107     if (booking.type === "custom-event" && booking.eventId) return;
0108     const hall = findHallForVenue(halls, booking);
0109     if (!hall) return;
0110     if (hallId && String(hall._id) !== hallId) return;
0111
0112     const startTime = bookingStart(booking);
0113     entries.push({
0114       source: "booking",
0115       sourceId: String(booking._id),
0116       requestType: booking.type,
0117       hallId: String(hall._id),
0118       hallName: hall.name,
0119       date: cleanText(booking.date),
0120       startTime,
0121       endTime: bookingEnd(booking),
0122       status: entryStatus(booking.status),
0123       requestStatus: booking.status,
0124       title: booking.type === "custom-event" ? cleanText(booking.eventTitle) || "Event announcement"
: cleanText(booking.hallName) || hall.name,
0125     });
0126   });
0127
0128   events.forEach((event) => {
0129     const hall = findHallForVenue(halls, { hallName: event.hallName, location: event.location });
0130     if (!hall) return;
0131     if (hallId && String(hall._id) !== hallId) return;
0132
0133     const startTime = cleanText(event.time || "10:00");
0134     entries.push({
0135       source: "event",
0136       sourceId: String(event._id),
0137       requestType: "event",
0138       hallId: String(hall._id),
0139       hallName: hall.name,
0140       date: cleanText(event.date),
0141       startTime,
0142       endTime: eventEnd(event),
0143       status: entryStatus(event.status),

```

```

0144     eventStatus: event.status,
0145     title: cleanText(event.title) || "Event",
0146   });
0147 });
0148
0149 return entries
0150   .filter((entry) => entry.date && entry.startTime)
0151   .sort((left, right) => `${left.date} ${left.startTime}`.localeCompare(`${right.date}
    ${right.startTime}`));
0152 }
0153
0154 async function findScheduleConflicts({
0155   hallId = "",
0156   hallName = "",
0157   location = "",
0158   date = "",
0159   time = "",
0160   duration = 2,
0161   endTime = "",
0162   excludeBookingId = "",
0163   excludeEventId = "",
0164   statuses = activeBookingStatuses,
0165 } = {}) {
0166   const cleanDate = cleanText(date);
0167   const startTime = cleanText(time || "00:00");
0168   const windowStart = minutesFromTime(startTime);
0169   const windowEnd = minutesFromTime(endTimeFrom(startTime, duration, endTime));
0170   if (!cleanDate || !startTime) return [];
0171
0172   const halls = await Hall.find().lean();
0173   const targetHall = findHallForVenue(halls, { hallId, hallName, location });
0174   const targetNames = targetHall ? hallNames(targetHall) : [hallName, location].map(cleanText)
    ).filter(Boolean);
0175   const targetHallId = targetHall ? String(targetHall._id) : cleanText(hallId);
0176
0177   if (!targetHallId && !targetNames.length) return [];
0178
0179   const [bookings, events] = await Promise.all([
0180     Booking.find({
0181       date: cleanDate,
0182       status: { $in: statuses },
0183       type: { $in: ["hall", "custom-event"] },
0184       ...(excludeBookingId ? { _id: { $ne: excludeBookingId } } : {}),
0185     }).lean(),
0186     Event.find({
0187       date: cleanDate,
0188       status: { $in: activeEventStatuses },
0189       ...(excludeEventId ? { _id: { $ne: excludeEventId } } : {}),
0190     }).lean(),
0191   ]);
0192
0193   const conflicts = [];
0194
0195   bookings.forEach((booking) => {
0196     if (booking.type === "custom-event" && booking.eventId) return;
0197     const sameVenue = targetHall
0198       ? bookingMatchesVenue(booking, targetHall)
0199       : targetNames.some((name) => sameText(name, booking.hallName) || sameText(name,
    booking.location));
0200     if (!sameVenue) return;

```

```

0201
0202   const existingStart = minutesFromTime(bookingStart(booking));
0203   const existingEnd = minutesFromTime(bookingEnd(booking));
0204   if (overlaps(windowStart, windowEnd, existingStart, existingEnd)) {
0205     conflicts.push({ source: "booking", id: String(booking._id), title: booking.eventTitle ||
      booking.hallName || booking.purpose || "Booking" });
0206   }
0207   });
0208
0209   events.forEach((event) => {
0210     const sameVenue = targetHall
0211       ? eventMatchesVenue(event, targetHall)
0212       : targetNames.some((name) => sameText(name, event.location) || sameText(name, event.hallName));
0213     if (!sameVenue) return;
0214
0215     const existingStart = minutesFromTime(event.time || "10:00");
0216     const existingEnd = minutesFromTime(eventEnd(event));
0217     if (overlaps(windowStart, windowEnd, existingStart, existingEnd)) {
0218       conflicts.push({ source: "event", id: String(event._id), title: event.title || "Event" });
0219     }
0220   });
0221
0222   return conflicts;
0223 }
0224
0225 module.exports = {
0226   activeBookingStatuses,
0227   confirmedBookingStatuses,
0228   activeEventStatuses,
0229   buildAvailability,
0230   cleanText,
0231   endTimeFrom,
0232   findScheduleConflicts,
0233   minutesFromTime,
0234   overlaps,
0235 };

```

A.26 Frontend HTTP helper

```

// Файл: docs/js/core/http.js
0001 (function initHttp(window) {
0002   const ORDA = window.ORDA || {};
0003
0004   async function request(url, options = {}) {
0005     const getToken = ORDA.getToken || (() => null);
0006     const getErrorMessage = ORDA.getErrorMessage || ((response, data) => data.message || "Request
      failed");
0007     const headers = {
0008       "Content-Type": "application/json",
0009       ...(options.headers || {}),
0010     };
0011     const token = getToken();
0012
0013     if (token) {
0014       headers.Authorization = `Bearer ${token}`;
0015     }
0016
0017     const response = await fetch(url, { ...options, headers });
0018     const raw = await response.text();
0019     let data = {};
0020

```

```

0021   if (raw) {
0022     try {
0023       data = JSON.parse(raw);
0024     } catch (parseError) {
0025       const error = new Error("Unexpected response from server.");
0026       if (!response.ok) error.status = response.status;
0027       error.responseStatus = response.status;
0028       error.data = { preview: raw.slice(0, 200) };
0029       throw error;
0030     }
0031   }
0032
0033   if (!response.ok) {
0034     const error = new Error(getErrorMessage(response, data));
0035     error.status = response.status;
0036     error.data = data;
0037     throw error;
0038   }
0039
0040   return data;
0041 }
0042
0043 ORDA.http = {
0044   request,
0045 };
0046
0047 window.ORDA = ORDA;
0048 })(window);

```

A.27 Frontend session helper

```

// Файл: docs/js/core/session.js
0001 (function initSession(window) {
0002   const ORDA = window.ORDA || {};
0003
0004   function storedValue(key) {
0005     const value = localStorage.getItem(key);
0006     if (!value || value === "undefined" || value === "null") return null;
0007     return value;
0008   }
0009
0010   function read() {
0011     return {
0012       token: storedValue("token"),
0013       email: storedValue("email"),
0014       name: storedValue("name"),
0015       role: storedValue("role"),
0016     };
0017   }
0018
0019   function save(data, state) {
0020     state.token = data.token;
0021     state.email = data.email;
0022     state.name = data.name;
0023     state.role = data.role;
0024
0025     if (data.token) localStorage.setItem("token", data.token);
0026     if (data.email) localStorage.setItem("email", data.email);
0027     if (data.name) localStorage.setItem("name", data.name);
0028     if (data.role) localStorage.setItem("role", data.role);
0029   }

```

```

0030
0031 function clear(state) {
0032     state.token = null;
0033     state.email = null;
0034     state.name = null;
0035     state.role = null;
0036     state.profile = null;
0037     state.bookings = [];
0038
0039     localStorage.removeItem("token");
0040     localStorage.removeItem("email");
0041     localStorage.removeItem("name");
0042     localStorage.removeItem("role");
0043 }
0044
0045 ORDA.session = {
0046     read,
0047     save,
0048     clear,
0049 };
0050
0051 window.ORDA = ORDA;
0052 })(window);

```

А.28 Көптілді интерфейс сөздігінің басы

```

// Файл: docs/js/core/i18n.js
0001 (function initI18n(window) {
0002     const ORDA = window.ORDA || {};
0003
0004     const ru = {
0005         guest: "Гость",
0006         roleAdmin: "Администратор",
0007         roleUser: "Клиент",
0008         logout: "Выйти",
0009         demoModeBadge: "Демо режим: localStorage",
0010
0011         projectLabel: " ",
0012         projectMeaning: "Веб-система для регистрации на мероприятия и управления конференц-залами.",
0013         platformLabel: "Платформа управления событиями",
0014         heroTitle: "ORDA Smart Event System",
0015         heroText: "Профессиональная система для регистрации участников, управления мероприятиями и заявками на конференц-залы.",
0016         browseEvents: "Смотреть мероприятия",
0017         viewHalls: "Смотреть залы",
0018         heroMetricSupport: "поддержка",
0019         heroMetricEvents: "Мероприятий",
0020         heroMetricHalls: "Конференц-залов",
0021         heroMetricRequests: "Заявок",
0022         brandPromise: "Единая платформа",
0023         brandPromiseText: "Каталог событий, бронирование залов, личный кабинет и административная аналитика в одном интерфейсе.",
0024
0025         accessLabel: "Доступ",
0026         authTitle: "Вход и регистрация",
0027         authText: "Создайте аккаунт, чтобы бронировать конференц-залы, организовывать мероприятия и участвовать в бизнес-форумах и семинарах.",
0028         login: "Войти",
0029         register: "Регистрация",
0030         createAccount: "Создать аккаунт",
0031

```

0032 navDashboard: "Главная",
 0033 navEvents: "Мероприятия",
 0034 navHalls: "Конференц-залы",
 0035 navCalendar: "Календарь",
 0036 navFloorPlan: "Карта",
 0037 navAbout: "О платформе",
 0038 navProfile: "Личный кабинет",
 0039 navAdmin: "Администрирование",
 0040 navAdminSection: "Администрирование",
 0041
 0042 dashboardLabel: "Обзор",
 0043 dashboardTitle: "Рабочее пространство",
 0044 discoverLabel: "Поиск мероприятий",
 0045 discoverTitle: "Подберите подходящее мероприятие или площадку за несколько кликов",
 0046 searchButton: "Искать",
 0047 demoData: "Обновить демо-данные",
 0048
 0049 clientWorkspaceLabel: "Режим клиента",
 0050 clientWorkspaceTitle: "Записи и бронирования",
 0051 clientWorkspaceText: "Регистрируйтесь на мероприятия, подавайте заявки на залы и отслеживайте статусы в личном кабинете.",
 0052 adminWorkspaceLabel: "Режим администратора",
 0053 adminWorkspaceTitle: "Управление площадкой",
 0054 adminWorkspaceText: "Публикуйте события, обрабатывайте заявки и контролируйте загрузку конференц-залов.",
 0055
 0056 metricEvents: "мероприятий",
 0057 metricSeats: "свободных мест",
 0058 metricBookings: "моих заявок",
 0059 metricBookingsTotal: "всего заявок",
 0060 metricUsers: "пользователей",
 0061 metricCheckedIn: "check-in",
 0062 metricNext: "ближайшая дата",
 0063 upcomingEvents: "Ближайшие мероприятия",
 0064 calendarPreview: "Календарь событий",
 0065
 0066 catalogLabel: "Каталог",
 0067 catalogTitle: "Каталог мероприятий",
 0068 calendarLabel: "Планирование",
 0069 calendarTitle: "Календарь мероприятий",
 0070
 0071 hallsLabel: "Ресурсы площадки",
 0072 hallsTitle: "Каталог конференц-залов",
 0073 floorPlanLabel: "Навигация по зданию",
 0074 floorPlanTitle: "План этажей и залов",
 0075 legendFree: "Свободно",
 0076 legendBusy: "Занято",
 0077 legendPartial: "Частично занято",
 0078 hallSearch: "Поиск по залу, этажу или оборудованию",
 0079 hallCapacityAny: "Любая вместимость",
 0080 hallCapacitySmall: "До 50 мест",
 0081 hallCapacityMedium: "50-150 мест",
 0082 hallCapacityLarge: "150+ мест",
 0083
 0084 profileLabel: "Профиль",
 0085 profileTitle: "Личный кабинет",
 0086 profileData: "Данные участника",
 0087 myRegistrations: "Мои заявки на мероприятия",
 0088 myHallRequests: "Мои заявки на конференц-залы",
 0089

0090 adminLabel: "Администрирование",
0091 adminTitle: "Управление мероприятиями и заявками",
0092 newEvent: "Новое мероприятие",
0093 editEvent: "Редактирование мероприятия",
0094 participantRequests: "Заявки участников",
0095 exportBookings: "Экспорт заявок",
0096 filterAllRequests: "Все заявки",
0097 filterEventRequests: "На мероприятие",
0098 filterHallRequests: "На зал",
0099 filterAllStatuses: "Все статусы",
0100
0101 allCategories: "Все категории",
0102 catConference: "Конференция",
0103 catForum: "Форум",
0104 catMeetup: "Meetup",
0105 catSeminar: "Семинар",
0106 catBriefing: "Брифинг",
0107 category: "Категория",
0108
0109 allFormats: "Все форматы",
0110 formatOffline: "Офлайн",
0111 formatOnline: "Онлайн",
0112 formatHybrid: "Гибрид",
0113
0114 statusPublished: "Опубликовано",
0115 statusDraft: "Черновик",
0116 statusClosed: "Закрето",
0117
0118 requestStatusPending: "Ожидает",
0119 requestStatusApproved: "Одобрено",
0120 requestStatusRejected: "Отклонено",
0121 requestStatusCancelled: "Отменено",
0122 requestTypeEvent: "Заявка на мероприятие",
0123 requestTypeHall: "Заявка на зал",
0124
0125 saveEvent: "Сохранить",
0126 clear: "Очистить",
0127 saveProfile: "Сохранить профиль",
0128 submitRequest: "Отправить заявку",
0129 approve: "Одобрить",
0130 reject: "Отклонить",
0131 checkIn: "Отметить check-in",
0132
0133 email: "Email",
0134 password: "Пароль",
0135 passwordMin: "Пароль от 6 символов",
0136 name: "Имя",
0137 phone: "Телефон",
0138 organization: "Организация",
0139 city: "Город",
0140 search: "Поиск по названию, организатору или площадке",
0141 eventTitle: "Название мероприятия",
0142 location: "Локация",
0143 organizer: "Организатор",
0144 capacity: "Вместимость",
0145 description: "Описание",
0146 agenda: "Программа",
0147 meetingUrl: "Ссылка на подключение",
0148 price: "Цена",
0149 tags: "Теги через запятую",

```

0150 featured: "Рекомендуемое мероприятие",
0151 hallPurposePlaceholder: "Например: стратегическая сессия, отраслевой семинар",
0152
0153 date: "Дата",
0154 time: "Время",
0155 startTime: "Время начала",
0156 hallDuration: "Длительность (ч)",
0157 hallGuests: "Количество участников",
0158 hallPurpose: "Цель бронирования",
0159 hallBookingModalTitle: "Бронирование зала",
0160 hallBookingTitleWithName: "Бронирование: {{name}}",
0161 hallBookingSubtitle: "{{floor}} - {{location}} - до {{capacity}} участников",
0162 hallBookingSummary: "{{date}} - {{time}} - {{duration}} ч",
0163 hallRequestMeta: "{{attendees}} участников - {{duration}} ч",
0164 hallCapacityNumber: "{{count}} мест",
0165 hallFloor: "Этаж",
0166 hallEquipmentTitle: "Оснащение",
0167 hallDetailsEyebrow: "Конференц-зал",
0168 hallActiveRequests: "Активных заявок",
0169 hallBookButton: "Забронировать",
0170 equipment: "Оснащение",
0171 eqProjector: "Проектор",
0172 eqMic: "Микрофоны",
0173 eqWebcast: "Трансляция",
0174 eqCatering: "Кейтеринг",
0175 availToday: "Доступность на сегодня",
0176
0177 details: "Подробнее",
0178 edit: "Изменить",
0179 delete: "Удалить",
0180 book: "Записаться",

```

A.29 Frontend негізгі app логикасының басы

```

// Файл: docs/app.js
0001 const STORAGE_KEYS = {
0002   lang: "lang",
0003   demoDb: "orda-demo-db-v4",
0004   portal: "portal",
0005   currentView: "orda-current-view",
0006   controlState: "orda-control-state-v1",
0007   cabinetTab: "orda-cabinet-tab",
0008 };
0009
0010 const API_BASE_URL = String(window.ORDA_CONFIG?.API_BASE_URL || "").trim().replace(/\/+$/,
0011   "");
0012 const LOCAL_FALLBACK_ENABLED = window.ORDA_CONFIG?.USE_LOCAL_FALLBACK !==
0013   false;
0014
0015 const state = {
0016   ...window.ORDA.session.read(),
0017   lang: localStorage.getItem(STORAGE_KEYS.lang) || "ru",
0018   portal: localStorage.getItem(STORAGE_KEYS.portal) || "client",
0019   events: [],
0020   halls: [],
0021   bookings: [],
0022   hallBookings: [],
0023   adminBookings: [],
0024   availability: [],
0025   adminUsers: [],
0026   adminNotifications: [],

```

```

0025 clientNotifications: [],
0026 supportMessages: [],
0027 profile: null,
0028 dataMode: API_BASE_URL ? "remote" : "local",
0029 currentView: localStorage.getItem(STORAGE_KEYS.currentView) || "dashboard",
0030 currentTicketId: "",
0031 };
0032
0033 const els = {
0034   sidebar: document.querySelector("#sidebar"),
0035   sidebarToggle: document.querySelector("#sidebarToggle"),
0036   browseBtn: document.querySelector("#browseBtn"),
0037   portalClientBtn: document.querySelector("#portalClientBtn"),
0038   portalAdminBtn: document.querySelector("#portalAdminBtn"),
0039   themeToggle: document.querySelector("#themeToggle"),
0040   languageSelect: document.querySelector("#languageSelect"),
0041   accountName: document.querySelector("#accountName"),
0042   loginBtn: document.querySelector("#loginBtn"),
0043   logoutBtn: document.querySelector("#logoutBtn"),
0044   dataModeBadge: document.querySelector("#dataModeBadge"),
0045   authSection: document.querySelector("#authSection"),
0046   authCloseBtn: document.querySelector("#authCloseBtn"),
0047   loginForm: document.querySelector("#loginForm"),
0048   registerForm: document.querySelector("#registerForm"),
0049   profileForm: document.querySelector("#profileForm"),
0050   message: document.querySelector("#message"),
0051   seedBtn: document.querySelector("#seedBtn"),
0052   homeSearchInput: document.querySelector("#homeSearchInput"),
0053   homeSearchBtn: document.querySelector("#homeSearchBtn"),
0054   searchInput: document.querySelector("#searchInput"),
0055   categoryFilter: document.querySelector("#categoryFilter"),
0056   formatFilter: document.querySelector("#formatFilter"),
0057   hallSearchInput: document.querySelector("#hallSearchInput"),
0058   hallCapacityFilter: document.querySelector("#hallCapacityFilter"),
0059   adminRequestTypeFilter: document.querySelector("#adminRequestTypeFilter"),
0060   adminRequestStatusFilter: document.querySelector("#adminRequestStatusFilter"),
0061   eventsList: document.querySelector("#eventsList"),
0062   hallsList: document.querySelector("#hallsList"),
0063   dashboardEvents: document.querySelector("#dashboardEvents"),
0064   dashboardCalendar: document.querySelector("#dashboardCalendar"),
0065   dashboardHallLoad: document.querySelector("#dashboardHallLoad"),
0066   recentActionsList: document.querySelector("#recentActionsList"),
0067   adminNotificationsList: document.querySelector("#adminNotificationsList"),
0068   adminNotificationsFeed: document.querySelector("#adminNotificationsFeed"),
0069   calendarList: document.querySelector("#calendarList"),
0070   calendarAgendaList: document.querySelector("#calendarAgendaList"),
0071   calendarSearchInput: document.querySelector("#calendarSearchInput"),
0072   calendarTypeFilter: document.querySelector("#calendarTypeFilter"),
0073   calendarHallFilter: document.querySelector("#calendarHallFilter"),
0074   calendarStatusFilter: document.querySelector("#calendarStatusFilter"),
0075   myBookingsList: document.querySelector("#myBookingsList"),
0076   myBookingsListFull: document.querySelector("#myBookingsListFull"),
0077   myHallBookingsList: document.querySelector("#myHallBookingsList"),
0078   myNotificationsList: document.querySelector("#myNotificationsList"),
0079   supportForm: document.querySelector("#supportForm"),
0080   supportMessageText: document.querySelector("#supportMessageText"),
0081   adminBookingsList: document.querySelector("#adminBookingsList"),
0082   adminSupportList: document.querySelector("#adminSupportList"),
0083   exportBookingsBtn: document.querySelector("#exportBookingsBtn"),
0084   eventForm: document.querySelector("#eventForm"),

```

0085 eventFormTitle: document.querySelector("#eventFormTitle"),
 0086 resetEventForm: document.querySelector("#resetEventForm"),
 0087 metricEvents: document.querySelector("#metricEvents"),
 0088 metricSeats: document.querySelector("#metricSeats"),
 0089 metricBookings: document.querySelector("#metricBookings"),
 0090 metricSeatsLabel: document.querySelector("#metricSeatsLabel"),
 0091 metricBookingsLabel: document.querySelector("#metricBookingsLabel"),
 0092 metricNext: document.querySelector("#metricNext"),
 0093 heroMetricEvents: document.querySelector("#heroMetricEvents"),
 0094 heroMetricHalls: document.querySelector("#heroMetricHalls"),
 0095 heroMetricRequests: document.querySelector("#heroMetricRequests"),
 0096 adminStatEvents: document.querySelector("#adminStatEvents"),
 0097 adminStatBookings: document.querySelector("#adminStatBookings"),
 0098 adminStatUsers: document.querySelector("#adminStatUsers"),
 0099 adminStatCheckedIn: document.querySelector("#adminStatCheckedIn"),
 0100 adminStatPaidTickets: document.querySelector("#adminStatPaidTickets"),
 0101 adminStatHallApproved: document.querySelector("#adminStatHallApproved"),
 0102 eventModal: document.querySelector("#eventModal"),
 0103 closeModal: document.querySelector("#closeModal"),
 0104 eventDetails: document.querySelector("#eventDetails"),
 0105 hallModal: document.querySelector("#hallModal"),
 0106 closeHallModal: document.querySelector("#closeHallModal"),
 0107 cancelHallBooking: document.querySelector("#cancelHallBooking"),
 0108 hallBookingForm: document.querySelector("#hallBookingForm"),
 0109 hallBookingTitle: document.querySelector("#hallBookingTitle"),
 0110 hallBookingSubtitle: document.querySelector("#hallBookingSubtitle"),
 0111 hallForm: document.querySelector("#hallForm"),
 0112 hallFormTitle: document.querySelector("#hallFormTitle"),
 0113 resetHallForm: document.querySelector("#resetHallForm"),
 0114 adminHallsList: document.querySelector("#adminHallsList"),
 0115 adminUsersList: document.querySelector("#adminUsersList"),
 0116 ticketModal: document.querySelector("#ticketModal"),
 0117 closeTicketModal: document.querySelector("#closeTicketModal"),
 0118 downloadTicketBtn: document.querySelector("#downloadTicketBtn"),
 0119 paymentModal: document.querySelector("#paymentModal"),
 0120 closePaymentModal: document.querySelector("#closePaymentModal"),
 0121 cancelPayment: document.querySelector("#cancelPayment"),
 0122 paymentForm: document.querySelector("#paymentForm"),
 0123 paymentEventTitle: document.querySelector("#paymentEventTitle"),
 0124 paymentAmount: document.querySelector("#paymentAmount"),
 0125 payLaterBtn: document.querySelector("#payLaterBtn"),
 0126 payCardFields: document.querySelector("#payCardFields"),
 0127 payKaspiFields: document.querySelector("#payKaspiFields"),
 0128 payWalletFields: document.querySelector("#payWalletFields"),
 0129 paymentMethodPreview: document.querySelector("#paymentMethodPreview"),
 0130 paymentOrderStatus: document.querySelector("#paymentOrderStatus"),
 0131 paymentProcessText: document.querySelector("#paymentProcessText"),
 0132 adminRequestModal: document.querySelector("#adminRequestModal"),
 0133 closeAdminRequestModal: document.querySelector("#closeAdminRequestModal"),
 0134 cancelAdminRequestEdit: document.querySelector("#cancelAdminRequestEdit"),
 0135 adminRequestForm: document.querySelector("#adminRequestForm"),
 0136 requestHallGrid: document.querySelector("#requestHallGrid"),
 0137 requestHallPlanner: document.querySelector("#requestHallPlanner"),
 0138 ownEventFields: document.querySelector("#ownEventFields"),
 0139 requestDate: document.querySelector("#requestDate"),
 0140 requestSlotGrid: document.querySelector("#requestSlotGrid"),
 0141 requestSlotModal: document.querySelector("#requestSlotModal"),
 0142 closeRequestSlotModal: document.querySelector("#closeRequestSlotModal"),
 0143 cancelRequestSlotModal: document.querySelector("#cancelRequestSlotModal"),
 0144 confirmRequestSlot: document.querySelector("#confirmRequestSlot"),

```

0145 requestSlotModalHallName: document.querySelector("#requestSlotModalHallName"),
0146 requestSlotModalMessage: document.querySelector("#requestSlotModalMessage"),
0147 requestSelectedSummary: document.querySelector("#requestSelectedSummary"),
0148 requestRentPreview: document.querySelector("#requestRentPreview"),
0149 customRequestForm: document.querySelector("#customRequestForm"),
0150 requestServicesSection: document.querySelector("#requestServicesSection"),
0151 requestTotalPreview: document.querySelector("#requestTotalPreview"),
0152 requestPaymentModal: document.querySelector("#requestPaymentModal"),
0153 closeRequestPaymentModal: document.querySelector("#closeRequestPaymentModal"),
0154 cancelRequestPayment: document.querySelector("#cancelRequestPayment"),
0155 requestPaymentSummary: document.querySelector("#requestPaymentSummary"),
0156 requestPaymentForm: document.querySelector("#requestPaymentForm"),
0157 requestAnnouncementPayment: document.querySelector("#requestAnnouncementPayment"),
0158 requestCardPaymentFields: document.querySelector("#requestCardPaymentFields"),
0159 requestPaymentProcessText: document.querySelector("#requestPaymentProcessText"),
0160 requestImageFile: document.querySelector("#requestImageFile"),
0161 requestImagePreview: document.querySelector("#requestImagePreview"),
0162 clientWorkspace: document.querySelector("#clientWorkspace"),
0163 adminWorkspace: document.querySelector("#adminWorkspace"),
0164 eventsCount: document.querySelector("#eventsCount"),
0165 sidebarBackdrop: document.querySelector("#sidebarBackdrop"),
0166 portalSwitcher: document.querySelector("#portalSwitcher"),
0167 };
0168
0169 const { categoryKeys, formatKeys } = window.ORDA.domain;
0170 const requestSlots = [
0171   { id: "morning", start: "09:00", end: "11:00" },
0172   { id: "midday", start: "12:00", end: "14:00" },
0173   { id: "afternoon", start: "15:00", end: "18:00" },
0174 ];
0175 const requestServiceConfig = [
0176   { id: "registration", key: "serviceRegistration", price: 9000 },
0177   { id: "assistants", key: "serviceAssistants", price: 22000 },
0178   { id: "tech", key: "serviceTech", price: 18000 },
0179   { id: "projector", key: "serviceProjector", price: 10000 },
0180   { id: "microphones", key: "serviceMicrophones", price: 8000 },
0181   { id: "audio", key: "serviceAudio", price: 12000 },
0182   { id: "led", key: "serviceLedScreen", price: 16000 },
0183   { id: "coffee", key: "serviceCoffee", price: 25000 },
0184   { id: "badges", key: "badgeLabel", price: 6000 },
0185   { id: "security", key: "serviceSecurity", price: 30000 },
0186   { id: "cleaning", key: "serviceCleaning", price: 12000 },
0187   { id: "streaming", key: "serviceStreaming", price: 35000 },
0188   { id: "translator", key: "serviceTranslator", price: 28000 },
0189 ];
0190 const equipmentLabelKeys = {
0191   projector: "eqProjector",
0192   mic: "eqMic",
0193   microphones: "serviceMicrophones",
0194   webcast: "eqWebcast",
0195   catering: "eqCatering",
0196   led: "serviceTv",
0197   wifi: "serviceWifi",
0198   tech: "serviceTech",
0199 };
0200 const announcementRequestPrice = 10000;
0201 const hallImageFallbacks = {
0202   h_grand: "https://images.unsplash.com/photo-1540575467063-178a50c2df87?auto=format&fit=crop&w=1200&q=80",

```

```

0203 h_orion: "https://images.unsplash.com/photo-1511578314322-
379afb476865?auto=format&fit=crop&w=1200&q
=80",
0204 h_workshop: "https://images.unsplash.com/photo-1497366811353-
6870744d04b2?auto=format&fit=crop&w=120
0&q=80",
0205 h_board: "https://images.unsplash.com/photo-1431540015161-
0bf868a2d407?auto=format&fit=crop&w=1200&q
=80",
0206 h_atrium: "https://images.unsplash.com/photo-1505373877841-
8d25f7d46678?auto=format&fit=crop&w=1200&
q=80",
0207 h_silk: "https://images.unsplash.com/photo-1517502884422-
41eaead166d4?auto=format&fit=crop&w=1200&q=
80",
0208 h_nova: "https://images.unsplash.com/photo-1521737604893-
d14cc237f11d?auto=format&fit=crop&w=1200&q=
80",
0209 h_sky: "https://images.unsplash.com/photo-1518005020951-
eccb494ad742?auto=format&fit=crop&w=1200&q=8
0",
0210 "Grand Hall A": "https://images.unsplash.com/photo-1540575467063-
178a50c2df87?auto=format&fit=crop&w
=1200&q=80",
0211 "Orion Hall B": "https://images.unsplash.com/photo-1511578314322-
379afb476865?auto=format&fit=crop&w
=1200&q=80",
0212 "Workshop Hub C": "https://images.unsplash.com/photo-1497366811353-
6870744d04b2?auto=format&fit=crop
&w=1200&q=80",
0213 "Boardroom D": "https://images.unsplash.com/photo-1431540015161-
0bf868a2d407?auto=format&fit=crop&w=
1200&q=80",
0214 "Atrium Expo Space": "https://images.unsplash.com/photo-1505373877841-
8d25f7d46678?auto=format&fit=c
rop&w=1200&q=80",
0215 "Silk Road Conference Room": "https://images.unsplash.com/photo-1517502884422-
41eaead166d4?auto=form
at&fit=crop&w=1200&q=80",
0216 "Nova Training Lab": "https://images.unsplash.com/photo-1521737604893-
d14cc237f11d?auto=format&fit=c
rop&w=1200&q=80",
0217 "Skyline Networking Lounge": "https://images.unsplash.com/photo-1518005020951-
eccb494ad742?auto=form
at&fit=crop&w=1200&q=80",
0218 default: "https://images.unsplash.com/photo-1517245386807-
bb43f82c33c4?auto=format&fit=crop&w=1200&q
=80",
0219 };
0220
0221 const demoHallTranslations = {
0222   h_grand: {
0223     ru: {
0224       name: "Grand Hall A",
0225       floor: "1 этаж",
0226       location: "Центральное крыло, район Astana Expo",
0227       description: "Флагманский зал для форумов, выставочных открытий и keynote-сессий с
профессиональным светом, сценой и синхронным переводом.",
0228       equipment: ["LED-экран 12 м", "Сценический свет", "Кабины перевода", "Пульт трансляции",
"VIP-вход"],

```

0229 advantages: ["Панорамная сцена", "Отдельная регистрация", "Потоковая трансляция"],
0230 },
0231 kk: {
0232 name: "Grand Hall A",
0233 floor: "1-қабат",
0234 location: "Орталық қанат, Astana Expo ауданы",
0235 description: "Форумдар, көрме ашылымдары және keynote-сессиялар үшін кәсіби жарығы, сахнасы және синхронды аудармасы бар басты зал.",
0236 equipment: ["12 м LED-экран", "Сахналық жарық", "Аударма кабиналары", "Трансляция пульті", "VIP-кіру"],
0237 advantages: ["Панорамалық сахна", "Жеке тіркеу аймағы", "Ағынды трансляция"],
0238 },
0239 en: {
0240 name: "Grand Hall A",
0241 floor: "Floor 1",
0242 location: "Central Wing, Astana Expo District",
0243 description: "A flagship hall for forums, exhibition openings, and keynote sessions with professional lighting, a stage, and simultaneous interpretation.",
0244 equipment: ["12 m LED wall", "Stage lighting", "Translation booths", "Broadcast desk", "VIP entrance"],
0245 advantages: ["Panoramic stage", "Separate registration", "Live streaming"],
0246 },
0247 },
0248 h_orion: {
0249 ru: {
0250 name: "Orion Hall B",
0251 floor: "2 этаж",
0252 location: "Северное крыло",
0253 description: "Универсальный зал для корпоративных конференций, продуктовых презентаций и гибридных программ.",
0254 equipment: ["4K-проектор", "Zoom room", "Акустическая система", "Видеостена", "Гибридный комплект"],
0255 advantages: ["Гибридный формат", "Сильная акустика", "Быстрая рассадка"],
0256 },
0257 kk: {
0258 name: "Orion Hall B",
0259 floor: "2-қабат",
0260 location: "Солтүстік қанат",
0261 description: "Корпоративтік конференциялар, өнім презентациялары және гибриді бағдарламаларға арналған әмбебап зал.",
0262 equipment: ["4K-проектор", "Zoom room", "Акустикалық жүйе", "Бейнекабырға", "Гибриді жинақ"],
0263 advantages: ["Гибриді формат", "Күшті акустика", "Жылдам отырғызу"],
0264 },
0265 en: {
0266 name: "Orion Hall B",
0267 floor: "Floor 2",
0268 location: "North Wing",
0269 description: "A flexible hall for corporate conferences, product presentations, and hybrid programs.",
0270 equipment: ["4K projector", "Zoom room", "Acoustic system", "Video wall", "Hybrid kit"],
0271 advantages: ["Hybrid format", "Strong acoustics", "Fast seating setup"],
0272 },
0273 },
0274 h_workshop: {
0275 ru: {
0276 name: "Workshop Hub C",
0277 floor: "2 этаж",

0278 location: "Южное крыло",
0279 description: "Гибкое пространство для тренингов, дизайн-сессий, образовательных интенсивов и командных workshop-программ.",
0280 equipment: ["Модульная мебель", "Флипчарты", "VR-комплект", "Интерактивные панели"],
0281 advantages: ["Модульная мебель", "Интерактивные панели", "Командные зоны"],
0282 },
0283 kk: {
0284 name: "Workshop Hub C",
0285 floor: "2-қабат",
0286 location: "Оңтүстік қанат",
0287 description: "Тренингтер, дизайн-сессиялар, білім беру интенсивтері және командалық workshop бағдарламаларына арналған икемді кеңістік.",
0288 equipment: ["Модульдік жиһаз", "Флипчарттар", "VR-жинақ", "Интерактивті панельдер"],
0289 advantages: ["Модульдік жиһаз", "Интерактивті панельдер", "Командалық аймақтар"],
0290 },
0291 en: {
0292 name: "Workshop Hub C",
0293 floor: "Floor 2",
0294 location: "South Wing",
0295 description: "A flexible space for trainings, design sessions, educational intensives, and team workshop programs.",
0296 equipment: ["Modular furniture", "Flipcharts", "VR kit", "Interactive panels"],
0297 advantages: ["Modular furniture", "Interactive panels", "Team zones"],
0298 },
0299 },
0300 h_board: {
0301 ru: {
0302 name: "Boardroom D",
0303 floor: "3 этаж",
0304 location: "Executive Zone",
0305 description: "Закрытая переговорная для совещаний руководителей, стратегических встреч и private briefing-сессий.",
0306 equipment: ["Видеоконференция", "Smart display", "Кофе-станция", "Приватное лобби"],
0307 advantages: ["Приватность", "Executive-сервис", "Видеосвязь"],
0308 },
0309 kk: {
0310 name: "Boardroom D",
0311 floor: "3-қабат",
0312 location: "Executive Zone",
0313 description: "Басшылар жиналыстары, стратегиялық кездесулер және private briefing-сессиялар үшін жабық келіссөз бөлмесі.",
0314 equipment: ["Бейнебайланыс", "Smart display", "Кофе станциясы", "Жеке лобби"],
0315 advantages: ["Құпиялылық", "Executive-сервис", "Бейнебайланыс"],
0316 },
0317 en: {
0318 name: "Boardroom D",
0319 floor: "Floor 3",
0320 location: "Executive Zone",